Jupyter Notebook File Evaluation of Football Players

April 8, 2024

[12]:
```
## Evaluation of Football Players
## DATA COLLECTION AND PREPARATION, Feature Engineering, Feature Selection &
 ↪Model training.
```

[3]:
```
import pandas as pd
import re
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

[4]:
```
## The dataset contains data from the website sofifa.com which provides
#  all information concerning soccer players from major leagues across the world
#  The dataset was downloaded from the website www.kaggle.com
```

[5]:
```
df = pd.read_csv('C:\\Users\\kweku\\Desktop\\Dataset\\players.csv')
pd.set_option('display.max_columns', 75)
pd.set_option('display.max_rows', 200)
```

[6]:
```
## Information & data columns of the dataset
```

[7]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 28371 entries, 0 to 28370
Data columns (total 23 columns):
 #   Column                                Non-Null Count  Dtype
---  ------                                --------------  -----
 0   player_id                             28371 non-null  int64
 1   name                                  28371 non-null  object
 2   current_club_id                       28371 non-null  int64
 3   current_club_name                     28371 non-null  object
 4   country_of_citizenship                28370 non-null  object
 5   country_of_birth                      26496 non-null  object
 6   city_of_birth                         26509 non-null  object
 7   date_of_birth                         28336 non-null  object
 8   position                              28371 non-null  object
 9   sub_position                          24935 non-null  object
 10  foot                                  26162 non-null  object
```

```
 11  height_in_cm                          28371 non-null  int64
 12  market_value_in_eur                   19418 non-null  float64
 13  highest_market_value_in_eur           27309 non-null  float64
 14  agent_name                            17349 non-null  object
 15  contract_expiration_date              17034 non-null  object
 16  current_club_domestic_competition_id  28371 non-null  object
 17  first_name                            26610 non-null  object
 18  last_name                             28371 non-null  object
 19  player_code                           28371 non-null  object
 20  image_url                             28371 non-null  object
 21  last_season                           28371 non-null  int64
 22  url                                   28371 non-null  object
dtypes: float64(2), int64(4), object(17)
memory usage: 5.0+ MB
```

[8]:
```
## The first 5 rows and columns of the dataset
```

[9]:
```
df.head()
```

[9]:

	player_id	name	current_club_id	current_club_name	country_of_citizenship	country_of_birth	city_of_birth	date_of_birth	position	sub_position	foot	height_in_cm	market_value_in_eur	highest_market_value_in_eur	agent_name	contract_expiration_date
0	134354	Ian Raeymaekers	498	Ksc Lokeren	Belgium	Belgium	Aalst	1995-01-30	Attack	Centre-Forward	Right	0	NaN	50000.0	NaN	NaN
1	99946	Mohamed Camara	1095	Es Troyes Ac	Guinea	Guinea	Conakry	1990-09-20	Attack	Centre-Forward	Right	180	NaN	300000.0	NaN	NaN
2	76948	Pablo Olivera	979	Moreirense Fc	Uruguay	Uruguay	Melo	1987-12-08	Attack	Centre-Forward	Right	175	25000.0	600000.0	NaN	NaN
3	108372	Aliosman Aydin	38	Fortuna Dusseldorf	Turkey	Germany	Dormagen	1992-02-06	Attack	Centre-Forward	Right	178	NaN	125000.0	NaN	NaN
4	78820	Jaime Alfonso Ruiz	354	Kv Mechelen	Colombia	Colombia	Cali	1984-01-03	Attack	Centre-Forward	Right	184	NaN	1700000.0	NaN	NaN

```
  current_club_domestic_competition_id     first_name    last_name  \
0                                  BE1            Ian  Raeymaekers
1                                  FR1        Mohamed       Camara
2                                  PO1          Pablo      Olivera
3                                   L1       Aliosman        Aydin
4                                  BE1  Jaime Alfonso         Ruiz

          player_code                                          image_url  \
0     ian-raeymaekers  https://img.a.transfermarkt.technology/portrai…
1      mohamed-camara  https://img.a.transfermarkt.technology/portrai…
2       pablo-olivera  https://img.a.transfermarkt.technology/portrai…
3      aliosman-aydin  https://img.a.transfermarkt.technology/portrai…
4  jaime-alfonso-ruiz  https://img.a.transfermarkt.technology/portrai…

   last_season                                                url
0         2012  https://www.transfermarkt.co.uk/ian-raeymaeker…
1         2012  https://www.transfermarkt.co.uk/mohamed-camara…
2         2012  https://www.transfermarkt.co.uk/pablo-olivera/…
3         2012  https://www.transfermarkt.co.uk/aliosman-aydin…
4         2012  https://www.transfermarkt.co.uk/jaime-alfonso-…
```

[10]:
```
## The descriptive statistics of the dataset
```

[11]:
```
df.describe()
```

[11]:
```
          player_id  current_club_id  height_in_cm  market_value_in_eur  \
count  2.837100e+04     28371.000000  28371.000000         1.941800e+04
mean   2.829145e+05      4038.466815    170.626238         2.073996e+06
std    2.248316e+05      9073.398714     44.867902         6.804635e+06
min    1.000000e+01         3.000000      0.000000         1.000000e+04
25%    8.734000e+04       398.000000    176.000000         1.500000e+05
50%    2.389810e+05      1063.000000    182.000000         3.500000e+05
75%    4.194370e+05      2999.000000    187.000000         1.000000e+06
max    1.059630e+06     83678.000000    206.000000         1.800000e+08

       highest_market_value_in_eur   last_season
count                 2.730900e+04  28371.000000
mean                  3.520911e+06   2018.213175
std                   9.131342e+06      3.347518
min                   1.000000e+04   2012.000000
25%                   2.750000e+05   2015.000000
50%                   8.000000e+05   2019.000000
75%                   2.750000e+06   2021.000000
max                   2.000000e+08   2022.000000
```

[12]:
```
## This data encompasses various attributes, including but not limited to␣
 ↪"Player_id" , "Name" ,"Current_Club_id" , ""Current_Club_name" ,
```

```
## "Country_of_citizenship", "Country_of_birth", "City_of_birth",␣
 ↪"Date_of_birth", "Position", "Sub_position", "Foot", "Heght","Market_Value"
```

[13]:
```
df
```

[13]:
```
       player_id                 name  current_club_id    current_club_name  \
0         134354      Ian Raeymaekers              498          Ksc Lokeren
1          99946       Mohamed Camara             1095         Es Troyes Ac
2          76948        Pablo Olivera              979        Moreirense Fc
3         108372       Aliosman Aydin               38   Fortuna Dusseldorf
4          78820   Jaime Alfonso Ruiz              354          Kv Mechelen
…            …                    …                …                    …
28366     452068          Ayman Azhil               15  Bayer 04 Leverkusen
28367      85543          Niklas Lomb               15  Bayer 04 Leverkusen
28368     315131  Timothy Fosu-Mensah               15  Bayer 04 Leverkusen
28369      39728         Mats Hummels               16    Borussia Dortmund
28370     659813       Piero Hincapié               15  Bayer 04 Leverkusen

      country_of_citizenship country_of_birth        city_of_birth  \
0                    Belgium          Belgium                Aalst
1                     Guinea           Guinea               Conakry
2                    Uruguay          Uruguay                 Melo
3                     Turkey          Germany              Dormagen
4                   Colombia         Colombia                 Cali
…                         …                …                    …
28366                Morocco          Germany           Düsseldorf
28367                Germany          Germany                 Köln
28368            Netherlands      Netherlands             Amsterdam
28369                Germany          Germany  Bergisch Gladbach
28370                Ecuador          Ecuador           Esmeraldas

      date_of_birth    position        sub_position   foot  height_in_cm  \
0        1995-01-30      Attack      Centre-Forward  Right             0
1        1990-09-20      Attack      Centre-Forward  Right           180
2        1987-12-08      Attack      Centre-Forward  Right           175
3        1992-02-06      Attack      Centre-Forward  Right           178
4        1984-01-03      Attack      Centre-Forward  Right           184
…               …           …                   …      …            …
28366    2001-04-10    Midfield  Defensive Midfield    NaN           175
28367    1993-07-28  Goalkeeper                 NaN  Right           186
28368    1998-01-02    Defender          Right-Back  Right           185
28369    1988-12-16    Defender         Centre-Back  Right           191
28370    2002-01-09    Defender         Centre-Back   Left           183

       market_value_in_eur  highest_market_value_in_eur           agent_name  \
0                      NaN                      50000.0                  NaN
1                      NaN                     300000.0                  NaN
```

```
2                  25000.0                    600000.0                  NaN
3                      NaN                    125000.0                  NaN
4                      NaN                   1700000.0                  NaN
…                      …                          …                      …
28366             400000.0                    400000.0         KL Sportsbase
28367             250000.0                    400000.0            Dirk Hebel
28368            3000000.0                   7000000.0          CAA Base Ltd
28369            6500000.0                  60000000.0   HMH Sportmanagement
28370           25000000.0                  25000000.0       Fútbol División

      contract_expiration_date current_club_domestic_competition_id  \
0                          NaN                                  BE1
1                          NaN                                  FR1
2                          NaN                                  PO1
3                          NaN                                   L1
4                          NaN                                  BE1
…                          …                                    …
28366                2023-06-30                                   L1
28367                2024-06-30                                   L1
28368                2024-06-30                                   L1
28369                2023-06-30                                   L1
28370                2026-06-30                                   L1

          first_name    last_name          player_code  \
0                Ian  Raeymaekers      ian-raeymaekers
1           Mohamed       Camara       mohamed-camara
2             Pablo      Olivera        pablo-olivera
3           Aliosman        Aydin       aliosman-aydin
4      Jaime Alfonso         Ruiz   jaime-alfonso-ruiz
…                 …            …                    …
28366          Ayman        Azhil          ayman-azhil
28367         Niklas         Lomb          niklas-lomb
28368        Timothy  Fosu-Mensah  timothy-fosu-mensah
28369           Mats      Hummels         mats-hummels
28370          Piero     Hincapié       piero-hincapie

                                                image_url  last_season  \
0      https://img.a.transfermarkt.technology/portrai…         2012
1      https://img.a.transfermarkt.technology/portrai…         2012
2      https://img.a.transfermarkt.technology/portrai…         2012
3      https://img.a.transfermarkt.technology/portrai…         2012
4      https://img.a.transfermarkt.technology/portrai…         2012
…                                                    …            …
28366  https://img.a.transfermarkt.technology/portrai…         2022
28367  https://img.a.transfermarkt.technology/portrai…         2022
28368  https://img.a.transfermarkt.technology/portrai…         2022
28369  https://img.a.transfermarkt.technology/portrai…         2022
```

```
28370  https://img.a.transfermarkt.technology/portrai…         2022

                                                      url
0      https://www.transfermarkt.co.uk/ian-raeymaeker…
1      https://www.transfermarkt.co.uk/mohamed-camara…
2      https://www.transfermarkt.co.uk/pablo-olivera/…
3      https://www.transfermarkt.co.uk/aliosman-aydin…
4      https://www.transfermarkt.co.uk/jaime-alfonso-…
…                                                    …
28366  https://www.transfermarkt.co.uk/ayman-azhil/pr…
28367  https://www.transfermarkt.co.uk/niklas-lomb/pr…
28368  https://www.transfermarkt.co.uk/timothy-fosu-m…
28369  https://www.transfermarkt.co.uk/mats-hummels/p…
28370  https://www.transfermarkt.co.uk/piero-hincapie…

[28371 rows x 23 columns]
```

[14]:
```
## CLEANING THE DATASET
## Checking for any missing values in the dataset
```

[15]:
```
df.isnull().sum()
```

[15]:
```
player_id                                0
name                                     0
current_club_id                          0
current_club_name                        0
country_of_citizenship                   1
country_of_birth                      1875
city_of_birth                         1862
date_of_birth                           35
position                                 0
sub_position                          3436
foot                                  2209
height_in_cm                             0
market_value_in_eur                   8953
highest_market_value_in_eur           1062
agent_name                           11022
contract_expiration_date             11337
current_club_domestic_competition_id     0
first_name                            1761
last_name                                0
player_code                              0
image_url                                0
last_season                              0
url                                      0
dtype: int64
```

[16]:
```
# Filling missing values for numerical columns with modes
numerical_columns = df.select_dtypes(include=['float64', 'int64']).columns
df[numerical_columns] = df[numerical_columns].fillna(df[numerical_columns].
 ↪mode().iloc[0])

# Filling missing values for categorical columns with the most frequent values
categorical_columns = df.select_dtypes(include=['object']).columns
df[categorical_columns] = df[categorical_columns].apply(lambda x: x.fillna(x.
 ↪value_counts().idxmax()))

# Example: Filling missing values for date_of_birth with a placeholder date
df['date_of_birth'].fillna('1996-01-01', inplace=True)

# Display the DataFrame with filled missing values
df.head()
```

[16]:

	player_id	name	current_club_id	current_club_name	\
0	134354	Ian Raeymaekers	498	Ksc Lokeren	
1	99946	Mohamed Camara	1095	Es Troyes Ac	
2	76948	Pablo Olivera	979	Moreirense Fc	
3	108372	Aliosman Aydin	38	Fortuna Dusseldorf	
4	78820	Jaime Alfonso Ruiz	354	Kv Mechelen	

	country_of_citizenship	country_of_birth	city_of_birth	date_of_birth	\
0	Belgium	Belgium	Aalst	1995-01-30	
1	Guinea	Guinea	Conakry	1990-09-20	
2	Uruguay	Uruguay	Melo	1987-12-08	
3	Turkey	Germany	Dormagen	1992-02-06	
4	Colombia	Colombia	Cali	1984-01-03	

	position	sub_position	foot	height_in_cm	market_value_in_eur	\
0	Attack	Centre-Forward	Right	0	200000.0	
1	Attack	Centre-Forward	Right	180	200000.0	
2	Attack	Centre-Forward	Right	175	25000.0	
3	Attack	Centre-Forward	Right	178	200000.0	
4	Attack	Centre-Forward	Right	184	200000.0	

	highest_market_value_in_eur	agent_name	contract_expiration_date	\
0	50000.0	Wasserman	2023-06-30	
1	300000.0	Wasserman	2023-06-30	
2	600000.0	Wasserman	2023-06-30	
3	125000.0	Wasserman	2023-06-30	
4	1700000.0	Wasserman	2023-06-30	

	current_club_domestic_competition_id	first_name	last_name	\
0	BE1	Ian	Raeymaekers	
1	FR1	Mohamed	Camara	

```
2                                PO1          Pablo      Olivera
3                                 L1       Aliosman        Aydin
4                                BE1  Jaime Alfonso         Ruiz

          player_code                                          image_url  \
0     ian-raeymaekers  https://img.a.transfermarkt.technology/portrai…
1      mohamed-camara  https://img.a.transfermarkt.technology/portrai…
2       pablo-olivera  https://img.a.transfermarkt.technology/portrai…
3      aliosman-aydin  https://img.a.transfermarkt.technology/portrai…
4  jaime-alfonso-ruiz  https://img.a.transfermarkt.technology/portrai…

   last_season                                                 url
0         2012  https://www.transfermarkt.co.uk/ian-raeymaeker…
1         2012  https://www.transfermarkt.co.uk/mohamed-camara…
2         2012  https://www.transfermarkt.co.uk/pablo-olivera/…
3         2012  https://www.transfermarkt.co.uk/aliosman-aydin…
4         2012  https://www.transfermarkt.co.uk/jaime-alfonso-…
```

[17]:
```
## There are no missing values after cleaning of the data
```

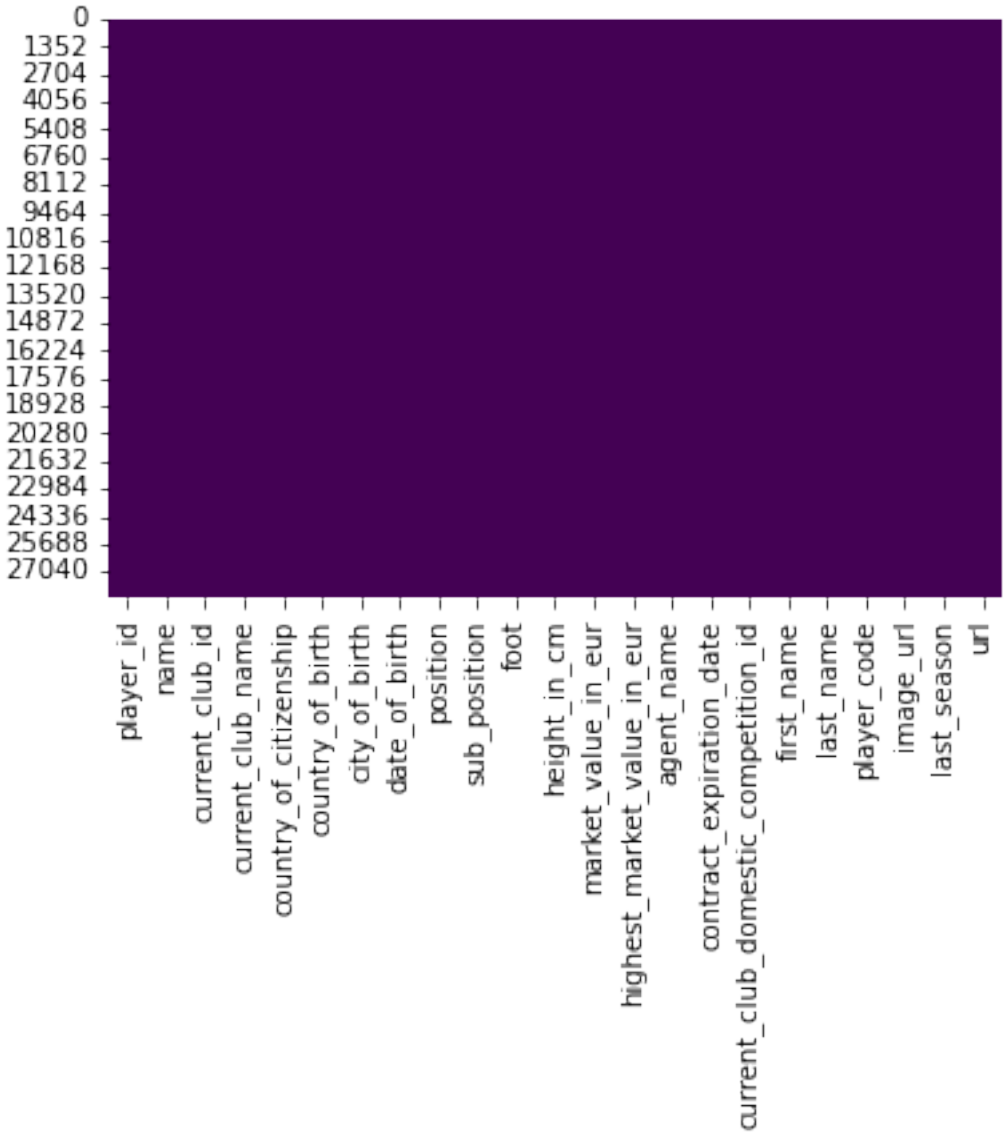
[18]:
```
df.isnull().sum()
```

[18]:
```
player_id                          0
name                               0
current_club_id                    0
current_club_name                  0
country_of_citizenship             0
country_of_birth                   0
city_of_birth                      0
date_of_birth                      0
position                           0
sub_position                       0
foot                               0
height_in_cm                       0
market_value_in_eur                0
highest_market_value_in_eur        0
agent_name                         0
contract_expiration_date           0
current_club_domestic_competition_id    0
first_name                         0
last_name                          0
player_code                        0
image_url                          0
last_season                        0
url                                0
dtype: int64
```

[19]:
```
## Heat map vosualization of missing values
```

[20]:
```
sns.heatmap(df.isnull(), cmap='viridis', cbar=False)
```

[20]: <AxesSubplot:>



[21]:
```
## WHAT ARE THE MAJOR ATTRIBUTES TO THE PLAYERS MARKET VALUE?
#  The major attributes that affect a players market value are
#  Age, Position, Rating & Potential, Workrate, Club, Reputation and others.
#  We are will find relationships and patterns between these attributes and␣
 ↪their market value.
```

[22]:
```
# Positions
```

[23]:
```
positions = df['position'].unique()
print("List of Positions:")
for position in positions:
    print(position)
```

```
List of Positions:
Attack
Midfield
Defender
Goalkeeper
```

[24]:
```
## Highest Valued Players and their positions
```

[25]:
```
df.nlargest(10,columns='highest_market_value_in_eur')
```

[25]:

	player_id	name	current_club_id	current_club_name	\
25705	342229	Kylian Mbappé	583	Fc Paris Saint Germain	
25694	28003	Lionel Messi	583	Fc Paris Saint Germain	
25707	68290	Neymar	583	Fc Paris Saint Germain	
23707	418560	Erling Haaland	281	Manchester City	
24465	134425	Raheem Sterling	631	Fc Chelsea	
22022	200512	Sadio Mané	27	Fc Bayern Munchen	
23688	148455	Mohamed Salah	31	Fc Liverpool	
23689	132098	Harry Kane	148	Tottenham Hotspur	
23711	88755	Kevin De Bruyne	281	Manchester City	
23956	80444	Philippe Coutinho	405	Aston Villa	

	country_of_citizenship	country_of_birth	city_of_birth	\
25705	France	France	Bondy	
25694	Argentina	Argentina	Rosario	
25707	Brazil	Brazil	Mogi das Cruzes	
23707	Norway	England	Leeds	
24465	England	Jamaica	Kingston	
22022	Senegal	Senegal	Bambaly	
23688	Egypt	Egypt	Nagrig, Basyoun	
23689	England	England	London	
23711	Belgium	Belgium	Drongen	
23956	Brazil	Brazil	Rio de Janeiro	

	date_of_birth	position	sub_position	foot	height_in_cm	\
25705	1998-12-20	Attack	Centre-Forward	Right	178	
25694	1987-06-24	Attack	Right Winger	Left	170	
25707	1992-02-05	Attack	Left Winger	Right	175	
23707	2000-07-21	Attack	Centre-Forward	Left	195	
24465	1994-12-08	Attack	Left Winger	Right	170	
22022	1992-04-10	Attack	Left Winger	Right	174	

```
23688    1992-06-15   Attack        Right Winger   Left           175
23689    1993-07-28   Attack      Centre-Forward  Right           188
23711    1991-06-28   Attack  Attacking Midfield  Right           181
23956    1992-06-12   Attack         Left Winger  Right           172

       market_value_in_eur  highest_market_value_in_eur            agent_name  \
25705          180000000.0                200000000.0             Wasserman
25694           50000000.0                180000000.0             Wasserman
25707           75000000.0                180000000.0             Wasserman
23707          170000000.0                170000000.0       Rafaela Pimenta
24465           70000000.0                160000000.0             Wasserman
22022           60000000.0                150000000.0                  ROOF
23688           80000000.0                150000000.0             Wasserman
23689           90000000.0                150000000.0                  CK66
23711           80000000.0                150000000.0             Wasserman
23956           18000000.0                150000000.0  Sports Invest UK ltd

      contract_expiration_date current_club_domestic_competition_id  \
25705               2025-06-30                                  FR1
25694               2023-06-30                                  FR1
25707               2025-06-30                                  FR1
23707               2027-06-30                                  GB1
24465               2027-06-30                                  GB1
22022               2025-06-30                                   L1
23688               2025-06-30                                  GB1
23689               2024-06-30                                  GB1
23711               2025-06-30                                  GB1
23956               2026-06-30                                  GB1

      first_name      last_name        player_code  \
25705     Kylian         Mbappé      kylian-mbappe
25694     Lionel          Messi       lionel-messi
25707      David         Neymar             neymar
23707     Erling        Haaland     erling-haaland
24465     Raheem       Sterling    raheem-sterling
22022      Sadio           Mané         sadio-mane
23688      David  Mohamed Salah      mohamed-salah
23689      Harry           Kane         harry-kane
23711      Kevin      De Bruyne    kevin-de-bruyne
23956   Philippe       Coutinho  philippe-coutinho

                                              image_url  last_season  \
25705  https://img.a.transfermarkt.technology/portrai…         2022
25694  https://img.a.transfermarkt.technology/portrai…         2022
25707  https://img.a.transfermarkt.technology/portrai…         2022
23707  https://img.a.transfermarkt.technology/portrai…         2022
24465  https://img.a.transfermarkt.technology/portrai…         2022
```

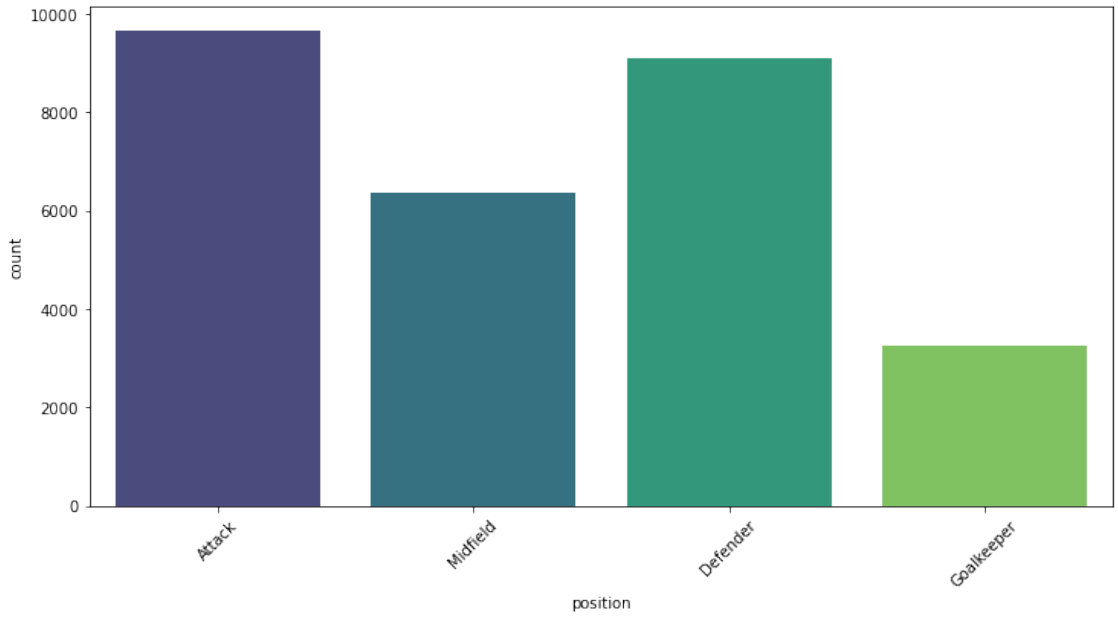
```
22022  https://img.a.transfermarkt.technology/portrai…        2022
23688  https://img.a.transfermarkt.technology/portrai…        2022
23689  https://img.a.transfermarkt.technology/portrai…        2022
23711  https://img.a.transfermarkt.technology/portrai…        2022
23956  https://img.a.transfermarkt.technology/portrai…        2022
```

```
                                                        url
25705  https://www.transfermarkt.co.uk/kylian-mbappe/…
25694  https://www.transfermarkt.co.uk/lionel-messi/p…
25707  https://www.transfermarkt.co.uk/neymar/profil/…
23707  https://www.transfermarkt.co.uk/erling-haaland…
24465  https://www.transfermarkt.co.uk/raheem-sterlin…
22022  https://www.transfermarkt.co.uk/sadio-mane/pro…
23688  https://www.transfermarkt.co.uk/mohamed-salah/…
23689  https://www.transfermarkt.co.uk/harry-kane/pro…
23711  https://www.transfermarkt.co.uk/kevin-de-bruyn…
23956  https://www.transfermarkt.co.uk/philippe-couti…
```

[26]:
```
## Player position count plot visualization
```

[27]:
```
plt.figure(figsize=(12, 6))
sns.countplot(x='position', data=df, palette='viridis')
plt.xticks(rotation=45)
```

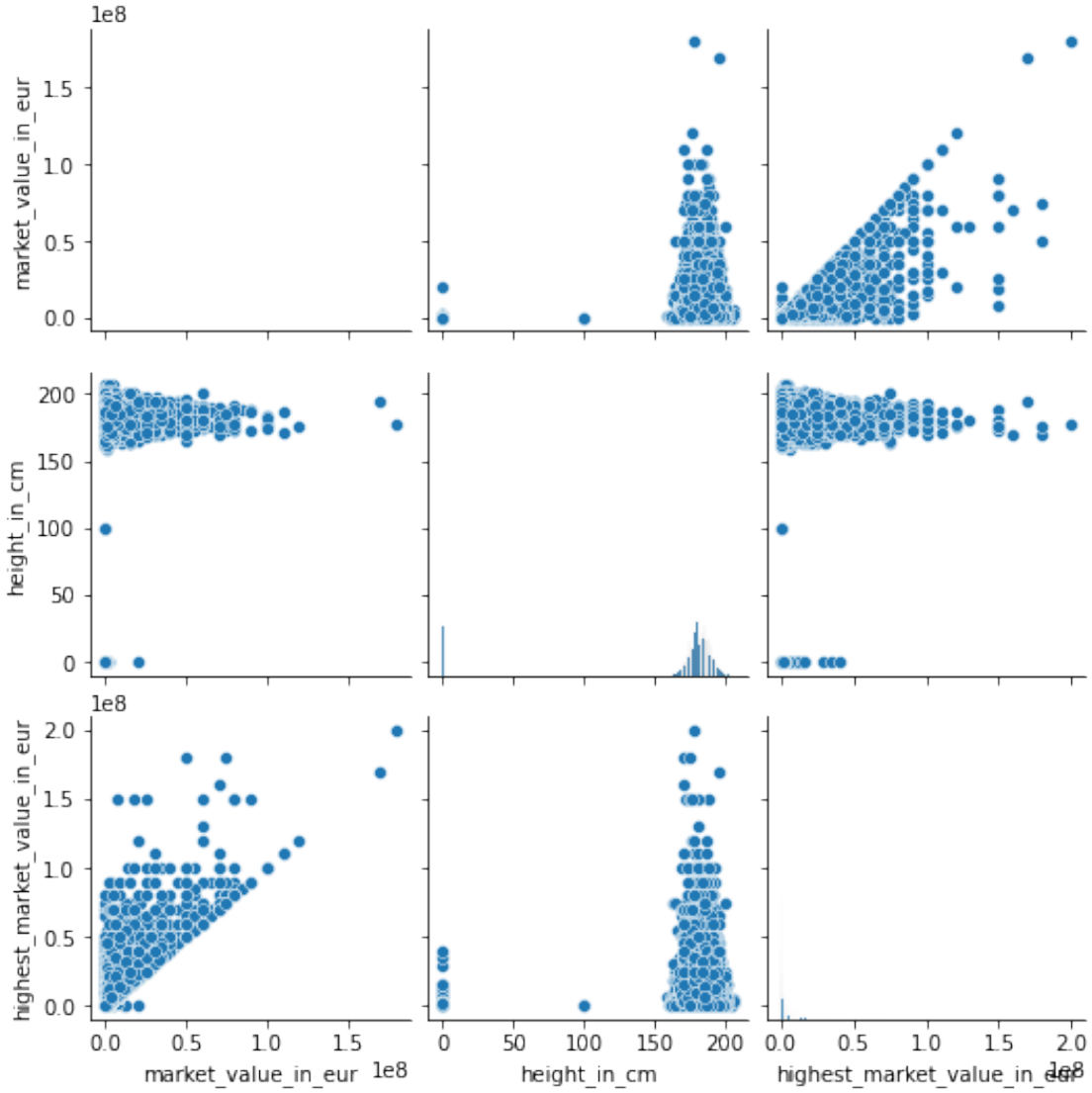
[27]:
```
(array([0, 1, 2, 3]),
 [Text(0, 0, 'Attack'),
  Text(1, 0, 'Midfield'),
  Text(2, 0, 'Defender'),
  Text(3, 0, 'Goalkeeper')])
```



[28]:
```
## Pairplot for numerical value Height in cm , market value in Euro , Highest␣
↪market value in Euro
```

[29]:
```
sns.pairplot(df[['market_value_in_eur', 'height_in_cm',␣
↪'highest_market_value_in_eur']])
```

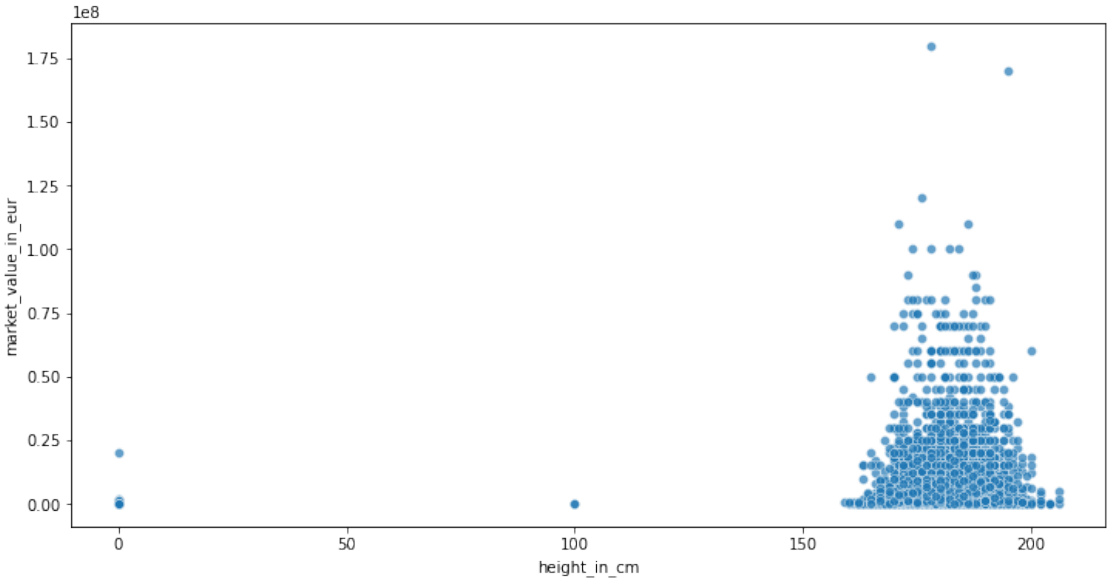
[29]: <seaborn.axisgrid.PairGrid at 0x2a24af65e80>



[30]:
```
## Market Value vs Height
```

[31]:
```
plt.figure(figsize=(12, 6))
sns.scatterplot(x='height_in_cm', y='market_value_in_eur', data=df, alpha=0.7)
```

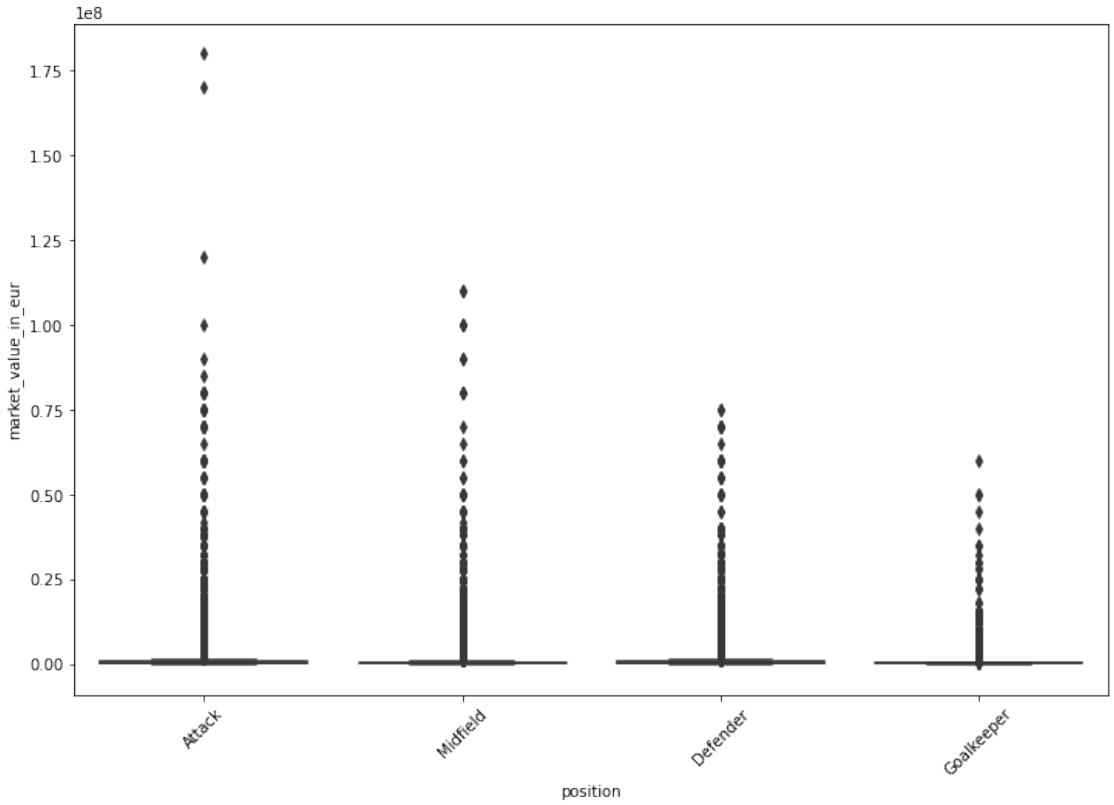
[31]:
```
<AxesSubplot:xlabel='height_in_cm', ylabel='market_value_in_eur'>
```



[32]:
```
## Market Value by position
```

[33]:
```
plt.figure(figsize=(12, 8))
sns.boxplot(x='position', y='market_value_in_eur', data=df, palette='viridis')
plt.xticks(rotation=45)
```

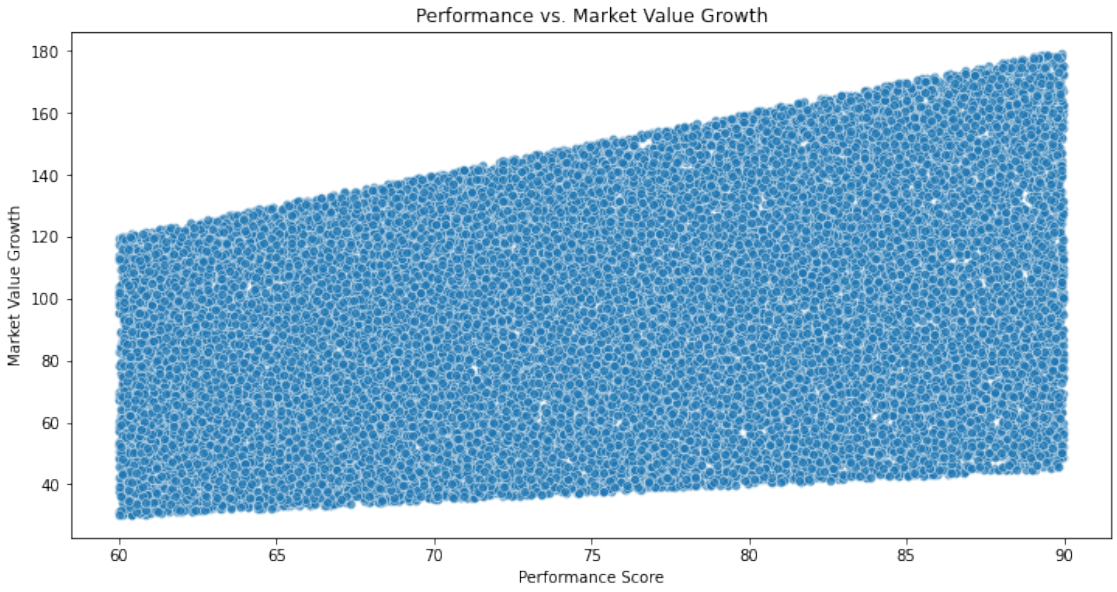
[33]:
```
(array([0, 1, 2, 3]),
 [Text(0, 0, 'Attack'),
  Text(1, 0, 'Midfield'),
  Text(2, 0, 'Defender'),
  Text(3, 0, 'Goalkeeper')])
```



[34]:
```
np.random.seed(42)  # For reproducibility
df['performance_score'] = np.random.uniform(60, 90, len(df))
df['market_value_growth'] = df['performance_score'] * np.random.uniform(0.5, 2.
↪0, len(df))
```

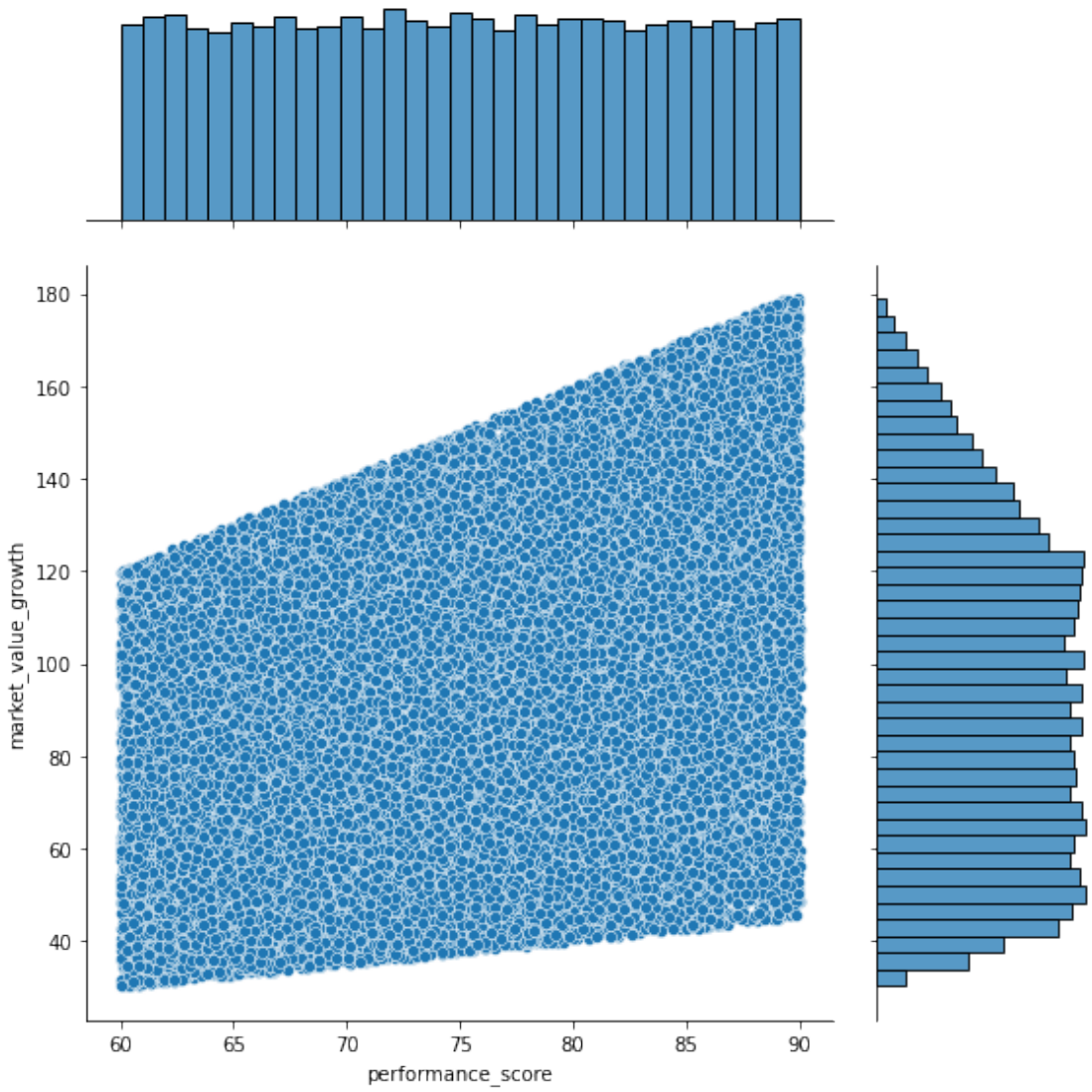
[35]:
```
## Performance vs Market Value Growth
```

[36]:
```
plt.figure(figsize=(12, 6))
sns.scatterplot(x='performance_score', y='market_value_growth', data=df,␣
↪alpha=0.7)
plt.title('Performance vs. Market Value Growth')
plt.xlabel('Performance Score')
plt.ylabel('Market Value Growth')
plt.show()
```

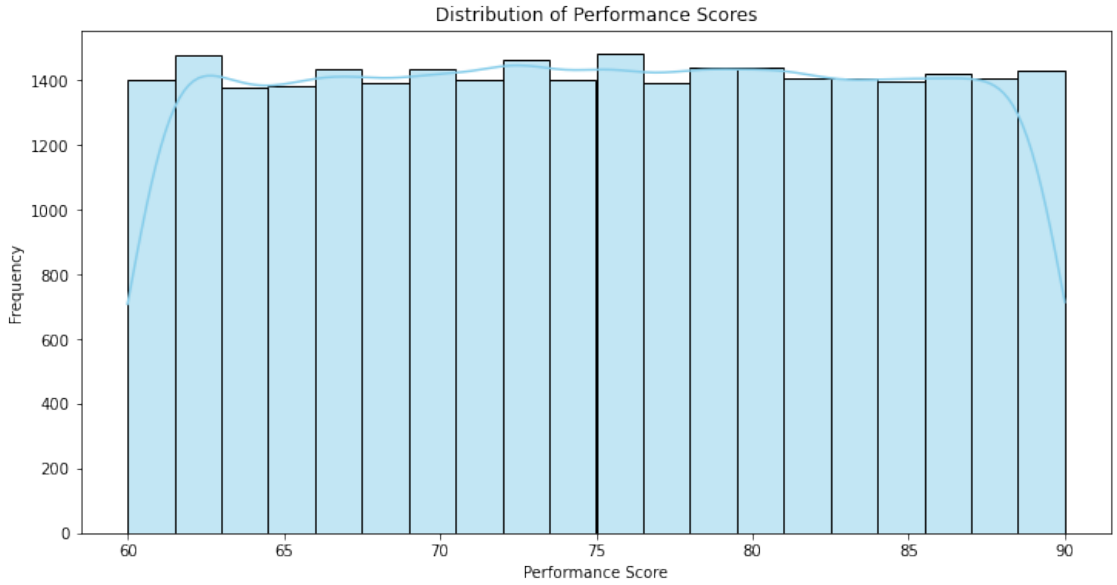


[37]:
```
# Jointplot with marginal histograms
sns.jointplot(x='performance_score', y='market_value_growth', data=df,␣
↪kind='scatter', height=8, ratio=3)
```

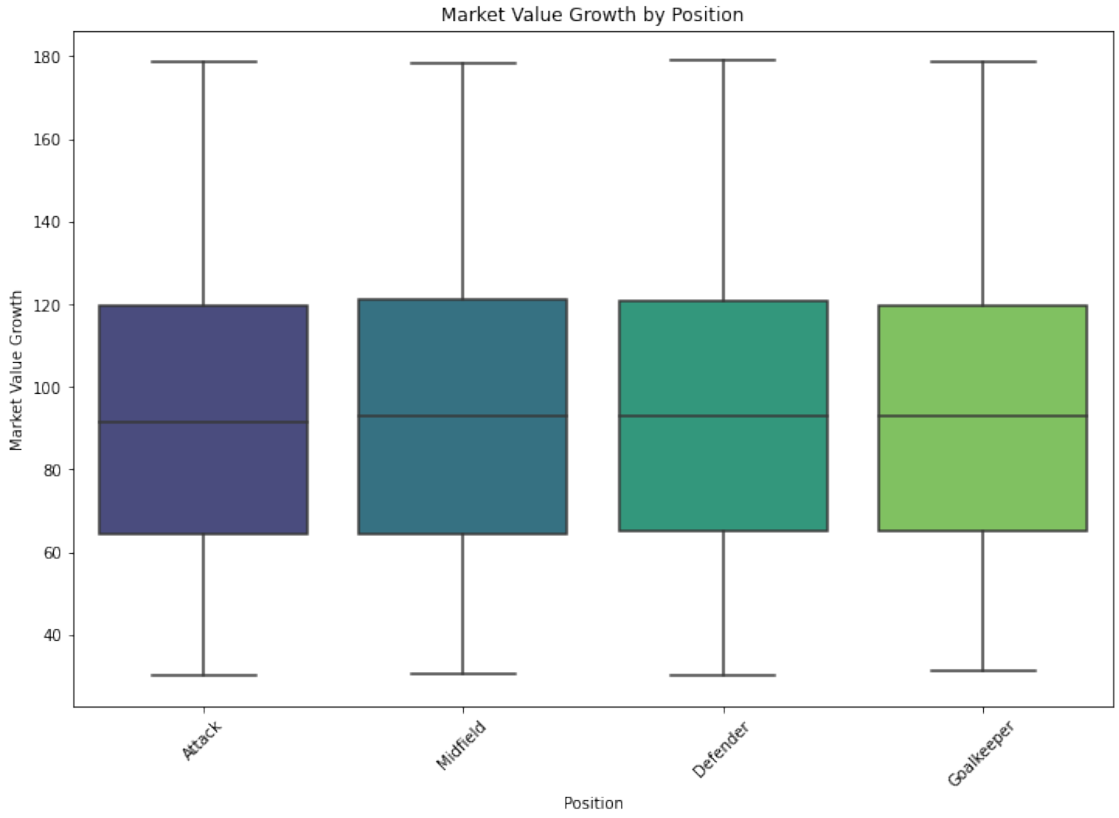
[37]:
```
<seaborn.axisgrid.JointGrid at 0x2a254079760>
```



[38]:
```
# Distribution of Performance Scores
plt.figure(figsize=(12, 6))
sns.histplot(df['performance_score'], bins=20, kde=True, color='skyblue')
plt.title('Distribution of Performance Scores')
plt.xlabel('Performance Score')
plt.ylabel('Frequency')
plt.show()
```



[39]:
```
# Boxplot of Market Value Growth by Position
plt.figure(figsize=(12, 8))
sns.boxplot(x='position', y='market_value_growth', data=df, palette='viridis')
plt.xticks(rotation=45)
plt.title('Market Value Growth by Position')
plt.xlabel('Position')
plt.ylabel('Market Value Growth')
plt.show()
```



[40]:
```
## Players market value in their prime
## The players age is a key indicator of the market Value
## Players value increases in their mid 20's and decreases while they age. Age␣
↪reflects both experience and ability
```

[41]:
```
# Convert 'date_of_birth' to datetime
df['date_of_birth'] = pd.to_datetime(df['date_of_birth'], errors='coerce')
```

[42]:
```
# Calculate age based on the current date
df['age'] = (pd.to_datetime('2022-12-31') - df['date_of_birth']).
↪astype('<m8[Y]')
```

[43]:
```
# Filter players between the ages of 22 and 27
filtered_players = df[(df['age'] >= 22) & (df['age'] <= 27)]
```

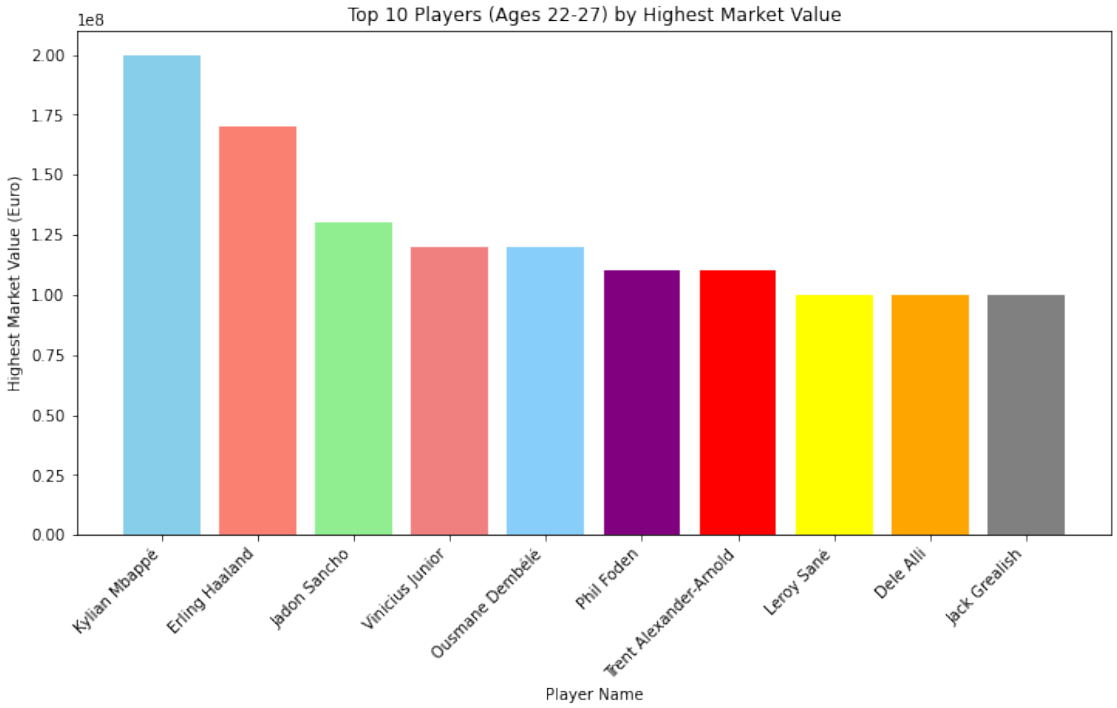
[44]:
```
# Select top 10 players based on highest market value
top_10_players = filtered_players.nlargest(10, 'highest_market_value_in_eur')
```

[45]:
```
# Display the selected players with their ages
for index, row in top_10_players[['name', 'age',␣
↪'highest_market_value_in_eur']].iterrows():
```

```
    print(f"Player: {row['name']}, Age: {int(row['age'])}, Highest Market Value:
 ↪ {row['highest_market_value_in_eur']} Euro")
```

```
Player: Kylian Mbappé, Age: 24, Highest Market Value: 200000000.0 Euro
Player: Erling Haaland, Age: 22, Highest Market Value: 170000000.0 Euro
Player: Jadon Sancho, Age: 22, Highest Market Value: 130000000.0 Euro
Player: Vinicius Junior, Age: 22, Highest Market Value: 120000000.0 Euro
Player: Ousmane Dembélé, Age: 25, Highest Market Value: 120000000.0 Euro
Player: Phil Foden, Age: 22, Highest Market Value: 110000000.0 Euro
Player: Trent Alexander-Arnold, Age: 24, Highest Market Value: 110000000.0 Euro
Player: Leroy Sané, Age: 26, Highest Market Value: 100000000.0 Euro
Player: Dele Alli, Age: 26, Highest Market Value: 100000000.0 Euro
Player: Jack Grealish, Age: 27, Highest Market Value: 100000000.0 Euro
```

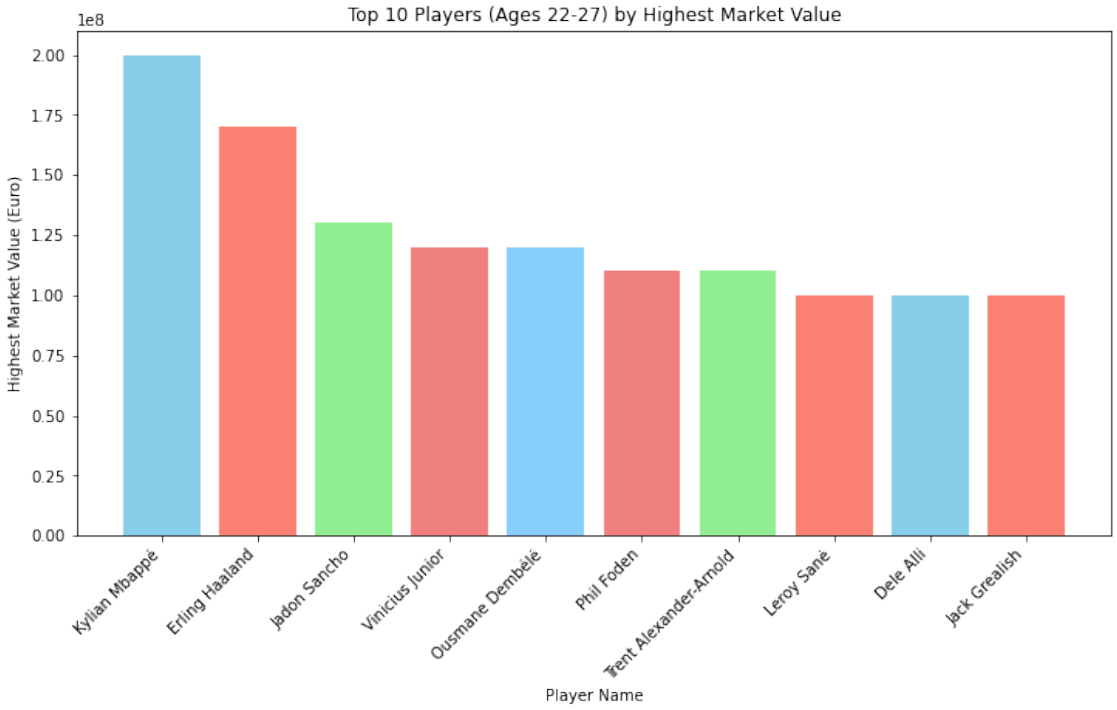
[46]:
```
# Draw a bar chart
colors = ['skyblue', 'salmon', 'lightgreen', 'lightcoral', 'lightskyblue',␣
 ↪'purple', 'red', 'yellow', 'orange', 'gray']
plt.figure(figsize=(12, 6))
plt.bar(top_10_players['name'], top_10_players['highest_market_value_in_eur'],␣
 ↪color=colors)
plt.xlabel('Player Name')
plt.ylabel('Highest Market Value (Euro)')
plt.title('Top 10 Players (Ages 22-27) by Highest Market Value')
plt.xticks(rotation=45, ha='right')  # Rotate x-axis labels for better␣
 ↪visibility
plt.show()
```



[47]:
```
df['date_of_birth'] = pd.to_datetime(df['date_of_birth'], errors='coerce')
df['age'] = (pd.to_datetime('2022-12-31') - df['date_of_birth']).
 ↪astype('<m8[Y]')
filtered_players = df[(df['age'] >= 22) & (df['age'] <= 27)]
top_10_players = filtered_players.nlargest(10, 'highest_market_value_in_eur')
for index, row in top_10_players[['name', 'age', 'current_club_name',␣
 ↪'country_of_citizenship', 'highest_market_value_in_eur']].iterrows():
    print(f"Player: {row['name']}, Age: {int(row['age'])}, Current Club:␣
 ↪{row['current_club_name']}, Country: {row['country_of_citizenship']},␣
 ↪Highest Market Value: {row['highest_market_value_in_eur']} Euro")
```

```
Player: Kylian Mbappé, Age: 24, Current Club: Fc Paris Saint Germain, Country:
France, Highest Market Value: 200000000.0 Euro
Player: Erling Haaland, Age: 22, Current Club: Manchester City, Country: Norway,
Highest Market Value: 170000000.0 Euro
Player: Jadon Sancho, Age: 22, Current Club: Manchester United, Country:
England, Highest Market Value: 130000000.0 Euro
Player: Vinicius Junior, Age: 22, Current Club: Real Madrid, Country: Brazil,
Highest Market Value: 120000000.0 Euro
Player: Ousmane Dembélé, Age: 25, Current Club: Fc Barcelona, Country: France,
Highest Market Value: 120000000.0 Euro
Player: Phil Foden, Age: 22, Current Club: Manchester City, Country: England,
Highest Market Value: 110000000.0 Euro
Player: Trent Alexander-Arnold, Age: 24, Current Club: Fc Liverpool, Country:
England, Highest Market Value: 110000000.0 Euro
Player: Leroy Sané, Age: 26, Current Club: Fc Bayern Munchen, Country: Germany,
Highest Market Value: 100000000.0 Euro
Player: Dele Alli, Age: 26, Current Club: Besiktas Istanbul, Country: England,
Highest Market Value: 100000000.0 Euro
Player: Jack Grealish, Age: 27, Current Club: Manchester City, Country: England,
Highest Market Value: 100000000.0 Euro
```

[48]:
```
colors = ['skyblue', 'salmon', 'lightgreen', 'lightcoral', 'lightskyblue',␣
 ↪'lightcoral', 'lightgreen', 'salmon', 'skyblue', 'salmon']
plt.figure(figsize=(12, 6))
plt.bar(top_10_players['name'], top_10_players['highest_market_value_in_eur'],␣
 ↪color=colors)
plt.xlabel('Player Name')
plt.ylabel('Highest Market Value (Euro)')
plt.title('Top 10 Players (Ages 22-27) by Highest Market Value')
plt.xticks(rotation=45, ha='right')  # Rotate x-axis labels for better␣
 ↪visibility
plt.show()
```



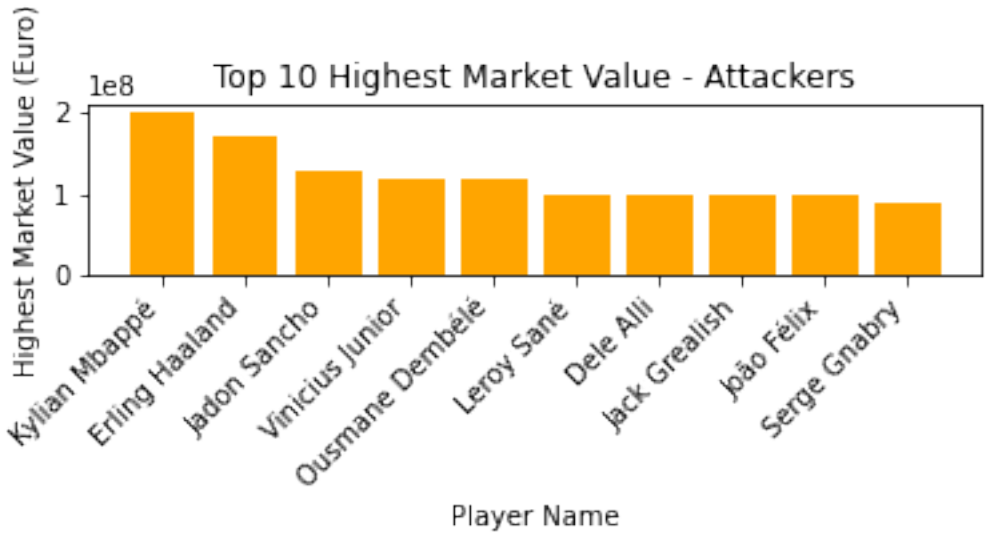
[49]:
```
## Separate players by position
```

[50]:
```
df['date_of_birth'] = pd.to_datetime(df['date_of_birth'], errors='coerce')
df['age'] = (pd.to_datetime('2022-12-31') - df['date_of_birth']).
 ↪astype('<m8[Y]')
filtered_players = df[(df['age'] >= 22) & (df['age'] <= 27)]
attackers = filtered_players[filtered_players['position'] == 'Attack'].
 ↪nlargest(10, 'highest_market_value_in_eur')
midfielders = filtered_players[filtered_players['position'] == 'Midfield'].
 ↪nlargest(10, 'highest_market_value_in_eur')
defenders = filtered_players[filtered_players['position'] == 'Defender'].
 ↪nlargest(10, 'highest_market_value_in_eur')
```

[51]:
```
## Attackers
```

[52]:
```
plt.subplot(3, 1, 1)
plt.bar(attackers['name'], attackers['highest_market_value_in_eur'],␣
 ↪color='orange')
plt.title('Top 10 Highest Market Value - Attackers')
plt.xlabel('Player Name')
plt.ylabel('Highest Market Value (Euro)')
plt.xticks(rotation=45, ha='right')
```

[52]:
```
([0, 1, 2, 3, 4, 5, 6, 7, 8, 9],
 [Text(0, 0, ''),
  Text(0, 0, ''),
  Text(0, 0, ''),
  Text(0, 0, ''),
  Text(0, 0, ''),
  Text(0, 0, ''),
  Text(0, 0, ''),
  Text(0, 0, ''),
  Text(0, 0, ''),
  Text(0, 0, '')])
```



[53]:
```
## Midfielders
```

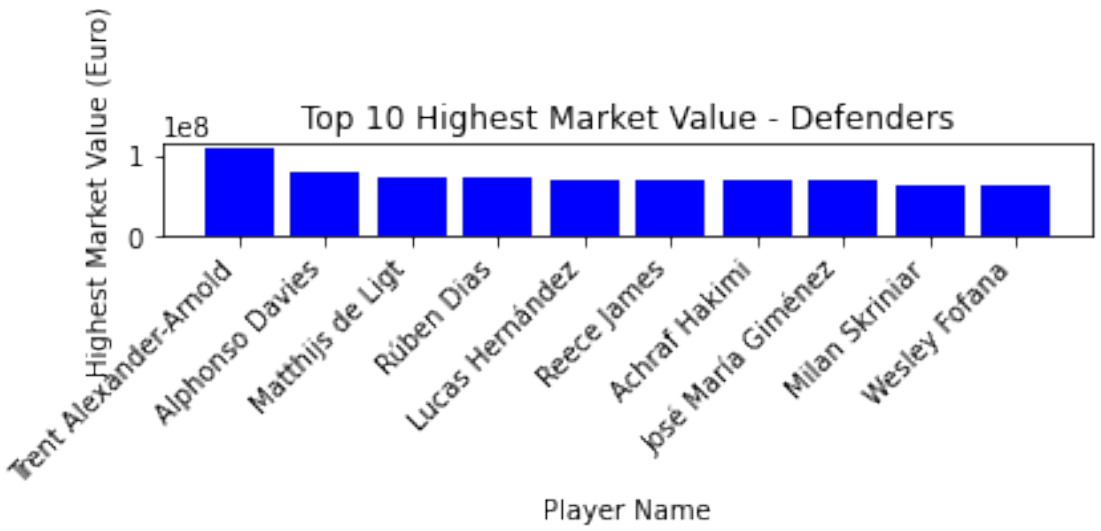
[54]:
```
plt.subplot(3, 1, 2)
plt.bar(midfielders['name'], midfielders['highest_market_value_in_eur'],␣
 ↪color='green')
plt.title('Top 10 Highest Market Value - Midfielders')
plt.xlabel('Player Name')
plt.ylabel('Highest Market Value (Euro)')
plt.xticks(rotation=45, ha='right')
```

[54]:
```
([0, 1, 2, 3, 4, 5, 6, 7, 8, 9],
 [Text(0, 0, ''),
  Text(0, 0, ''),
  Text(0, 0, ''),
  Text(0, 0, ''),
  Text(0, 0, ''),
```

```
Text(0, 0, ''),
Text(0, 0, ''),
Text(0, 0, ''),
Text(0, 0, ''),
Text(0, 0, '')])
```

[55]:
```
##Defenders
```

[56]:
```
plt.subplot(3, 1, 3)
plt.bar(defenders['name'], defenders['highest_market_value_in_eur'],␣
↪color='blue')
plt.title('Top 10 Highest Market Value - Defenders')
plt.xlabel('Player Name')
plt.ylabel('Highest Market Value (Euro)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



[57]:
```
## The second dataset contains more perdormance metrics such as
#  skill moves, international reputation, workrate, passing, finishing,␣
 ↪aggression and others.
#  Like dataset 1, the attributes can be found on sofifa.com and dataset␣
 ↪downloaded from kaggle.com
```

[58]:
```
df2 = pd.read_csv('C:\\Users\\kweku\\Desktop\\Dataset\\fifadata.csv')
```

[59]:
```
## Information and data columns of dataset 2
```

[60]:
```
df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18207 entries, 0 to 18206
Data columns (total 89 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Unnamed: 0                18207 non-null  int64
 1   ID                        18207 non-null  int64
 2   Name                      18207 non-null  object
 3   Age                       18207 non-null  int64
 4   Photo                     18207 non-null  object
 5   Nationality               18207 non-null  object
 6   Flag                      18207 non-null  object
 7   Overall                   18207 non-null  int64
 8   Potential                 18207 non-null  int64
 9   Club                      17966 non-null  object
 10  Club Logo                 18207 non-null  object
 11  Value                     18207 non-null  object
```

```
12  Wage                      18207 non-null  object
13  Special                   18207 non-null  int64
14  Preferred Foot            18159 non-null  object
15  International Reputation  18159 non-null  float64
16  Weak Foot                 18159 non-null  float64
17  Skill Moves               18159 non-null  float64
18  Work Rate                 18159 non-null  object
19  Body Type                 18159 non-null  object
20  Real Face                 18159 non-null  object
21  Position                  18147 non-null  object
22  Jersey Number             18147 non-null  float64
23  Joined                    16654 non-null  object
24  Loaned From               1264 non-null   object
25  Contract Valid Until      17918 non-null  object
26  Height                    18159 non-null  object
27  Weight                    18159 non-null  object
28  LS                        16122 non-null  object
29  ST                        16122 non-null  object
30  RS                        16122 non-null  object
31  LW                        16122 non-null  object
32  LF                        16122 non-null  object
33  CF                        16122 non-null  object
34  RF                        16122 non-null  object
35  RW                        16122 non-null  object
36  LAM                       16122 non-null  object
37  CAM                       16122 non-null  object
38  RAM                       16122 non-null  object
39  LM                        16122 non-null  object
40  LCM                       16122 non-null  object
41  CM                        16122 non-null  object
42  RCM                       16122 non-null  object
43  RM                        16122 non-null  object
44  LWB                       16122 non-null  object
45  LDM                       16122 non-null  object
46  CDM                       16122 non-null  object
47  RDM                       16122 non-null  object
48  RWB                       16122 non-null  object
49  LB                        16122 non-null  object
50  LCB                       16122 non-null  object
51  CB                        16122 non-null  object
52  RCB                       16122 non-null  object
53  RB                        16122 non-null  object
54  Crossing                  18159 non-null  float64
55  Finishing                 18159 non-null  float64
56  HeadingAccuracy           18159 non-null  float64
57  ShortPassing              18159 non-null  float64
58  Volleys                   18159 non-null  float64
59  Dribbling                 18159 non-null  float64
```

```
 60  Curve                     18159 non-null  float64
 61  FKAccuracy                18159 non-null  float64
 62  LongPassing               18159 non-null  float64
 63  BallControl               18159 non-null  float64
 64  Acceleration              18159 non-null  float64
 65  SprintSpeed               18159 non-null  float64
 66  Agility                   18159 non-null  float64
 67  Reactions                 18159 non-null  float64
 68  Balance                   18159 non-null  float64
 69  ShotPower                 18159 non-null  float64
 70  Jumping                   18159 non-null  float64
 71  Stamina                   18159 non-null  float64
 72  Strength                  18159 non-null  float64
 73  LongShots                 18159 non-null  float64
 74  Aggression                18159 non-null  float64
 75  Interceptions             18159 non-null  float64
 76  Positioning               18159 non-null  float64
 77  Vision                    18159 non-null  float64
 78  Penalties                 18159 non-null  float64
 79  Composure                 18159 non-null  float64
 80  Marking                   18159 non-null  float64
 81  StandingTackle            18159 non-null  float64
 82  SlidingTackle             18159 non-null  float64
 83  GKDiving                  18159 non-null  float64
 84  GKHandling                18159 non-null  float64
 85  GKKicking                 18159 non-null  float64
 86  GKPositioning             18159 non-null  float64
 87  GKReflexes                18159 non-null  float64
 88  Release Clause            16643 non-null  object
dtypes: float64(38), int64(6), object(45)
memory usage: 12.4+ MB
```

[61]:
```
## first 5 rows and columns of dataset 2
```

[62]:
```
df2.head()
```

[62]:

	Unnamed: 0	ID	Name	Age	Photo	Nationality
0	0	158023	L. Messi	31	https://cdn.sofifa.org/players/4/19/158023.png	Argentina
1	1	20801	Cristiano Ronaldo	33	https://cdn.sofifa.org/players/4/19/20801.png	Portugal
2	2	190871	Neymar Jr	26	https://cdn.sofifa.org/players/4/19/190871.png	Brazil
3	3	193080	De Gea	27		
4	4	192985	K. De Bruyne	27		

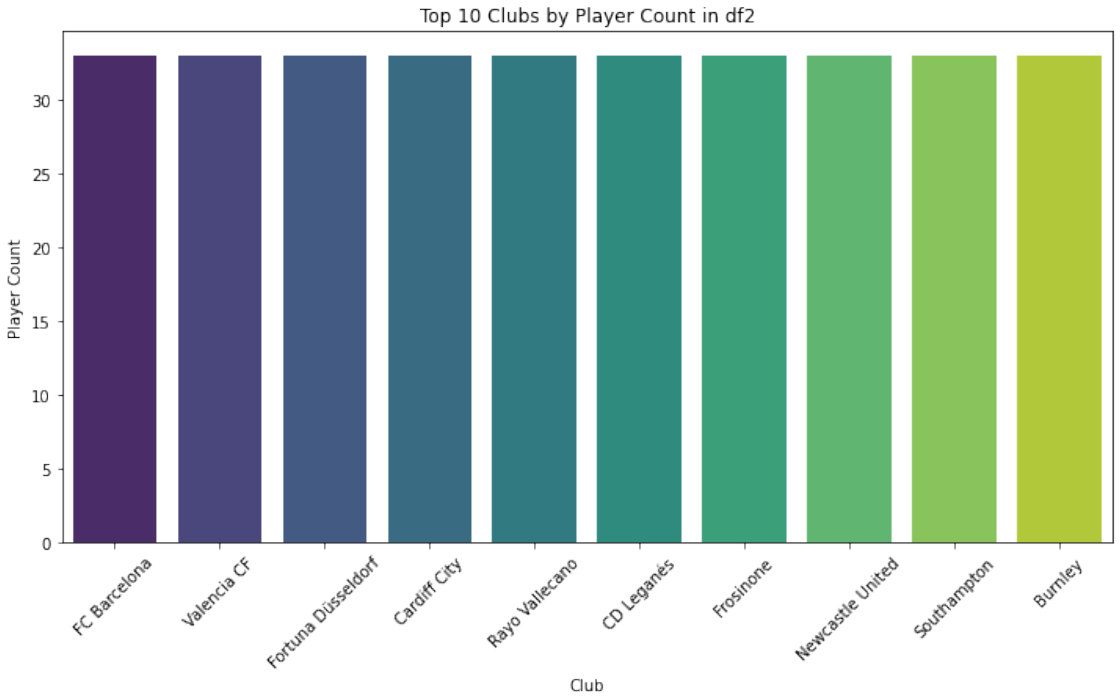
```
3  https://cdn.sofifa.org/players/4/19/193080.png       Spain
4  https://cdn.sofifa.org/players/4/19/192985.png     Belgium

                                Flag  Overall  Potential  \
0  https://cdn.sofifa.org/flags/52.png       94         94
1  https://cdn.sofifa.org/flags/38.png       94         94
2  https://cdn.sofifa.org/flags/54.png       92         93
3  https://cdn.sofifa.org/flags/45.png       91         93
4   https://cdn.sofifa.org/flags/7.png       91         92

                 Club                                     Club Logo    Value  \
0        FC Barcelona  https://cdn.sofifa.org/teams/2/light/241.png  €110.5M
1            Juventus   https://cdn.sofifa.org/teams/2/light/45.png     €77M
2  Paris Saint-Germain   https://cdn.sofifa.org/teams/2/light/73.png  €118.5M
3   Manchester United   https://cdn.sofifa.org/teams/2/light/11.png     €72M
4      Manchester City   https://cdn.sofifa.org/teams/2/light/10.png    €102M

    Wage  Special Preferred Foot  International Reputation  Weak Foot  \
0  €565K     2202           Left                       5.0        4.0
1  €405K     2228          Right                       5.0        4.0
2  €290K     2143          Right                       5.0        5.0
3  €260K     1471          Right                       4.0        3.0
4  €355K     2281          Right                       4.0        5.0

   Skill Moves       Work Rate   Body Type Real Face Position  Jersey Number  \
0          4.0  Medium/ Medium       Messi       Yes       RF           10.0
1          5.0       High/ Low  C. Ronaldo       Yes       ST            7.0
2          5.0    High/ Medium      Neymar       Yes       LW           10.0
3          1.0  Medium/ Medium        Lean       Yes       GK            1.0
4          4.0      High/ High      Normal       Yes      RCM            7.0

         Joined Loaned From Contract Valid Until Height  Weight    LS    ST  \
0   Jul 1, 2004         NaN                2021    5'7  159lbs  88+2  88+2
1  Jul 10, 2018         NaN                2022    6'2  183lbs  91+3  91+3
2   Aug 3, 2017         NaN                2022    5'9  150lbs  84+3  84+3
3   Jul 1, 2011         NaN                2020    6'4  168lbs   NaN   NaN
4  Aug 30, 2015         NaN                2023   5'11  154lbs  82+3  82+3

     RS    LW    LF    CF    RF    RW   LAM  …   RCB    RB Crossing  \
0  88+2  92+2  93+2  93+2  93+2  92+2  93+2  …  47+2  59+2     84.0
1  91+3  89+3  90+3  90+3  90+3  89+3  88+3  …  53+3  61+3     84.0
2  84+3  89+3  89+3  89+3  89+3  89+3  89+3  …  47+3  60+3     79.0
3   NaN   NaN   NaN   NaN   NaN   NaN   NaN  …   NaN   NaN     17.0
4  82+3  87+3  87+3  87+3  87+3  87+3  88+3  …  66+3  73+3     93.0

  Finishing HeadingAccuracy ShortPassing Volleys Dribbling Curve FKAccuracy  \
0      95.0            70.0         90.0    86.0      97.0  93.0       94.0
```

```
1      94.0           89.0         81.0    87.0      88.0  81.0       76.0
2      87.0           62.0         84.0    84.0      96.0  88.0       87.0
3      13.0           21.0         50.0    13.0      18.0  21.0       19.0
4      82.0           55.0         92.0    82.0      86.0  85.0       83.0

  LongPassing BallControl Acceleration SprintSpeed Agility Reactions Balance  \
0        87.0        96.0         91.0        86.0    91.0      95.0    95.0
1        77.0        94.0         89.0        91.0    87.0      96.0    70.0
2        78.0        95.0         94.0        90.0    96.0      94.0    84.0
3        51.0        42.0         57.0        58.0    60.0      90.0    43.0
4        91.0        91.0         78.0        76.0    79.0      91.0    77.0

   ShotPower  Jumping  Stamina  Strength  LongShots  Aggression  \
0       85.0     68.0     72.0      59.0       94.0        48.0
1       95.0     95.0     88.0      79.0       93.0        63.0
2       80.0     61.0     81.0      49.0       82.0        56.0
3       31.0     67.0     43.0      64.0       12.0        38.0
4       91.0     63.0     90.0      75.0       91.0        76.0

   Interceptions  Positioning  Vision  Penalties  Composure  Marking  \
0           22.0         94.0    94.0       75.0       96.0     33.0
1           29.0         95.0    82.0       85.0       95.0     28.0
2           36.0         89.0    87.0       81.0       94.0     27.0
3           30.0         12.0    68.0       40.0       68.0     15.0
4           61.0         87.0    94.0       79.0       88.0     68.0

   StandingTackle  SlidingTackle  GKDiving  GKHandling  GKKicking  \
0            28.0           26.0       6.0        11.0       15.0
1            31.0           23.0       7.0        11.0       15.0
2            24.0           33.0       9.0         9.0       15.0
3            21.0           13.0      90.0        85.0       87.0
4            58.0           51.0      15.0        13.0        5.0

   GKPositioning  GKReflexes  Release Clause
0           14.0         8.0         €226.5M
1           14.0        11.0         €127.1M
2           15.0        11.0         €228.1M
3           88.0        94.0         €138.6M
4           10.0        13.0         €196.4M

[5 rows x 89 columns]
```

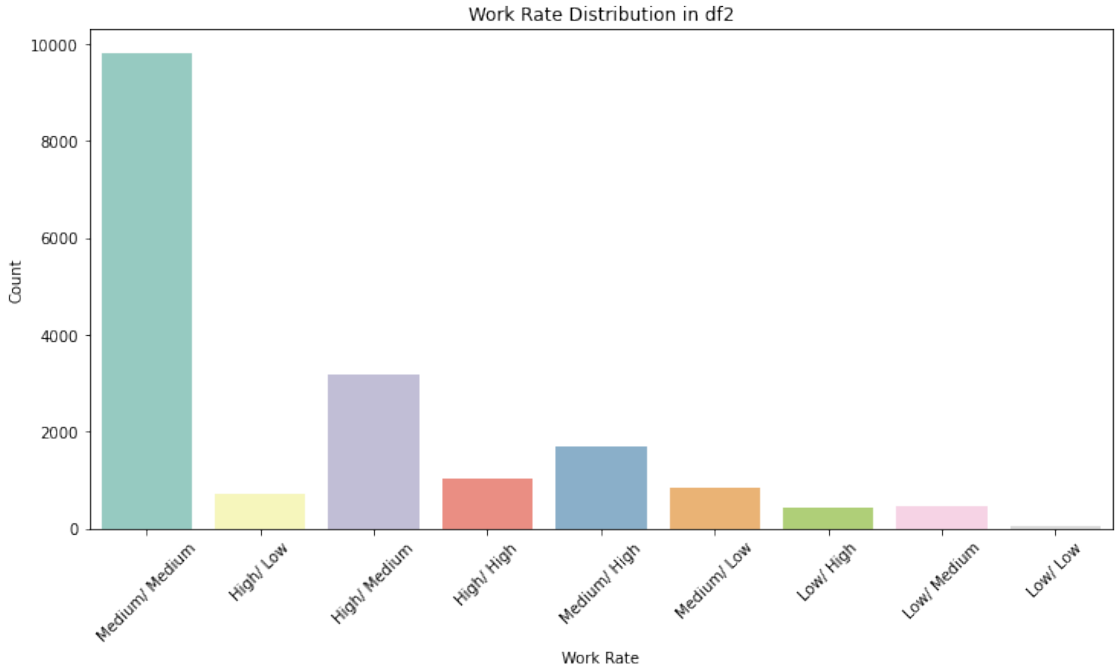
[63]:
```
## FURTHER RELATIONSHIPS BETWEEN FOOTBALL PLAYER ATTRIBUTES AND MARKET VALUE
```

[64]:
```
# Top 10 clubs by player count
top_clubs = df2['Club'].value_counts().nlargest(10)
plt.figure(figsize=(12, 6))
```

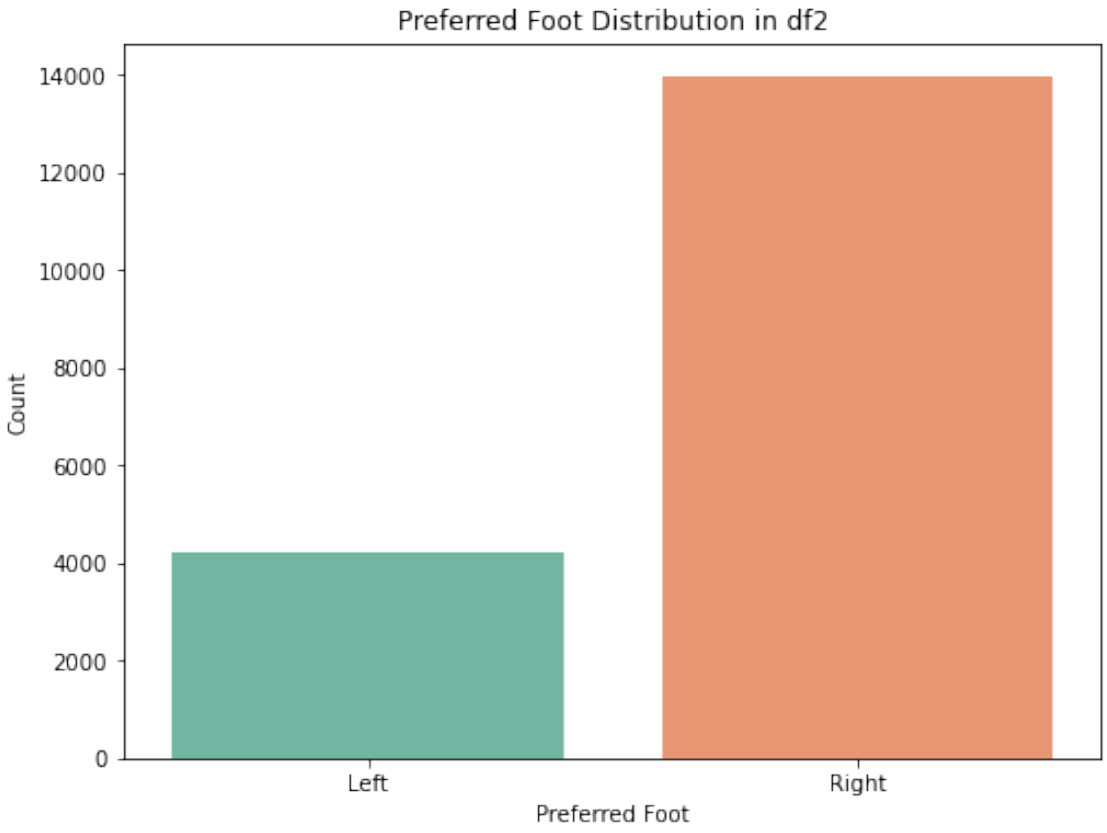
```
sns.barplot(x=top_clubs.index, y=top_clubs.values, palette='viridis')
plt.title('Top 10 Clubs by Player Count in df2')
plt.xlabel('Club')
plt.ylabel('Player Count')
plt.xticks(rotation=45)
plt.show()
```



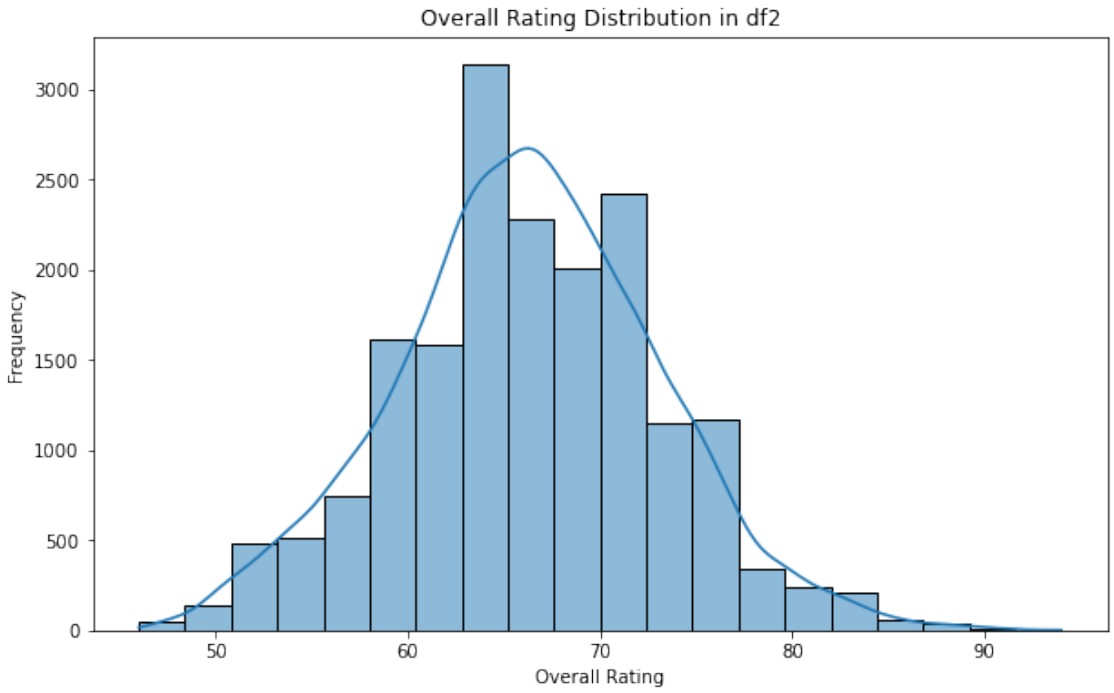
[65]:
```
# Work rate distribution
plt.figure(figsize=(12, 6))
sns.countplot(x='Work Rate', data=df2, palette='Set3')
plt.title('Work Rate Distribution in df2')
plt.xlabel('Work Rate')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.show()
```



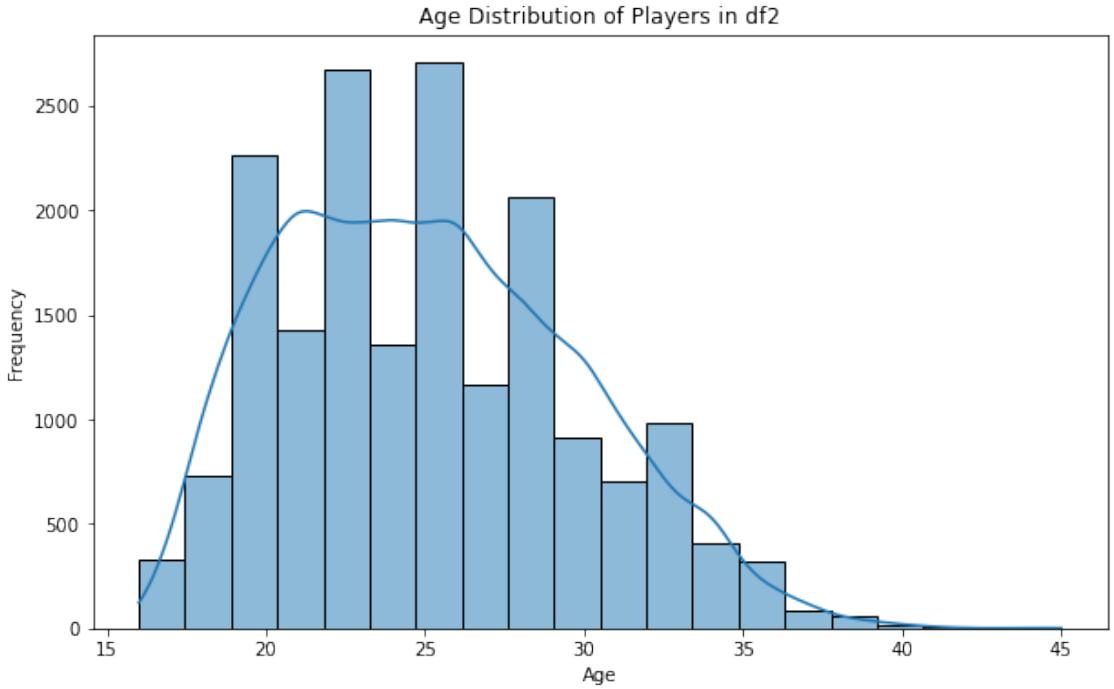
[66]:
```
# Preferred foot distribution
plt.figure(figsize=(8, 6))
sns.countplot(x='Preferred Foot', data=df2, palette='Set2')
plt.title('Preferred Foot Distribution in df2')
plt.xlabel('Preferred Foot')
plt.ylabel('Count')
plt.show()
```



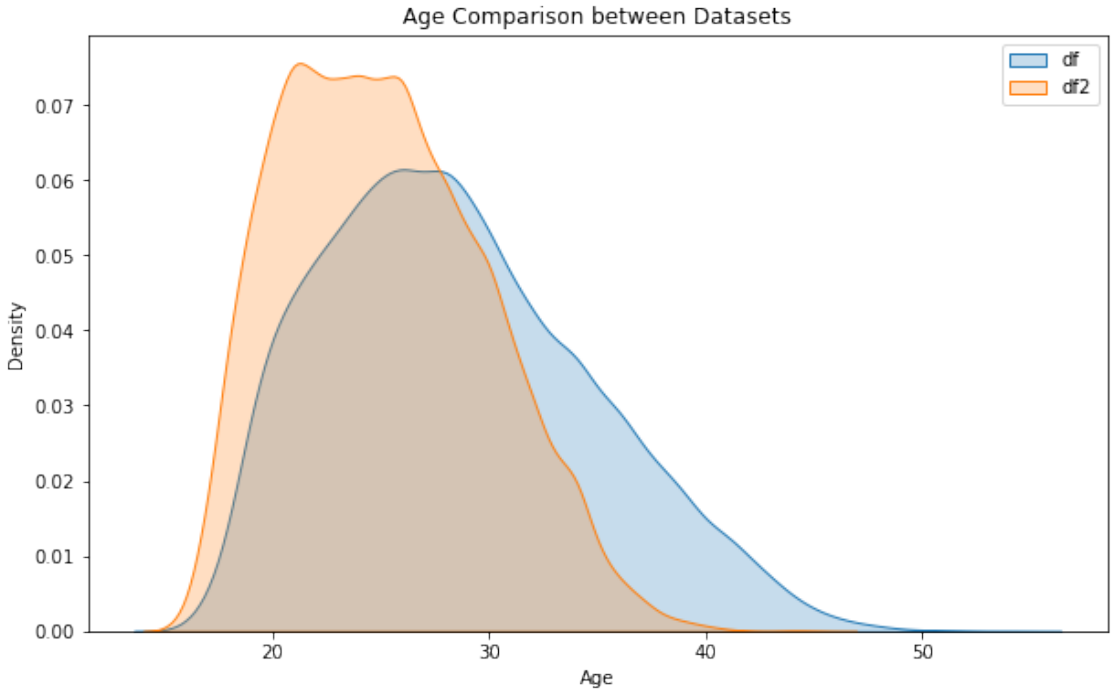
[67]:
```
# Overall rating distribution
plt.figure(figsize=(10, 6))
sns.histplot(df2['Overall'], bins=20, kde=True)
plt.title('Overall Rating Distribution in df2')
plt.xlabel('Overall Rating')
plt.ylabel('Frequency')
plt.show()
```



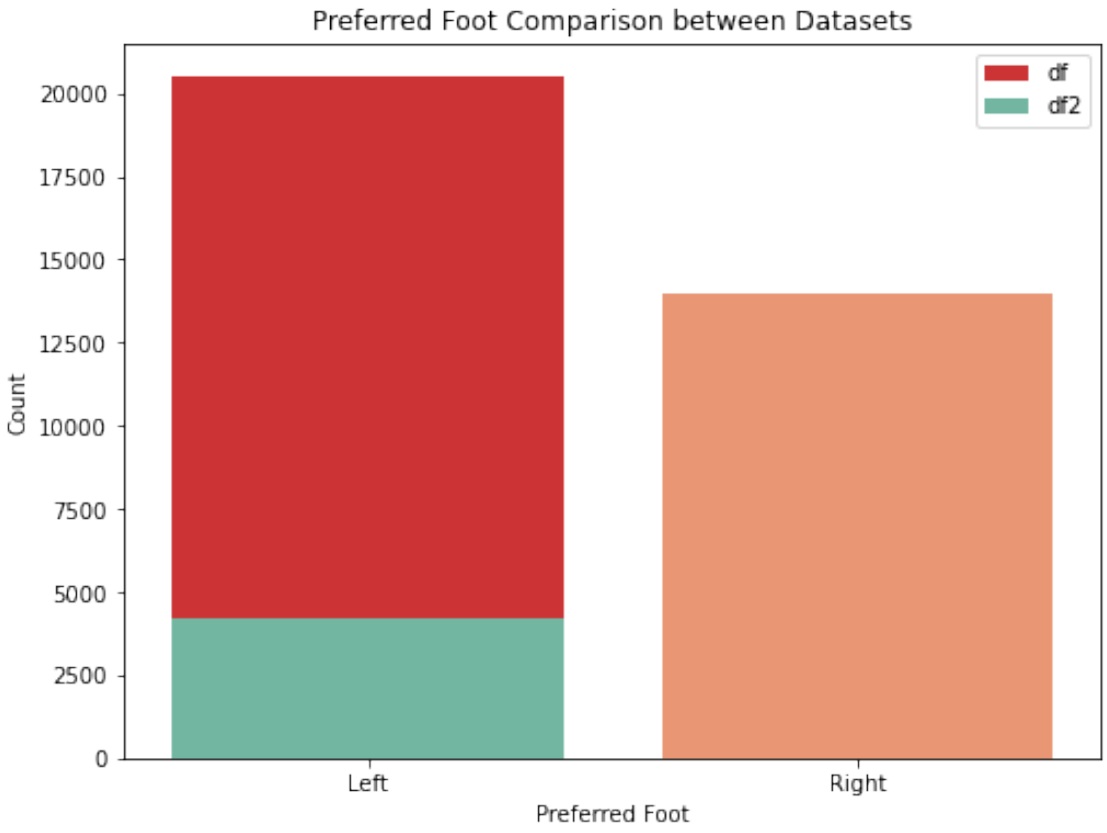
[68]:
```
# Age distribution
plt.figure(figsize=(10, 6))
sns.histplot(df2['Age'], bins=20, kde=True)
plt.title('Age Distribution of Players in df2')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```



[69]:
```
# Age comparison between the two datasets
plt.figure(figsize=(10, 6))
sns.kdeplot(df['age'], label='df', fill=True)
sns.kdeplot(df2['Age'], label='df2', fill=True)
plt.title('Age Comparison between Datasets')
plt.xlabel('Age')
plt.ylabel('Density')
plt.legend()
plt.show()
```

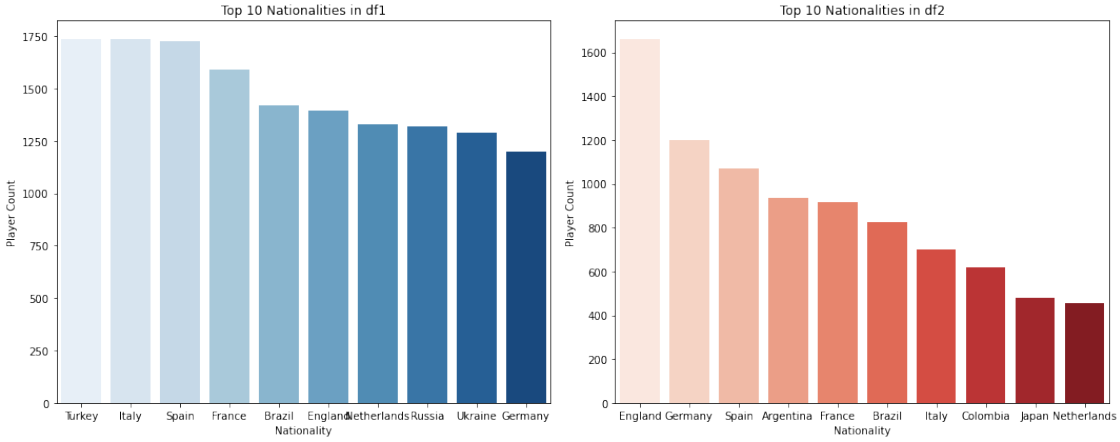


[70]:
```
# Preferred foot comparison between the two datasets
plt.figure(figsize=(8, 6))
sns.countplot(x='foot', data=df, palette='Set1', label='df')
sns.countplot(x='Preferred Foot', data=df2, palette='Set2', label='df2')
plt.title('Preferred Foot Comparison between Datasets')
plt.xlabel('Preferred Foot')
plt.ylabel('Count')
plt.legend()
plt.show()
```



[71]:
```
# Top 10 nationalities comparison between the two datasets
top_nationalities_df = df['country_of_citizenship'].value_counts().nlargest(10)
top_nationalities_df2 = df2['Nationality'].value_counts().nlargest(10)

plt.figure(figsize=(15, 6))
plt.subplot(1, 2, 1)
sns.barplot(x=top_nationalities_df.index, y=top_nationalities_df.values,␣
 ↪palette='Blues')
plt.title('Top 10 Nationalities in df1')
plt.xlabel('Nationality')
plt.ylabel('Player Count')

plt.subplot(1, 2, 2)
sns.barplot(x=top_nationalities_df2.index, y=top_nationalities_df2.values,␣
 ↪palette='Reds')
plt.title('Top 10 Nationalities in df2')
plt.xlabel('Nationality')
plt.ylabel('Player Count')

plt.tight_layout()
```

```
plt.show()
```



[72]:
```
## FEATURE SELECTION & EXPLORATION
#  I combined dataset 1 and dataset 2 for training and testing commonly␣
 ↪referred to as train_test_split
```

[73]:
```
# COMBINATION OF DATASET 1 & 2 (DF & DF2)
combined_df = pd.merge(df, df2, left_on='player_id', right_on='ID', how='inner')
combined_df = combined_df.drop(columns='ID')
print(combined_df.head())
```

```
   player_id                 name  current_club_id current_club_name  \
0     192366           Gozie Ugwu             1032        Fc Reading
1     240063       Georgios Koudas             5219       Aok Kerkyra
2     240074      Christos Armenis             5219       Aok Kerkyra
3     187915  Panagiotis Petrousis             5219       Aok Kerkyra
4     234324        Danny Cavanagh              511         Dundee Fc

  country_of_citizenship country_of_birth city_of_birth date_of_birth  \
0                England          England      Oxford      1993-04-22
1                 Greece           Greece     Kerkyra      1994-12-27
2                 Greece           Greece   Velonades      1993-06-22
3                 Greece           Greece   Giannitsa      1992-08-11
4               Scotland         Scotland      Dundee      1994-05-09

   position    sub_position   foot  height_in_cm  market_value_in_eur  \
0    Attack  Centre-Forward  Right           188             200000.0
1  Midfield   Left Midfield   Left           175             200000.0
2  Defender       Left-Back   Left           176             200000.0
3  Defender     Centre-Back  Right           186             200000.0
4  Midfield  Right Midfield  Right             0             200000.0
```

```
   highest_market_value_in_eur                    agent_name  \
0                     250000.0  RT Management & Consultancy
1                     100000.0                    Wasserman
2                     125000.0                    Wasserman
3                     150000.0                    Wasserman
4                     300000.0                    Wasserman

  contract_expiration_date current_club_domestic_competition_id  first_name  \
0               2023-06-01                                  GB1       Gozie
1               2023-06-30                                  GR1    Georgios
2               2023-06-30                                  GR1    Christos
3               2023-06-30                                  GR1  Panagiotis
4               2023-06-30                                  SC1       Danny

   last_name           player_code  \
0       Ugwu            gozie-ugwu
1     Koudas       georgios-koudas
2    Armenis      christos-armenis
3  Petrousis  panagiotis-petrousis
4   Cavanagh        danny-cavanagh

                                      image_url  last_season  \
0  https://img.a.transfermarkt.technology/portrai…         2012
1  https://img.a.transfermarkt.technology/portrai…         2012
2  https://img.a.transfermarkt.technology/portrai…         2012
3  https://img.a.transfermarkt.technology/portrai…         2012
4  https://img.a.transfermarkt.technology/portrai…         2012

                                           url  performance_score  \
0  https://www.transfermarkt.co.uk/gozie-ugwu/pro…          65.502135
1  https://www.transfermarkt.co.uk/georgios-kouda…          87.279612
2  https://www.transfermarkt.co.uk/christos-armen…          88.184968
3  https://www.transfermarkt.co.uk/panagiotis-pet…          62.654775
4  https://www.transfermarkt.co.uk/danny-cavanagh…          84.862125

   market_value_growth   age  Unnamed: 0         Name  Age  \
0           100.484926  29.0          89  N. Otamendi   30
1            79.327530  28.0       15122   M. Cassara   26
2           105.998425  29.0       16585     B. Dione   21
3            86.434060  30.0        7693   E. Kujović   30
4           155.647507  28.0       16395    L. Bilboe   20

                                      Photo Nationality  \
0  https://cdn.sofifa.org/players/4/19/192366.png   Argentina
1  https://cdn.sofifa.org/players/4/19/240063.png      France
2  https://cdn.sofifa.org/players/4/19/240074.png     Belgium
3  https://cdn.sofifa.org/players/4/19/187915.png      Sweden
4  https://cdn.sofifa.org/players/4/19/234324.png     England
```

```
                                  Flag  Overall  Potential  \
0  https://cdn.sofifa.org/flags/52.png       85         85
1  https://cdn.sofifa.org/flags/18.png       60         64
2   https://cdn.sofifa.org/flags/7.png       57         66
3  https://cdn.sofifa.org/flags/46.png       67         67
4  https://cdn.sofifa.org/flags/14.png       57         67

                  Club                                     Club Logo  \
0       Manchester City      https://cdn.sofifa.org/teams/2/light/10.png
1           GFC Ajaccio  https://cdn.sofifa.org/teams/2/light/110316.png
2  Royal Excel Mouscron  https://cdn.sofifa.org/teams/2/light/111560.png
3    Fortuna Düsseldorf  https://cdn.sofifa.org/teams/2/light/110636.png
4      Rotherham United    https://cdn.sofifa.org/teams/2/light/1797.png

    Value  …   RCB    RB Crossing  Finishing  HeadingAccuracy  ShortPassing  \
0  €28.5M  …  82+3  74+3     52.0       54.0             85.0          75.0
1   €190K  …   NaN   NaN     13.0        8.0             13.0          18.0
2   €160K  …  30+2  34+2     33.0       57.0             49.0          53.0
3   €725K  …  44+2  38+2     33.0       69.0             76.0          60.0
4   €130K  …   NaN   NaN     14.0        6.0             10.0          26.0

  Volleys Dribbling Curve FKAccuracy  LongPassing BallControl Acceleration  \
0    57.0      51.0  50.0       39.0         72.0        70.0         57.0
1     8.0       6.0  14.0       13.0         22.0        12.0         32.0
2    50.0      65.0  43.0       29.0         42.0        63.0         66.0
3    69.0      58.0  66.0       70.0         54.0        65.0         39.0
4     9.0      11.0  10.0       13.0         36.0        12.0         34.0

  SprintSpeed Agility Reactions Balance ShotPower Jumping Stamina Strength  \
0        61.0    64.0      79.0    62.0      69.0    92.0    67.0     80.0
1        31.0    32.0      43.0    22.0      23.0    71.0    29.0     72.0
2        58.0    63.0      48.0    74.0      61.0    65.0    49.0     49.0
3        36.0    34.0      59.0    32.0      74.0    42.0    50.0     90.0
4        38.0    28.0      43.0    22.0      18.0    38.0    27.0     65.0

  LongShots Aggression Interceptions Positioning Vision Penalties Composure  \
0      56.0       91.0          84.0        51.0   53.0      45.0      80.0
1       9.0       21.0          13.0         7.0   29.0      12.0      36.0
2      53.0       28.0          13.0        49.0   55.0      62.0      47.0
3      67.0       54.0          21.0        70.0   47.0      69.0      68.0
4       5.0       23.0          10.0         5.0   34.0      16.0      37.0

  Marking StandingTackle SlidingTackle GKDiving GKHandling GKKicking  \
0    83.0           85.0          84.0     12.0        5.0       8.0
1    10.0           13.0          10.0     63.0       61.0      60.0
2    16.0           14.0          17.0      5.0        6.0      13.0
3    24.0           34.0          15.0     16.0       13.0      13.0
```

```
4    13.0          11.0          13.0     60.0       57.0       60.0

  GKPositioning GKReflexes Release Clause
0          11.0       12.0         €52.7M
1          58.0       62.0          €347K
2          10.0        8.0          €264K
3           9.0        9.0          €1.3M
4          56.0       55.0          €267K

[5 rows x 114 columns]
```

[74]:
```
combined_df.head()
```

[74]:
```
   player_id                  name  current_club_id current_club_name  \
0     192366            Gozie Ugwu             1032        Fc Reading
1     240063       Georgios Koudas             5219       Aok Kerkyra
2     240074      Christos Armenis             5219       Aok Kerkyra
3     187915  Panagiotis Petrousis             5219       Aok Kerkyra
4     234324        Danny Cavanagh              511         Dundee Fc

  country_of_citizenship country_of_birth city_of_birth date_of_birth  \
0                England          England      Oxford      1993-04-22
1                 Greece           Greece     Kerkyra      1994-12-27
2                 Greece           Greece   Velonades      1993-06-22
3                 Greece           Greece   Giannitsa      1992-08-11
4               Scotland         Scotland      Dundee      1994-05-09

   position    sub_position   foot  height_in_cm  market_value_in_eur  \
0    Attack  Centre-Forward  Right           188             200000.0
1  Midfield   Left Midfield   Left           175             200000.0
2  Defender       Left-Back   Left           176             200000.0
3  Defender     Centre-Back  Right           186             200000.0
4  Midfield  Right Midfield  Right             0             200000.0

   highest_market_value_in_eur                   agent_name  \
0                     250000.0  RT Management & Consultancy
1                     100000.0                    Wasserman
2                     125000.0                    Wasserman
3                     150000.0                    Wasserman
4                     300000.0                    Wasserman

  contract_expiration_date current_club_domestic_competition_id  first_name  \
0               2023-06-01                                  GB1       Gozie
1               2023-06-30                                  GR1    Georgios
2               2023-06-30                                  GR1    Christos
3               2023-06-30                                  GR1  Panagiotis
4               2023-06-30                                  SC1       Danny
```

```
   last_name           player_code  \
0       Ugwu            gozie-ugwu
1     Koudas       georgios-koudas
2    Armenis      christos-armenis
3  Petrousis  panagiotis-petrousis
4   Cavanagh        danny-cavanagh

                                          image_url  last_season  \
0  https://img.a.transfermarkt.technology/portrai…         2012
1  https://img.a.transfermarkt.technology/portrai…         2012
2  https://img.a.transfermarkt.technology/portrai…         2012
3  https://img.a.transfermarkt.technology/portrai…         2012
4  https://img.a.transfermarkt.technology/portrai…         2012

                                              url  performance_score  \
0  https://www.transfermarkt.co.uk/gozie-ugwu/pro…          65.502135
1  https://www.transfermarkt.co.uk/georgios-kouda…          87.279612
2  https://www.transfermarkt.co.uk/christos-armen…          88.184968
3  https://www.transfermarkt.co.uk/panagiotis-pet…          62.654775
4  https://www.transfermarkt.co.uk/danny-cavanagh…          84.862125

   market_value_growth   age  Unnamed: 0         Name  Age  \
0           100.484926  29.0          89  N. Otamendi   30
1            79.327530  28.0       15122   M. Cassara   26
2           105.998425  29.0       16585     B. Dione   21
3            86.434060  30.0        7693   E. Kujović   30
4           155.647507  28.0       16395    L. Bilboe   20

                                       Photo Nationality  \
0  https://cdn.sofifa.org/players/4/19/192366.png   Argentina
1  https://cdn.sofifa.org/players/4/19/240063.png      France
2  https://cdn.sofifa.org/players/4/19/240074.png     Belgium
3  https://cdn.sofifa.org/players/4/19/187915.png      Sweden
4  https://cdn.sofifa.org/players/4/19/234324.png     England

                                Flag  Overall  Potential  \
0  https://cdn.sofifa.org/flags/52.png       85         85
1  https://cdn.sofifa.org/flags/18.png       60         64
2   https://cdn.sofifa.org/flags/7.png       57         66
3  https://cdn.sofifa.org/flags/46.png       67         67
4  https://cdn.sofifa.org/flags/14.png       57         67

                 Club                                     Club Logo  \
0      Manchester City      https://cdn.sofifa.org/teams/2/light/10.png
1          GFC Ajaccio  https://cdn.sofifa.org/teams/2/light/110316.png
2  Royal Excel Mouscron  https://cdn.sofifa.org/teams/2/light/111560.png
```

```
3    Fortuna Düsseldorf  https://cdn.sofifa.org/teams/2/light/110636.png
4      Rotherham United    https://cdn.sofifa.org/teams/2/light/1797.png

    Value  …   RCB    RB Crossing  Finishing  HeadingAccuracy  ShortPassing  \
0  €28.5M  …  82+3  74+3     52.0       54.0             85.0          75.0
1   €190K  …   NaN   NaN     13.0        8.0             13.0          18.0
2   €160K  …  30+2  34+2     33.0       57.0             49.0          53.0
3   €725K  …  44+2  38+2     33.0       69.0             76.0          60.0
4   €130K  …   NaN   NaN     14.0        6.0             10.0          26.0

  Volleys Dribbling Curve FKAccuracy  LongPassing BallControl Acceleration  \
0    57.0      51.0  50.0       39.0         72.0        70.0         57.0
1     8.0       6.0  14.0       13.0         22.0        12.0         32.0
2    50.0      65.0  43.0       29.0         42.0        63.0         66.0
3    69.0      58.0  66.0       70.0         54.0        65.0         39.0
4     9.0      11.0  10.0       13.0         36.0        12.0         34.0

  SprintSpeed Agility Reactions Balance ShotPower Jumping Stamina Strength  \
0        61.0    64.0      79.0    62.0      69.0    92.0    67.0     80.0
1        31.0    32.0      43.0    22.0      23.0    71.0    29.0     72.0
2        58.0    63.0      48.0    74.0      61.0    65.0    49.0     49.0
3        36.0    34.0      59.0    32.0      74.0    42.0    50.0     90.0
4        38.0    28.0      43.0    22.0      18.0    38.0    27.0     65.0

  LongShots Aggression Interceptions Positioning Vision Penalties Composure  \
0      56.0       91.0          84.0        51.0   53.0      45.0      80.0
1       9.0       21.0          13.0         7.0   29.0      12.0      36.0
2      53.0       28.0          13.0        49.0   55.0      62.0      47.0
3      67.0       54.0          21.0        70.0   47.0      69.0      68.0
4       5.0       23.0          10.0         5.0   34.0      16.0      37.0

  Marking StandingTackle SlidingTackle GKDiving GKHandling GKKicking  \
0    83.0           85.0          84.0     12.0        5.0       8.0
1    10.0           13.0          10.0     63.0       61.0      60.0
2    16.0           14.0          17.0      5.0        6.0      13.0
3    24.0           34.0          15.0     16.0       13.0      13.0
4    13.0           11.0          13.0     60.0       57.0      60.0

  GKPositioning GKReflexes Release Clause
0          11.0       12.0         €52.7M
1          58.0       62.0          €347K
2          10.0        8.0          €264K
3           9.0        9.0          €1.3M
4          56.0       55.0          €267K

[5 rows x 114 columns]
```

[75]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import scipy.stats as st
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PowerTransformer
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score
from sklearn.metrics import r2_score
pd.pandas.set_option('display.max_columns',None)
pd.options.display.max_rows=100
import warnings
warnings.filterwarnings("ignore")
from tqdm import tqdm
```

[76]:
```
combined_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 904 entries, 0 to 903
Columns: 114 entries, player_id to Release Clause
dtypes: datetime64[ns](1), float64(43), int64(9), object(61)
memory usage: 812.2+ KB
```

[77]:
```
combined_df.head(20)
```

[77]:

	player_id	name	current_club_id	\
0	192366	Gozie Ugwu	1032	
1	240063	Georgios Koudas	5219	
2	240074	Christos Armenis	5219	
3	187915	Panagiotis Petrousis	5219	
4	234324	Danny Cavanagh	511	
5	193105	James Thomson	511	
6	201455	Mathias Barthel	1177	
7	197154	Jô	979	
8	211305	Nilson Júnior	979	
9	53842	Jorge Luiz	1085	
10	237291	Abdulkadir Korkut	3216	
11	238230	Kerem Yigit	2292	
12	237655	Viktor Ryashko	6996	
13	50632	André Martins	2451	
14	205676	Federico Gagliardini	506	
15	199598	Joyce Anacoura	130	
16	176442	Taha Can Velioglu	20	
17	240190	Niccolò Belloni	46	
18	240289	Marco Ferrara	46	
19	199584	Gianluca Austoni	1038	

```
                current_club_name country_of_citizenship country_of_birth  \
0                       Fc Reading                England          England
1                      Aok Kerkyra                 Greece           Greece
2                      Aok Kerkyra                 Greece           Greece
3                      Aok Kerkyra                 Greece           Greece
4                        Dundee Fc               Scotland         Scotland
5                        Dundee Fc               Scotland         Scotland
6                     Silkeborg If                Denmark           France
7                    Moreirense Fc                 Brazil           Brazil
8                    Moreirense Fc                 Brazil           Brazil
9                Vitoria Setubal Fc                Brazil           Brazil
10                Mersin Idmanyurdu                Turkey           Turkey
11                       Elazigspor                Turkey           Turkey
12                 Goverla Uzhgorod               Ukraine          Ukraine
13  Inverness Caledonian Thistle Fc               Portugal         Portugal
14                   Juventus Turin                  Italy            Italy
15                Parma Calcio 1913                  Italy            Italy
16                        Bursaspor                 Turkey           Turkey
17                   Inter Mailand                  Italy            Italy
18                   Inter Mailand                  Italy            Italy
19                  Sampdoria Genua                 Italy            Italy

                    city_of_birth date_of_birth    position  \
0                          Oxford      1993-04-22      Attack
1                         Kerkyra      1994-12-27    Midfield
2                       Velonades      1993-06-22    Defender
3                       Giannitsa      1992-08-11    Defender
4                          Dundee      1994-05-09    Midfield
5                          Dundee      1994-11-07    Defender
6                          London      1993-05-30      Attack
7                       Rio Branco      1988-09-19      Attack
8                          Maceió      1991-09-09    Defender
9                  Rio de Janeiro      1982-04-22    Defender
10                         Mersin      1993-11-07    Defender
11                         Elazig      1995-07-10    Midfield
12  Mukacheve, Zakarpattya Region      1992-11-27    Midfield
13           Ferreira do Alentejo      1987-03-26      Attack
14                         Ancona      1994-01-24  Goalkeeper
15                         Milano      1994-08-01  Goalkeeper
16                        Sakarya      1994-02-21    Defender
17                          Massa      1994-07-10      Attack
18                         Milano      1994-05-05    Defender
19                       Calcinate      1994-02-04      Attack

          sub_position   foot  height_in_cm  market_value_in_eur  \
0       Centre-Forward  Right           188             200000.0
1        Left Midfield   Left           175             200000.0
```

```
2            Left-Back   Left           176              200000.0
3          Centre-Back  Right           186              200000.0
4       Right Midfield  Right             0              200000.0
5          Centre-Back  Right           180              200000.0
6       Centre-Forward  Right           178              200000.0
7       Second Striker   Both           170              200000.0
8          Centre-Back   Left           190              250000.0
9          Centre-Back  Right           192              200000.0
10          Right-Back  Right           180              125000.0
11    Central Midfield  Right             0              200000.0
12  Defensive Midfield  Right           188              200000.0
13      Centre-Forward   Both           180              200000.0
14         Centre-Back  Right           185               50000.0
15         Centre-Back  Right           190              100000.0
16         Centre-Back  Right           187              100000.0
17        Right Winger   Left           176              300000.0
18           Left-Back   Left           181              200000.0
19      Centre-Forward   Left             0              200000.0

    highest_market_value_in_eur                          agent_name  \
0                      250000.0           RT Management & Consultancy
1                      100000.0                            Wasserman
2                      125000.0                            Wasserman
3                      150000.0                            Wasserman
4                      300000.0                            Wasserman
5                       25000.0                            Wasserman
6                       50000.0                            Wasserman
7                      850000.0                          Eurofoot4all
8                      250000.0                            Wasserman
9                     2000000.0                            Wasserman
10                     300000.0                            Wasserman
11                      50000.0                            Wasserman
12                      75000.0                            Wasserman
13                     400000.0                            Wasserman
14                      50000.0  I.F.A. di Giuseppe Bonetto & C. S.A.S.
15                     150000.0               Model T Sport Management
16                     300000.0                          Gurel Bergen
17                     500000.0                            Wasserman
18                     200000.0                            Wasserman
19                      75000.0                            Wasserman

   contract_expiration_date current_club_domestic_competition_id  first_name  \
0                2023-06-01                                  GB1       Gozie
1                2023-06-30                                  GR1    Georgios
2                2023-06-30                                  GR1    Christos
3                2023-06-30                                  GR1  Panagiotis
4                2023-06-30                                  SC1       Danny
```

5	2023-06-30		SC1	James
6	2023-06-30		DK1	Mathias
7	2023-06-30		PO1	David
8	2024-06-30		PO1	Nilson
9	2023-06-30		PO1	David
10	2025-06-30		TR1	Abdulkadir
11	2023-06-30		TR1	Kerem
12	2023-06-30		UKR1	Viktor
13	2023-06-30		SC1	André
14	2023-06-30		IT1	Federico
15	2023-06-30		IT1	Joyce
16	2024-06-30		TR1	Taha Can
17	2026-06-30		IT1	Niccolò
18	2023-06-30		IT1	Marco
19	2023-06-30		IT1	Gianluca

	last_name	player_code	\
0	Ugwu	gozie-ugwu	
1	Koudas	georgios-koudas	
2	Armenis	christos-armenis	
3	Petrousis	panagiotis-petrousis	
4	Cavanagh	danny-cavanagh	
5	Thomson	james-thomson	
6	Barthel	mathias-barthel	
7	Jô	jotm	
8	Júnior	nilson-junior	
9	Jorge Luiz	jorge-luiz	
10	Korkut	abdulkadir-korkut	
11	Yigit	kerem-yigit	
12	Ryashko	viktor-ryashko	
13	Martins	andre-martins	
14	Gagliardini	federico-gagliardini	
15	Anacoura	joyce-anacoura	
16	Velioglu	taha-can-velioglu	
17	Belloni	niccolo-belloni	
18	Ferrara	marco-ferrara	
19	Austoni	gianluca-austoni	

	image_url	last_season	\
0	https://img.a.transfermarkt.technology/portrai…	2012	
1	https://img.a.transfermarkt.technology/portrai…	2012	
2	https://img.a.transfermarkt.technology/portrai…	2012	
3	https://img.a.transfermarkt.technology/portrai…	2012	
4	https://img.a.transfermarkt.technology/portrai…	2012	
5	https://img.a.transfermarkt.technology/portrai…	2012	
6	https://img.a.transfermarkt.technology/portrai…	2012	
7	https://img.a.transfermarkt.technology/portrai…	2012	

```
8   https://img.a.transfermarkt.technology/portrai…         2012
9   https://img.a.transfermarkt.technology/portrai…         2012
10  https://img.a.transfermarkt.technology/portrai…         2012
11  https://img.a.transfermarkt.technology/portrai…         2012
12  https://img.a.transfermarkt.technology/portrai…         2012
13  https://img.a.transfermarkt.technology/portrai…         2012
14  https://img.a.transfermarkt.technology/portrai…         2012
15  https://img.a.transfermarkt.technology/portrai…         2012
16  https://img.a.transfermarkt.technology/portrai…         2012
17  https://img.a.transfermarkt.technology/portrai…         2012
18  https://img.a.transfermarkt.technology/portrai…         2012
19  https://img.a.transfermarkt.technology/portrai…         2012

                                                  url  performance_score  \
0   https://www.transfermarkt.co.uk/gozie-ugwu/pro…          65.502135
1   https://www.transfermarkt.co.uk/georgios-kouda…          87.279612
2   https://www.transfermarkt.co.uk/christos-armen…          88.184968
3   https://www.transfermarkt.co.uk/panagiotis-pet…          62.654775
4   https://www.transfermarkt.co.uk/danny-cavanagh…          84.862125
5   https://www.transfermarkt.co.uk/james-thomson/…          60.165664
6   https://www.transfermarkt.co.uk/mathias-barthe…          86.776770
7   https://www.transfermarkt.co.uk/jotm/profil/sp…          60.208564
8   https://www.transfermarkt.co.uk/nilson-junior/…          70.908888
9   https://www.transfermarkt.co.uk/jorge-luiz/pro…          77.726788
10  https://www.transfermarkt.co.uk/abdulkadir-kor…          88.908599
11  https://www.transfermarkt.co.uk/kerem-yigit/pr…          61.550452
12  https://www.transfermarkt.co.uk/viktor-ryashko…          61.221864
13  https://www.transfermarkt.co.uk/andre-martins/…          65.196056
14  https://www.transfermarkt.co.uk/federico-gagli…          62.135659
15  https://www.transfermarkt.co.uk/joyce-anacoura…          78.546542
16  https://www.transfermarkt.co.uk/taha-can-velio…          76.082891
17  https://www.transfermarkt.co.uk/niccolo-bellon…          87.327816
18  https://www.transfermarkt.co.uk/marco-ferrara/…          87.981855
19  https://www.transfermarkt.co.uk/gianluca-austo…          89.618284

    market_value_growth   age  Unnamed: 0             Name  Age  \
0             100.484926  29.0          89      N. Otamendi   30
1              79.327530  28.0       15122       M. Cassara   26
2             105.998425  29.0       16585         B. Dione   21
3              86.434060  30.0        7693       E. Kujović   30
4             155.647507  28.0       16395        L. Bilboe   20
5              87.980523  28.0         372        A. Areola   25
6             102.322821  29.0         262     G. Kondogbia   25
7              39.379564  34.0        4895       A. Machado   33
8             120.315284  31.0       13432     A. Tabanelli   28
9              81.761424  40.0       14858         M. Tonge   35
10            150.737270  29.0        3159        G. Blanco   26
```

```
11            50.892351  27.0       17644        J. Campaz   18
12            46.991886  30.0        8040       B. De Alba   25
13            67.342457  35.0       11825        P. Clarke   36
14            39.177147  28.0        8125            Tekio   27
15           127.090302  28.0        9564      Jony Ñíguez   33
16           106.342497  28.0       12297        R. Milsom   31
17           134.478750  28.0        3408  P. Kunde Malong   22
18           154.959573  28.0        2993           Calero   22
19            71.287057  28.0        5312   M. Kieftenbeld   28

                                                 Photo           Nationality  \
0   https://cdn.sofifa.org/players/4/19/192366.png             Argentina
1   https://cdn.sofifa.org/players/4/19/240063.png                France
2   https://cdn.sofifa.org/players/4/19/240074.png               Belgium
3   https://cdn.sofifa.org/players/4/19/187915.png                Sweden
4   https://cdn.sofifa.org/players/4/19/234324.png               England
5   https://cdn.sofifa.org/players/4/19/193105.png                France
6   https://cdn.sofifa.org/players/4/19/201455.png  Central African Rep.
7   https://cdn.sofifa.org/players/4/19/197154.png                Panama
8   https://cdn.sofifa.org/players/4/19/211305.png                 Italy
9    https://cdn.sofifa.org/players/4/19/53842.png               England
10  https://cdn.sofifa.org/players/4/19/237291.png             Argentina
11  https://cdn.sofifa.org/players/4/19/238230.png              Colombia
12  https://cdn.sofifa.org/players/4/19/237655.png              Colombia
13   https://cdn.sofifa.org/players/4/19/50632.png               England
14  https://cdn.sofifa.org/players/4/19/205676.png                 Spain
15  https://cdn.sofifa.org/players/4/19/199598.png                 Spain
16  https://cdn.sofifa.org/players/4/19/176442.png               England
17  https://cdn.sofifa.org/players/4/19/240190.png              Cameroon
18  https://cdn.sofifa.org/players/4/19/240289.png                 Spain
19  https://cdn.sofifa.org/players/4/19/199584.png           Netherlands

                                    Flag  Overall  Potential  \
0    https://cdn.sofifa.org/flags/52.png       85         85
1    https://cdn.sofifa.org/flags/18.png       60         64
2     https://cdn.sofifa.org/flags/7.png       57         66
3    https://cdn.sofifa.org/flags/46.png       67         67
4    https://cdn.sofifa.org/flags/14.png       57         67
5    https://cdn.sofifa.org/flags/18.png       81         85
6   https://cdn.sofifa.org/flags/105.png       82         85
7    https://cdn.sofifa.org/flags/87.png       70         70
8    https://cdn.sofifa.org/flags/27.png       62         62
9    https://cdn.sofifa.org/flags/14.png       60         60
10   https://cdn.sofifa.org/flags/52.png       73         74
11   https://cdn.sofifa.org/flags/56.png       53         75
12   https://cdn.sofifa.org/flags/56.png       67         72
13   https://cdn.sofifa.org/flags/14.png       64         64
```

```
14   https://cdn.sofifa.org/flags/45.png       67         67
15   https://cdn.sofifa.org/flags/45.png       66         66
16   https://cdn.sofifa.org/flags/14.png       63         63
17  https://cdn.sofifa.org/flags/103.png       72         79
18   https://cdn.sofifa.org/flags/45.png       73         82
19   https://cdn.sofifa.org/flags/34.png       70         71

                   Club                                      Club Logo  \
0        Manchester City      https://cdn.sofifa.org/teams/2/light/10.png
1            GFC Ajaccio  https://cdn.sofifa.org/teams/2/light/110316.png
2   Royal Excel Mouscron  https://cdn.sofifa.org/teams/2/light/111560.png
3     Fortuna Düsseldorf  https://cdn.sofifa.org/teams/2/light/110636.png
4       Rotherham United    https://cdn.sofifa.org/teams/2/light/1797.png
5    Paris Saint-Germain      https://cdn.sofifa.org/teams/2/light/73.png
6            Valencia CF     https://cdn.sofifa.org/teams/2/light/461.png
7         Houston Dynamo     https://cdn.sofifa.org/teams/2/light/698.png
8                  Lecce     https://cdn.sofifa.org/teams/2/light/347.png
9              Port Vale    https://cdn.sofifa.org/teams/2/light/1928.png
10             Málaga CF     https://cdn.sofifa.org/teams/2/light/573.png
11       Deportes Tolima  https://cdn.sofifa.org/teams/2/light/111722.png
12            La Equidad  https://cdn.sofifa.org/teams/2/light/112523.png
13       Oldham Athletic    https://cdn.sofifa.org/teams/2/light/1920.png
14             Elche CF     https://cdn.sofifa.org/teams/2/light/468.png
15             Elche CF     https://cdn.sofifa.org/teams/2/light/468.png
16          Notts County    https://cdn.sofifa.org/teams/2/light/1937.png
17       1. FSV Mainz 05     https://cdn.sofifa.org/teams/2/light/169.png
18    Real Valladolid CF     https://cdn.sofifa.org/teams/2/light/462.png
19       Birmingham City      https://cdn.sofifa.org/teams/2/light/88.png

     Value   Wage  Special Preferred Foot  International Reputation  \
0   €28.5M  €170K     1916          Right                       3.0
1    €190K    €1K      910          Right                       1.0
2    €160K    €1K     1366          Right                       1.0
3    €725K   €11K     1537          Right                       1.0
4    €130K    €2K      878          Right                       1.0
5   €16.5M   €61K     1266          Right                       2.0
6   €27.5M   €46K     2171           Left                       2.0
7    €725K    €5K     1569          Right                       1.0
8    €300K    €1K     1590          Right                       1.0
9     €90K    €2K     1621          Right                       2.0
10   €4.8M   €10K     1567          Right                       1.0
11   €130K    €1K     1342           Left                       1.0
12     €1M    €1K     1649           Left                       1.0
13    €60K    €2K     1414          Right                       1.0
14   €625K    €3K     1622          Right                       1.0
15   €260K    €2K     1774          Right                       1.0
16   €325K    €2K     1840           Left                       1.0
```

17	€3.7M	€15K	1944	Right		1.0
18	€5.5M	€13K	1645	Right		1.0
19	€1.6M	€15K	1937	Right		1.0

	Weak Foot	Skill Moves	Work Rate	Body Type	Real Face	Position	\
0	3.0	2.0	High/ High	Normal	Yes	CB	
1	3.0	1.0	Medium/ Medium	Normal	No	GK	
2	3.0	3.0	Medium/ Medium	Normal	No	ST	
3	4.0	2.0	Low/ Low	Normal	No	ST	
4	3.0	1.0	Medium/ Medium	Normal	No	GK	
5	3.0	1.0	Medium/ Medium	Normal	Yes	GK	
6	2.0	4.0	High/ Medium	Lean	Yes	CM	
7	2.0	2.0	Low/ High	Normal	No	CB	
8	3.0	2.0	Medium/ Medium	Lean	No	CM	
9	4.0	2.0	Medium/ High	Normal	No	CM	
10	3.0	2.0	High/ Medium	Normal	No	LS	
11	2.0	2.0	Medium/ Medium	Normal	No	LM	
12	2.0	3.0	Medium/ Medium	Lean	No	CAM	
13	3.0	2.0	Medium/ High	Normal	No	LCB	
14	3.0	2.0	Medium/ Medium	Normal	No	RB	
15	2.0	2.0	Medium/ Medium	Normal	No	CDM	
16	2.0	2.0	High/ High	Lean	No	LDM	
17	4.0	3.0	Medium/ Medium	Stocky	No	CDM	
18	4.0	2.0	Medium/ Medium	Lean	No	LCB	
19	3.0	3.0	Medium/ High	Normal	No	LCM	

	Jersey Number	Joined	Loaned From	Contract Valid Until	Height	\
0	30.0	Aug 20, 2015	NaN	2022	6'0	
1	1.0	Jun 30, 2017	NaN	2019	6'3	
2	17.0	Jul 1, 2017	NaN	2020	5'6	
3	21.0	Jul 19, 2017	NaN	2020	6'4	
4	30.0	Mar 22, 2016	NaN	2019	6'3	
5	23.0	Jul 1, 2010	NaN	2019	6'5	
6	6.0	Jul 2, 2018	NaN	2022	6'2	
7	3.0	Dec 21, 2016	NaN	2022	5'11	
8	23.0	Jan 31, 2018	NaN	2019	6'4	
9	18.0	Aug 4, 2017	NaN	2019	6'0	
10	9.0	Jan 31, 2017	NaN	2020	6'3	
11	28.0	Apr 1, 2017	NaN	2021	5'7	
12	7.0	Jan 1, 2017	NaN	2021	5'6	
13	26.0	Jul 19, 2016	NaN	2019	6'0	
14	2.0	Jul 1, 2017	NaN	2020	5'9	
15	20.0	Jan 1, 2018	NaN	2019	5'9	
16	24.0	NaN	Crawley Town	Jan 1, 2019	5'10	
17	13.0	Jul 6, 2018	NaN	2022	5'11	
18	5.0	Jul 1, 2016	NaN	2021	6'0	
19	6.0	Jul 27, 2015	NaN	2020	5'10	

	Weight	LS	ST	RS	LW	LF	CF	RF	RW	LAM	CAM	RAM	\
0	179lbs	64+3	64+3	64+3	59+3	61+3	61+3	61+3	59+3	62+3	62+3	62+3	
1	201lbs	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
2	137lbs	55+2	55+2	55+2	55+2	57+2	57+2	57+2	55+2	56+2	56+2	56+2	
3	194lbs	66+2	66+2	66+2	55+2	60+2	60+2	60+2	55+2	57+2	57+2	57+2	
4	174lbs	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
5	207lbs	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
6	176lbs	77+2	77+2	77+2	76+2	78+2	78+2	78+2	76+2	78+2	78+2	78+2	
7	157lbs	49+2	49+2	49+2	49+2	50+2	50+2	50+2	49+2	52+2	52+2	52+2	
8	172lbs	57+2	57+2	57+2	58+2	58+2	58+2	58+2	58+2	60+2	60+2	60+2	
9	163lbs	55+2	55+2	55+2	56+2	56+2	56+2	56+2	56+2	58+2	58+2	58+2	
10	194lbs	72+2	72+2	72+2	63+2	67+2	67+2	67+2	63+2	63+2	63+2	63+2	
11	150lbs	48+2	48+2	48+2	51+2	50+2	50+2	50+2	51+2	50+2	50+2	50+2	
12	143lbs	58+2	58+2	58+2	65+2	64+2	64+2	64+2	65+2	65+2	65+2	65+2	
13	183lbs	45+2	45+2	45+2	40+2	43+2	43+2	43+2	40+2	45+2	45+2	45+2	
14	159lbs	49+2	49+2	49+2	51+2	48+2	48+2	48+2	51+2	48+2	48+2	48+2	
15	159lbs	58+2	58+2	58+2	57+2	59+2	59+2	59+2	57+2	60+2	60+2	60+2	
16	159lbs	61+2	61+2	61+2	60+2	60+2	60+2	60+2	60+2	60+2	60+2	60+2	
17	174lbs	68+2	68+2	68+2	69+2	69+2	69+2	69+2	69+2	69+2	69+2	69+2	
18	165lbs	51+2	51+2	51+2	55+2	54+2	54+2	54+2	55+2	58+2	58+2	58+2	
19	172lbs	64+2	64+2	64+2	63+2	63+2	63+2	63+2	63+2	63+2	63+2	63+2	

	LM	LCM	CM	RCM	RM	LWB	LDM	CDM	RDM	RWB	LB	LCB	\
0	61+3	67+3	67+3	67+3	61+3	72+3	77+3	77+3	77+3	72+3	74+3	82+3	
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
2	54+2	49+2	49+2	49+2	54+2	37+2	35+2	35+2	35+2	37+2	34+2	30+2	
3	54+2	55+2	55+2	55+2	54+2	39+2	45+2	45+2	45+2	39+2	38+2	44+2	
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
6	77+2	80+2	80+2	80+2	77+2	79+2	82+2	82+2	82+2	79+2	79+2	82+2	
7	51+2	55+2	55+2	55+2	51+2	62+2	64+2	64+2	64+2	62+2	63+2	68+2	
8	59+2	61+2	61+2	61+2	59+2	55+2	57+2	57+2	57+2	55+2	53+2	52+2	
9	56+2	58+2	58+2	58+2	56+2	54+2	56+2	56+2	56+2	54+2	53+2	54+2	
10	60+2	54+2	54+2	54+2	60+2	45+2	43+2	43+2	43+2	45+2	43+2	44+2	
11	51+2	45+2	45+2	45+2	51+2	43+2	39+2	39+2	39+2	43+2	41+2	34+2	
12	66+2	60+2	60+2	60+2	66+2	53+2	49+2	49+2	49+2	53+2	49+2	39+2	
13	42+2	50+2	50+2	50+2	42+2	50+2	57+2	57+2	57+2	50+2	52+2	62+2	
14	54+2	53+2	53+2	53+2	54+2	65+2	62+2	62+2	62+2	65+2	66+2	65+2	
15	58+2	62+2	62+2	62+2	58+2	61+2	65+2	65+2	65+2	61+2	61+2	64+2	
16	61+2	61+2	61+2	61+2	61+2	62+2	63+2	63+2	63+2	62+2	62+2	63+2	
17	70+2	70+2	70+2	70+2	70+2	70+2	71+2	71+2	71+2	70+2	69+2	69+2	
18	59+2	63+2	63+2	63+2	59+2	67+2	70+2	70+2	70+2	67+2	69+2	71+2	
19	64+2	65+2	65+2	65+2	64+2	68+2	69+2	69+2	69+2	68+2	68+2	69+2	

	CB	RCB	RB	Crossing	Finishing	HeadingAccuracy	ShortPassing	\
0	82+3	82+3	74+3	52.0	54.0	85.0	75.0	

1	NaN	NaN	NaN	13.0	8.0	13.0	18.0
2	30+2	30+2	34+2	33.0	57.0	49.0	53.0
3	44+2	44+2	38+2	33.0	69.0	76.0	60.0
4	NaN	NaN	NaN	14.0	6.0	10.0	26.0
5	NaN	NaN	NaN	20.0	19.0	14.0	41.0
6	82+2	82+2	79+2	65.0	69.0	75.0	81.0
7	68+2	68+2	63+2	33.0	27.0	62.0	60.0
8	52+2	52+2	53+2	54.0	42.0	54.0	66.0
9	54+2	54+2	53+2	63.0	53.0	49.0	62.0
10	44+2	44+2	43+2	29.0	77.0	80.0	67.0
11	34+2	34+2	41+2	46.0	46.0	39.0	48.0
12	39+2	39+2	49+2	62.0	50.0	35.0	68.0
13	62+2	62+2	52+2	25.0	32.0	66.0	50.0
14	65+2	65+2	66+2	69.0	36.0	58.0	56.0
15	64+2	64+2	61+2	42.0	45.0	45.0	65.0
16	63+2	63+2	62+2	60.0	61.0	59.0	63.0
17	69+2	69+2	69+2	68.0	64.0	54.0	75.0
18	71+2	71+2	69+2	44.0	23.0	72.0	69.0
19	69+2	69+2	68+2	63.0	58.0	59.0	68.0

	Volleys	Dribbling	Curve	FKAccuracy	LongPassing	BallControl	\
0	57.0	51.0	50.0	39.0	72.0	70.0	
1	8.0	6.0	14.0	13.0	22.0	12.0	
2	50.0	65.0	43.0	29.0	42.0	63.0	
3	69.0	58.0	66.0	70.0	54.0	65.0	
4	9.0	11.0	10.0	13.0	36.0	12.0	
5	16.0	12.0	16.0	16.0	32.0	22.0	
6	59.0	78.0	62.0	56.0	79.0	82.0	
7	19.0	45.0	27.0	33.0	40.0	59.0	
8	58.0	63.0	62.0	52.0	63.0	62.0	
9	46.0	54.0	66.0	63.0	61.0	63.0	
10	54.0	63.0	30.0	39.0	22.0	60.0	
11	43.0	53.0	37.0	35.0	41.0	52.0	
12	46.0	67.0	67.0	72.0	64.0	66.0	
13	13.0	35.0	38.0	37.0	48.0	44.0	
14	20.0	48.0	56.0	25.0	56.0	53.0	
15	51.0	53.0	53.0	51.0	60.0	58.0	
16	56.0	59.0	60.0	65.0	62.0	58.0	
17	44.0	69.0	62.0	40.0	70.0	74.0	
18	28.0	61.0	39.0	27.0	70.0	72.0	
19	74.0	65.0	57.0	62.0	65.0	65.0	

	Acceleration	SprintSpeed	Agility	Reactions	Balance	ShotPower	\
0	57.0	61.0	64.0	79.0	62.0	69.0	
1	32.0	31.0	32.0	43.0	22.0	23.0	
2	66.0	58.0	63.0	48.0	74.0	61.0	
3	39.0	36.0	34.0	59.0	32.0	74.0	

4	34.0	38.0	28.0	43.0	22.0	18.0
5	54.0	52.0	53.0	77.0	58.0	25.0
6	69.0	76.0	76.0	84.0	67.0	86.0
7	62.0	60.0	59.0	73.0	62.0	42.0
8	48.0	64.0	73.0	62.0	38.0	61.0
9	43.0	37.0	56.0	64.0	56.0	64.0
10	69.0	67.0	53.0	69.0	45.0	72.0
11	71.0	64.0	70.0	42.0	81.0	49.0
12	79.0	82.0	79.0	65.0	88.0	64.0
13	32.0	33.0	49.0	62.0	32.0	50.0
14	66.0	71.0	58.0	54.0	72.0	54.0
15	64.0	61.0	70.0	67.0	73.0	75.0
16	63.0	66.0	69.0	62.0	70.0	65.0
17	78.0	78.0	69.0	68.0	72.0	79.0
18	67.0	69.0	68.0	73.0	54.0	40.0
19	66.0	67.0	68.0	64.0	72.0	80.0

	Jumping	Stamina	Strength	LongShots	Aggression	Interceptions	\
0	92.0	67.0	80.0	56.0	91.0	84.0	
1	71.0	29.0	72.0	9.0	21.0	13.0	
2	65.0	49.0	49.0	53.0	28.0	13.0	
3	42.0	50.0	90.0	67.0	54.0	21.0	
4	38.0	27.0	65.0	5.0	23.0	10.0	
5	59.0	28.0	80.0	14.0	26.0	23.0	
6	83.0	85.0	91.0	80.0	86.0	84.0	
7	67.0	73.0	77.0	37.0	75.0	76.0	
8	35.0	68.0	75.0	59.0	55.0	54.0	
9	52.0	40.0	55.0	59.0	53.0	50.0	
10	60.0	64.0	82.0	70.0	48.0	25.0	
11	33.0	68.0	43.0	50.0	33.0	20.0	
12	48.0	64.0	39.0	53.0	31.0	24.0	
13	71.0	62.0	80.0	43.0	69.0	63.0	
14	73.0	90.0	70.0	39.0	77.0	63.0	
15	75.0	71.0	68.0	65.0	77.0	68.0	
16	77.0	77.0	78.0	59.0	68.0	64.0	
17	73.0	78.0	84.0	74.0	73.0	70.0	
18	75.0	67.0	67.0	21.0	63.0	75.0	
19	80.0	80.0	70.0	70.0	80.0	70.0	

	Positioning	Vision	Penalties	Composure	Marking	StandingTackle	\
0	51.0	53.0	45.0	80.0	83.0	85.0	
1	7.0	29.0	12.0	36.0	10.0	13.0	
2	49.0	55.0	62.0	47.0	16.0	14.0	
3	70.0	47.0	69.0	68.0	24.0	34.0	
4	5.0	34.0	16.0	37.0	13.0	11.0	
5	17.0	34.0	25.0	64.0	11.0	18.0	
6	78.0	78.0	45.0	82.0	79.0	83.0	

7	41.0	56.0	44.0	75.0	67.0	68.0
8	58.0	63.0	54.0	60.0	41.0	46.0
9	56.0	62.0	60.0	74.0	50.0	58.0
10	81.0	49.0	66.0	62.0	29.0	24.0
11	45.0	48.0	50.0	47.0	45.0	22.0
12	62.0	67.0	52.0	55.0	59.0	22.0
13	43.0	56.0	37.0	63.0	65.0	60.0
14	35.0	40.0	36.0	62.0	70.0	66.0
15	61.0	64.0	50.0	54.0	68.0	64.0
16	61.0	60.0	66.0	67.0	56.0	63.0
17	61.0	65.0	57.0	60.0	68.0	71.0
18	35.0	57.0	39.0	70.0	73.0	73.0
19	55.0	61.0	58.0	66.0	68.0	72.0

	SlidingTackle	GKDiving	GKHandling	GKKicking	GKPositioning	GKReflexes	\
0	84.0	12.0	5.0	8.0	11.0	12.0	
1	10.0	63.0	61.0	60.0	58.0	62.0	
2	17.0	5.0	6.0	13.0	10.0	8.0	
3	15.0	16.0	13.0	13.0	9.0	9.0	
4	13.0	60.0	57.0	60.0	56.0	55.0	
5	12.0	83.0	78.0	70.0	77.0	84.0	
6	82.0	15.0	10.0	10.0	8.0	10.0	
7	66.0	14.0	14.0	14.0	7.0	10.0	
8	40.0	4.0	4.0	4.0	4.0	4.0	
9	55.0	15.0	11.0	16.0	16.0	13.0	
10	17.0	8.0	11.0	14.0	13.0	10.0	
11	28.0	10.0	12.0	6.0	5.0	7.0	
12	25.0	9.0	14.0	8.0	10.0	8.0	
13	59.0	11.0	10.0	14.0	10.0	15.0	
14	71.0	9.0	9.0	8.0	6.0	8.0	
15	58.0	13.0	7.0	16.0	10.0	6.0	
16	56.0	13.0	12.0	6.0	10.0	16.0	
17	60.0	11.0	13.0	5.0	8.0	9.0	
18	72.0	11.0	13.0	7.0	14.0	7.0	
19	72.0	13.0	7.0	12.0	10.0	6.0	

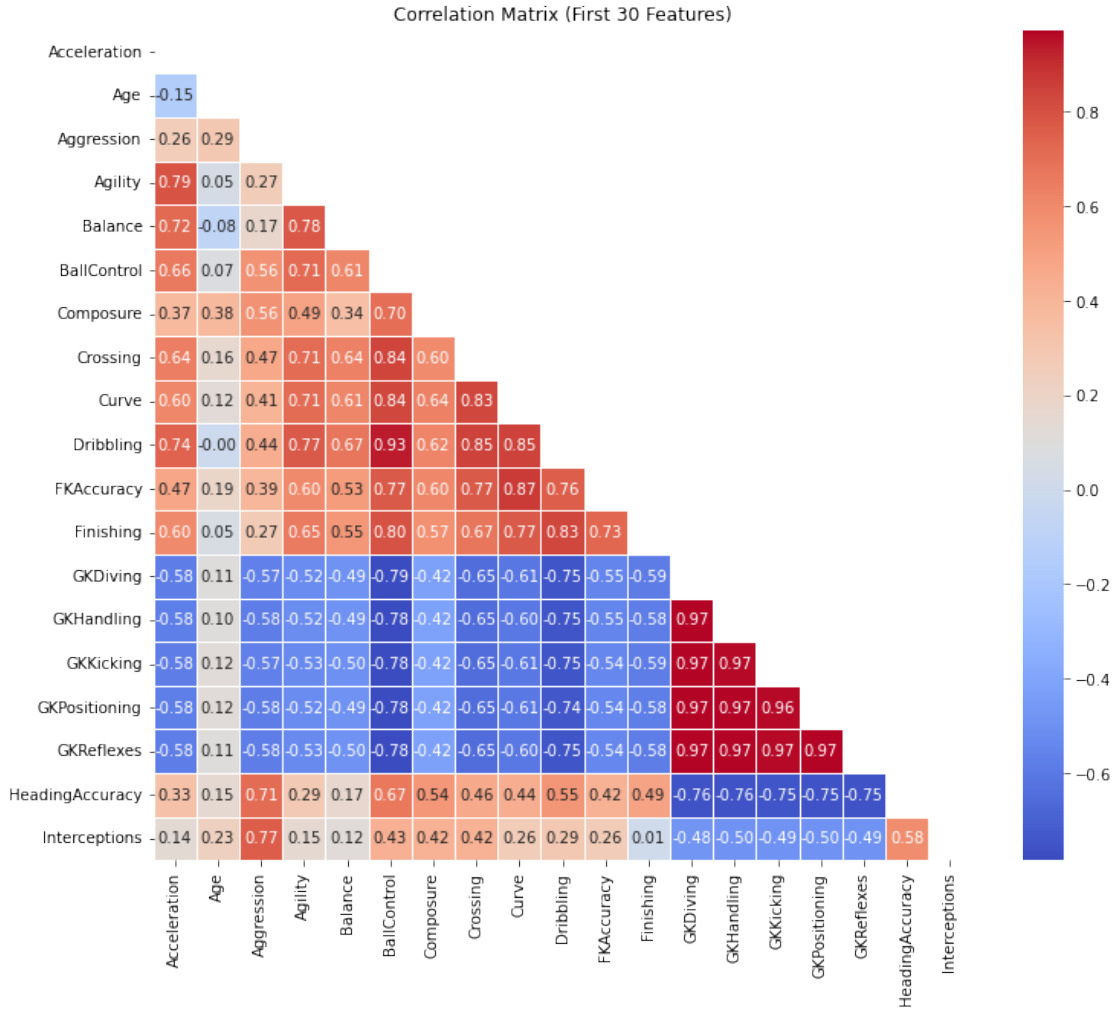
	Release Clause
0	€52.7M
1	€347K
2	€264K
3	€1.3M
4	€267K
5	€31.8M
6	€59.8M
7	€1.1M
8	€465K
9	€158K

```
10            €8M
11          €312K
12          €1.4M
13          €105K
14          €969K
15          €403K
16            NaN
17          €7.5M
18         €13.2M
19          €3.2M
```

[78]:
```
## HOW TO EVALUATE A PLAYERS MARKET VALUE?
#  This part of the analysis is to evaluate price.
```

[79]:
```
# POTENTIAL PREDICTORS OF MARKET VALUE
```

[80]:
```
combined_df = combined_df.drop_duplicates(subset='player_id', keep='first')
features = combined_df.columns.difference(['market_value_in_eur'])
target_variable = 'market_value_in_eur'
selected_features = features[:30]
correlation_matrix = combined_df[selected_features].corr()
mask = np.triu(np.ones_like(correlation_matrix, dtype=bool))
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, mask=mask, annot=True, cmap='coolwarm', fmt=".
↪2f", linewidths=.5)
plt.title("Correlation Matrix (First 30 Features)")
plt.show()
```



[81]:
```
## CLEANING DATASET 2
# MISSING VALUES
```

[82]:
```
combined_df.isnull().sum()
```

[82]:
```
player_id                 0
name                      0
current_club_id           0
current_club_name         0
country_of_citizenship    0
                         ..
GKHandling                1
GKKicking                 1
GKPositioning             1
GKReflexes                1
Release Clause           79
```

```
Length: 114, dtype: int64
```

[83]:
```
# FILLING MISSING VALUES OF COMBINED_DF
numerical_columns = combined_df.select_dtypes(include=['float64', 'int64']).
 ↪columns
combined_df[numerical_columns] = combined_df[numerical_columns].
 ↪fillna(combined_df[numerical_columns].mode().iloc[0])
categorical_columns = combined_df.select_dtypes(include=['object']).columns
combined_df[categorical_columns] = combined_df[categorical_columns].
 ↪apply(lambda x: x.fillna(x.value_counts().idxmax()))
combined_df.head()
```

[83]:

	player_id	name	current_club_id	current_club_name	country_of_citizenship	country_of_birth	city_of_birth	date_of_birth	position	sub_position	foot	height_in_cm	market_value_in_eur	highest_market_value_in_eur	agent_name	contract_expiration_date	current_club_domestic_competition_id	first_name
0	192366	Gozie Ugwu	1032	Fc Reading	England	England	Oxford	1993-04-22	Attack	Centre-Forward	Right	188	200000.0	250000.0	RT Management & Consultancy	2023-06-01	GB1	Gozie
1	240063	Georgios Koudas	5219	Aok Kerkyra	Greece	Greece	Kerkyra	1994-12-27	Midfield	Left Midfield	Left	175	200000.0	100000.0	Wasserman	2023-06-30	GR1	Georgios
2	240074	Christos Armenis	5219	Aok Kerkyra	Greece	Greece	Velonades	1993-06-22	Defender	Left-Back	Left	176	200000.0	125000.0	Wasserman	2023-06-30	GR1	Christos
3	187915	Panagiotis Petrousis	5219	Aok Kerkyra	Greece	Greece	Giannitsa	1992-08-11	Defender	Centre-Back	Right	186	200000.0	150000.0	Wasserman	2023-06-30	GR1	Panagiotis
4	234324	Danny Cavanagh	511	Dundee Fc	Scotland	Scotland	Dundee	1994-05-09	Midfield	Right Midfield	Right	0	200000.0	300000.0	Wasserman	2023-06-30	SC1	Danny

```
   last_name           player_code  \
0       Ugwu            gozie-ugwu
1     Koudas       georgios-koudas
2    Armenis      christos-armenis
3  Petrousis  panagiotis-petrousis
4   Cavanagh         danny-cavanagh

                                          image_url  last_season  \
0  https://img.a.transfermarkt.technology/portrai…         2012
1  https://img.a.transfermarkt.technology/portrai…         2012
2  https://img.a.transfermarkt.technology/portrai…         2012
3  https://img.a.transfermarkt.technology/portrai…         2012
4  https://img.a.transfermarkt.technology/portrai…         2012

                                              url  performance_score  \
0  https://www.transfermarkt.co.uk/gozie-ugwu/pro…          65.502135
1  https://www.transfermarkt.co.uk/georgios-kouda…          87.279612
2  https://www.transfermarkt.co.uk/christos-armen…          88.184968
3  https://www.transfermarkt.co.uk/panagiotis-pet…          62.654775
4  https://www.transfermarkt.co.uk/danny-cavanagh…          84.862125

   market_value_growth   age  Unnamed: 0         Name  Age  \
0           100.484926  29.0          89  N. Otamendi   30
1            79.327530  28.0       15122   M. Cassara   26
2           105.998425  29.0       16585     B. Dione   21
3            86.434060  30.0        7693   E. Kujović   30
4           155.647507  28.0       16395    L. Bilboe   20

                                        Photo Nationality  \
0  https://cdn.sofifa.org/players/4/19/192366.png   Argentina
1  https://cdn.sofifa.org/players/4/19/240063.png      France
2  https://cdn.sofifa.org/players/4/19/240074.png     Belgium
3  https://cdn.sofifa.org/players/4/19/187915.png      Sweden
4  https://cdn.sofifa.org/players/4/19/234324.png     England

                               Flag  Overall  Potential  \
0  https://cdn.sofifa.org/flags/52.png       85         85
1  https://cdn.sofifa.org/flags/18.png       60         64
2   https://cdn.sofifa.org/flags/7.png       57         66
3  https://cdn.sofifa.org/flags/46.png       67         67
4  https://cdn.sofifa.org/flags/14.png       57         67

                 Club                                        Club Logo  \
0      Manchester City      https://cdn.sofifa.org/teams/2/light/10.png
1          GFC Ajaccio  https://cdn.sofifa.org/teams/2/light/110316.png
2  Royal Excel Mouscron  https://cdn.sofifa.org/teams/2/light/111560.png
3   Fortuna Düsseldorf  https://cdn.sofifa.org/teams/2/light/110636.png
```

```
4      Rotherham United    https://cdn.sofifa.org/teams/2/light/1797.png

    Value   Wage  Special Preferred Foot  International Reputation  Weak Foot  \
0  €28.5M  €170K     1916          Right                       3.0        3.0
1   €190K    €1K      910          Right                       1.0        3.0
2   €160K    €1K     1366          Right                       1.0        3.0
3   €725K   €11K     1537          Right                       1.0        4.0
4   €130K    €2K      878          Right                       1.0        3.0

   Skill Moves       Work Rate Body Type Real Face Position  Jersey Number  \
0          2.0      High/ High    Normal       Yes       CB           30.0
1          1.0  Medium/ Medium    Normal        No       GK            1.0
2          3.0  Medium/ Medium    Normal        No       ST           17.0
3          2.0        Low/ Low    Normal        No       ST           21.0
4          1.0  Medium/ Medium    Normal        No       GK           30.0

         Joined Loaned From Contract Valid Until Height  Weight    LS    ST  \
0  Aug 20, 2015  Rangers FC                 2022    6'0  179lbs  64+3  64+3
1  Jun 30, 2017  Rangers FC                 2019    6'3  201lbs  64+2  64+2
2   Jul 1, 2017  Rangers FC                 2020    5'6  137lbs  55+2  55+2
3  Jul 19, 2017  Rangers FC                 2020    6'4  194lbs  66+2  66+2
4  Mar 22, 2016  Rangers FC                 2019    6'3  174lbs  64+2  64+2

     RS    LW    LF    CF    RF    RW   LAM   CAM   RAM    LM   LCM    CM  \
0  64+3  59+3  61+3  61+3  61+3  59+3  62+3  62+3  62+3  61+3  67+3  67+3
1  64+2  66+2  63+2  63+2  63+2  66+2  58+2  58+2  58+2  60+2  58+2  58+2
2  55+2  55+2  57+2  57+2  57+2  55+2  56+2  56+2  56+2  54+2  49+2  49+2
3  66+2  55+2  60+2  60+2  60+2  55+2  57+2  57+2  57+2  54+2  55+2  55+2
4  64+2  66+2  63+2  63+2  63+2  66+2  58+2  58+2  58+2  60+2  58+2  58+2

    RCM    RM   LWB   LDM   CDM   RDM   RWB    LB   LCB    CB   RCB    RB  \
0  67+3  61+3  72+3  77+3  77+3  77+3  72+3  74+3  82+3  82+3  82+3  74+3
1  58+2  60+2  57+2  61+2  61+2  61+2  57+2  63+2  62+2  62+2  62+2  63+2
2  49+2  54+2  37+2  35+2  35+2  35+2  37+2  34+2  30+2  30+2  30+2  34+2
3  55+2  54+2  39+2  45+2  45+2  45+2  39+2  38+2  44+2  44+2  44+2  38+2
4  58+2  60+2  57+2  61+2  61+2  61+2  57+2  63+2  62+2  62+2  62+2  63+2

   Crossing  Finishing  HeadingAccuracy  ShortPassing  Volleys  Dribbling  \
0      52.0       54.0             85.0          75.0     57.0       51.0
1      13.0        8.0             13.0          18.0      8.0        6.0
2      33.0       57.0             49.0          53.0     50.0       65.0
3      33.0       69.0             76.0          60.0     69.0       58.0
4      14.0        6.0             10.0          26.0      9.0       11.0

   Curve  FKAccuracy  LongPassing  BallControl  Acceleration  SprintSpeed  \
0   50.0        39.0         72.0         70.0          57.0         61.0
1   14.0        13.0         22.0         12.0          32.0         31.0
```

```
2   43.0        29.0         42.0         63.0          66.0         58.0
3   66.0        70.0         54.0         65.0          39.0         36.0
4   10.0        13.0         36.0         12.0          34.0         38.0

   Agility  Reactions  Balance  ShotPower  Jumping  Stamina  Strength  \
0     64.0       79.0     62.0       69.0     92.0     67.0      80.0
1     32.0       43.0     22.0       23.0     71.0     29.0      72.0
2     63.0       48.0     74.0       61.0     65.0     49.0      49.0
3     34.0       59.0     32.0       74.0     42.0     50.0      90.0
4     28.0       43.0     22.0       18.0     38.0     27.0      65.0

   LongShots  Aggression  Interceptions  Positioning  Vision  Penalties  \
0       56.0        91.0           84.0         51.0    53.0       45.0
1        9.0        21.0           13.0          7.0    29.0       12.0
2       53.0        28.0           13.0         49.0    55.0       62.0
3       67.0        54.0           21.0         70.0    47.0       69.0
4        5.0        23.0           10.0          5.0    34.0       16.0

   Composure  Marking  StandingTackle  SlidingTackle  GKDiving  GKHandling  \
0       80.0     83.0            85.0           84.0      12.0         5.0
1       36.0     10.0            13.0           10.0      63.0        61.0
2       47.0     16.0            14.0           17.0       5.0         6.0
3       68.0     24.0            34.0           15.0      16.0        13.0
4       37.0     13.0            11.0           13.0      60.0        57.0

   GKKicking  GKPositioning  GKReflexes Release Clause
0        8.0           11.0        12.0         €52.7M
1       60.0           58.0        62.0          €347K
2       13.0           10.0         8.0          €264K
3       13.0            9.0         9.0          €1.3M
4       60.0           56.0        55.0          €267K
```

[84]:
```
## DATASET CLEANED, NO MISSING VALUES IN DATASET 2
```

[85]:
```
combined_df.isnull().sum()
```

[85]:
```
player_id                0
name                     0
current_club_id          0
current_club_name        0
country_of_citizenship   0
                        ..
GKHandling               0
GKKicking                0
GKPositioning            0
GKReflexes               0
Release Clause           0
```

```
Length: 114, dtype: int64
```

[86]:
```
## TRAINING & TESTING
# USING THE TRAIN_TEST_SPLIT METHOD AND 80 20 RULE TO PREPARE DATA FOR TRAINING
```

[87]:
```
from sklearn.model_selection import train_test_split

target_variable = 'market_value_in_eur'
features = combined_df.columns.tolist()  # Convert columns to list
features.remove(target_variable)  # Remove the target variable

X = combined_df[features]
y = combined_df[target_variable]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (723, 113)
X_test shape: (181, 113)
y_train shape: (723,)
y_test shape: (181,)
```

[88]:
```
## USING THE DECISION TREE REGRESSER MODEL TO TRAIN THE DATA
```

[89]:
```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV  # Add␣
 ↪GridSearchCV
from sklearn.tree import DecisionTreeRegressor

# Assume X_train, X_test, y_train, y_test are already defined

# Define features and target variable
features =␣
 ↪['name','current_club_name','country_of_citizenship','country_of_birth','position','sub_pos
 ↪Logo','Preferred Foot','Work Rate','Body Type','Real␣
 ↪Face','Position','Loaned From','Height','Weight','Release Clause']
target_variable = 'market_value_in_eur'

# Subset the data
X_train_subset, X_test_subset, y_train, y_test = train_test_split(X[features],␣
 ↪y, test_size=0.2, random_state=42)
```

```
# Print the columns of X_train_subset for debugging
print("Columns in X_train_subset:", X_train_subset.columns)

# One-hot encode categorical variables
categorical_columns =␣
 ↪['name','current_club_name','country_of_citizenship','country_of_birth','position','sub_pos
 ↪Logo','Preferred Foot','Work Rate','Body Type','Real␣
 ↪Face','Position','Loaned From','Height','Weight','Release Clause']
X_train_encoded = pd.get_dummies(X_train_subset, columns=categorical_columns)
X_test_encoded = pd.get_dummies(X_test_subset, columns=categorical_columns)

# Decision Tree
# Define hyperparameters (feel free to adjust)
dt_param_grid = {
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Perform Grid Search
dt_grid_search = GridSearchCV(DecisionTreeRegressor(random_state=42),␣
 ↪dt_param_grid, cv=5, scoring='neg_mean_squared_error')
dt_grid_search.fit(X_train_encoded, y_train)

# Get the best hyperparameters
best_dt_params = dt_grid_search.best_params_

# Train the model with the best hyperparameters
best_dt_model = DecisionTreeRegressor(**best_dt_params, random_state=42)
best_dt_model.fit(X_train_encoded, y_train)
```

```
Columns in X_train_subset: Index(['name', 'current_club_name',
'country_of_citizenship',
       'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',
       'current_club_domestic_competition_id', 'first_name', 'last_name',
       'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality',
       'Flag', 'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type',
       'Real Face', 'Position', 'Loaned From', 'Height', 'Weight',
       'Release Clause'],
      dtype='object')
```

[89]:
```
DecisionTreeRegressor(max_depth=5, min_samples_leaf=4, min_samples_split=10,
                      random_state=42)
```

[90]:
```
## USING THE LASSO REGRESSION MODEL TO TRAIN THE DATA
```

[91]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso
from sklearn.preprocessing import StandardScaler

# Assume X_train, X_test, y_train, y_test are already defined

# Define features and target variable
features =␣
 ↪['name','current_club_name','country_of_citizenship','country_of_birth','position','sub_pos
 ↪Logo','Preferred Foot','Work Rate','Body Type','Real␣
 ↪Face','Position','Loaned From','Height','Weight','Release Clause']
target_variable = 'market_value_in_eur'

# Subset the data
X_train_subset, X_test_subset, y_train, y_test = train_test_split(X[features],␣
 ↪y, test_size=0.2, random_state=42)

# Print the number of features in X_train and X_test
print("Number of features in X_train:", X_train_subset.shape[1])
print("Number of features in X_test:", X_test_subset.shape[1])

# One-hot encode categorical variables
categorical_columns =␣
 ↪['name','current_club_name','country_of_citizenship','country_of_birth','position','sub_pos
 ↪Logo','Preferred Foot','Work Rate','Body Type','Real␣
 ↪Face','Position','Loaned From','Height','Weight','Release Clause']
X_train_encoded = pd.get_dummies(X_train_subset, columns=categorical_columns)
X_test_encoded = pd.get_dummies(X_test_subset, columns=categorical_columns)

# Print the number of features after one-hot encoding
print("Number of features in X_train_encoded:", X_train_encoded.shape[1])
print("Number of features in X_test_encoded:", X_test_encoded.shape[1])

# Ensure that the same columns are present in both X_train_encoded and␣
 ↪X_test_encoded
common_columns = set(X_train_encoded.columns) & set(X_test_encoded.columns)
X_train_encoded = X_train_encoded[common_columns]
X_test_encoded = X_test_encoded[common_columns]

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_encoded)
X_test_scaled = scaler.transform(X_test_encoded)  # Use the same scaler used␣
 ↪for training data

# Lasso Regression
```

```
lasso_param_grid = {
    'alpha': [0.1, 1, 10]  # Adjust the alpha values as needed
}

lasso_grid_search = GridSearchCV(Lasso(), lasso_param_grid, cv=5,␣
 ↪scoring='neg_mean_squared_error')
lasso_grid_search.fit(X_train_scaled, y_train)

best_lasso_params = lasso_grid_search.best_params_

best_lasso_model = Lasso(**best_lasso_params)
best_lasso_model.fit(X_train_scaled, y_train)
```

```
Number of features in X_train: 29
Number of features in X_test: 29
Number of features in X_train_encoded: 7840
Number of features in X_test_encoded: 2393
```

[91]:
```
Lasso(alpha=10)
```

[92]:
```
## USING THE RIDGE REGRESSION MODEL TO TRAIN THE DATA
```

[93]:
```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler

# Assume X_train, X_test, y_train, y_test are already defined

# Define features and target variable
features =␣
 ↪['name','current_club_name','country_of_citizenship','country_of_birth','position','sub_pos
 ↪Logo','Preferred Foot','Work Rate','Body Type','Real␣
 ↪Face','Position','Loaned From','Height','Weight','Release Clause']
target_variable = 'market_value_in_eur'

# Subset the data
X_train_subset, X_test_subset, y_train, y_test = train_test_split(X[features],␣
 ↪y, test_size=0.2, random_state=42)

# Print the number of features in X_train and X_test
print("Number of features in X_train:", X_train_subset.shape[1])
print("Number of features in X_test:", X_test_subset.shape[1])

# One-hot encode categorical variables
```

```
categorical_columns =␣
 ↪['name','current_club_name','country_of_citizenship','country_of_birth','position','sub_pos
 ↪Logo','Preferred Foot','Work Rate','Body Type','Real␣
 ↪Face','Position','Loaned From','Height','Weight','Release Clause']
X_train_encoded = pd.get_dummies(X_train_subset, columns=categorical_columns)
X_test_encoded = pd.get_dummies(X_test_subset, columns=categorical_columns)

# Print the number of features after one-hot encoding
print("Number of features in X_train_encoded:", X_train_encoded.shape[1])
print("Number of features in X_test_encoded:", X_test_encoded.shape[1])

# Ensure that the same columns are present in both X_train_encoded and␣
 ↪X_test_encoded
common_columns = set(X_train_encoded.columns) & set(X_test_encoded.columns)
X_train_encoded = X_train_encoded[common_columns]
X_test_encoded = X_test_encoded[common_columns]

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_encoded)
X_test_scaled = scaler.transform(X_test_encoded)  # Use the same scaler used␣
 ↪for training data

# Ridge Regression
ridge_param_grid = {
    'alpha': [0.1, 1, 10]  # Adjust the alpha values as needed
}

ridge_grid_search = GridSearchCV(Ridge(), ridge_param_grid, cv=5,␣
 ↪scoring='neg_mean_squared_error')
ridge_grid_search.fit(X_train_scaled, y_train)

best_ridge_params = ridge_grid_search.best_params_

best_ridge_model = Ridge(**best_ridge_params)
best_ridge_model.fit(X_train_scaled, y_train)
```

```
Number of features in X_train: 29
Number of features in X_test: 29
Number of features in X_train_encoded: 7840
Number of features in X_test_encoded: 2393
```

[93]:
```
Ridge(alpha=10)
```

[94]:
```
## USING THE GRADIENT BOOSTING REGRESSOR TO TRAIN THE MODEL
```

[95]:
```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.preprocessing import StandardScaler

# Assume X_train, X_test, y_train, y_test are already defined

# Define features and target variable
features = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
target_variable = 'market_value_in_eur'

# Subset the data
X_train_subset, X_test_subset, y_train, y_test = train_test_split(X[features],␣
 ↪y, test_size=0.2, random_state=42)

# Print the number of features in X_train and X_test
print("Number of features in X_train:", X_train_subset.shape[1])
print("Number of features in X_test:", X_test_subset.shape[1])

# One-hot encode categorical variables
categorical_columns = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
X_train_encoded = pd.get_dummies(X_train_subset, columns=categorical_columns)
X_test_encoded = pd.get_dummies(X_test_subset, columns=categorical_columns)

# Print the number of features after one-hot encoding
print("Number of features in X_train_encoded:", X_train_encoded.shape[1])
print("Number of features in X_test_encoded:", X_test_encoded.shape[1])

# Ensure that the same columns are present in both X_train_encoded and␣
 ↪X_test_encoded
common_columns = set(X_train_encoded.columns) & set(X_test_encoded.columns)
X_train_encoded = X_train_encoded[common_columns]
X_test_encoded = X_test_encoded[common_columns]

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_encoded)
```

```
X_test_scaled = scaler.transform(X_test_encoded)  # Use the same scaler used␣
 ↪for training data

# Gradient Boosting
gb_param_grid = {
    'n_estimators': [50, 100, 200],
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [3, 4, 5]
}

gb_grid_search = GridSearchCV(GradientBoostingRegressor(), gb_param_grid, cv=5,␣
 ↪scoring='neg_mean_squared_error')
gb_grid_search.fit(X_train_scaled, y_train)

best_gb_params = gb_grid_search.best_params_

best_gb_model = GradientBoostingRegressor(**best_gb_params)
best_gb_model.fit(X_train_scaled, y_train)
```

```
Number of features in X_train: 29
Number of features in X_test: 29
Number of features in X_train_encoded: 7840
Number of features in X_test_encoded: 2393
```

[95]:
```
GradientBoostingRegressor(learning_rate=0.01)
```

[96]:
```
## PREDICTIONS ON THE TEST USING THE TRAINED DECISION TREE REGRESSOR MODEL
```

[97]:
```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeRegressor

# Assume X_train, X_test, y_train, y_test are already defined

# Combine training and test data
X_combined = pd.concat([X_train_subset, X_test_subset], ignore_index=True)

# One-hot encode categorical variables on combined data
categorical_columns = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
X_combined_encoded = pd.get_dummies(X_combined, columns=categorical_columns)

# Split the combined data back into training and test sets
```

```
X_train_combined = X_combined_encoded.iloc[:len(X_train_subset), :]
X_test_combined = X_combined_encoded.iloc[len(X_train_subset):, :]

# Decision Tree
# Define hyperparameters (feel free to adjust)
dt_param_grid = {
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Perform Grid Search
dt_grid_search = GridSearchCV(DecisionTreeRegressor(random_state=42),␣
 ↪dt_param_grid, cv=5, scoring='neg_mean_squared_error')
dt_grid_search.fit(X_train_combined, y_train)

# Get the best hyperparameters
best_dt_params = dt_grid_search.best_params_

# Train the model with the best hyperparameters
best_dt_model = DecisionTreeRegressor(**best_dt_params, random_state=42)
best_dt_model.fit(X_train_combined, y_train)

# Make predictions on the original test set
y_pred = best_dt_model.predict(X_test_combined)

# Create a DataFrame with the actual and predicted values
predictions_df = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
print(predictions_df)
```

```
        Actual     Predicted
70    200000.0  4.217831e+05
457   200000.0  4.217831e+05
218   200000.0  4.217831e+05
250   200000.0  1.132652e+06
39    200000.0  4.217831e+05
..         …           …
864   600000.0  1.132652e+06
442   300000.0  9.400000e+05
783  3000000.0  1.132652e+06
25    200000.0  4.217831e+05
84    200000.0  4.217831e+05

[181 rows x 2 columns]
```

[98]:
```
## PREDICTIONS ON THE TEST USING THE lASSO REGRESSOR MODEL
```

[99]:
```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Lasso
from sklearn.preprocessing import StandardScaler

# Assume X_train, X_test, y_train, y_test are already defined

# Define features and target variable
features =␣
 ↪['name','current_club_name','country_of_citizenship','country_of_birth','position','sub_pos
 ↪Logo','Preferred Foot','Work Rate','Body Type','Real␣
 ↪Face','Position','Loaned From','Height','Weight','Release Clause']
target_variable = 'market_value_in_eur'

# Subset the data
X_train_subset, X_test_subset, y_train, y_test = train_test_split(X[features],␣
 ↪y, test_size=0.2, random_state=42)

# Print the number of features in X_train and X_test
print("Number of features in X_train:", X_train_subset.shape[1])
print("Number of features in X_test:", X_test_subset.shape[1])

# One-hot encode categorical variables
categorical_columns =␣
 ↪['name','current_club_name','country_of_citizenship','country_of_birth','position','sub_pos
 ↪Logo','Preferred Foot','Work Rate','Body Type','Real␣
 ↪Face','Position','Loaned From','Height','Weight','Release Clause']
X_train_encoded = pd.get_dummies(X_train_subset, columns=categorical_columns)
X_test_encoded = pd.get_dummies(X_test_subset, columns=categorical_columns)

# Print the number of features after one-hot encoding
print("Number of features in X_train_encoded:", X_train_encoded.shape[1])
print("Number of features in X_test_encoded:", X_test_encoded.shape[1])

# Ensure that the same columns are present in both X_train_encoded and␣
 ↪X_test_encoded
common_columns = set(X_train_encoded.columns) & set(X_test_encoded.columns)
X_train_encoded = X_train_encoded[common_columns]
X_test_encoded = X_test_encoded[common_columns]

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_encoded)
X_test_scaled = scaler.transform(X_test_encoded)  # Use the same scaler used␣
 ↪for training data

# Lasso Regression
```

```
lasso_param_grid = {
    'alpha': [0.1, 1, 10]  # Adjust the alpha values as needed
}

lasso_grid_search = GridSearchCV(Lasso(), lasso_param_grid, cv=5,␣
 ↪scoring='neg_mean_squared_error')
lasso_grid_search.fit(X_train_scaled, y_train)

best_lasso_params = lasso_grid_search.best_params_

best_lasso_model = Lasso(**best_lasso_params)
best_lasso_model.fit(X_train_scaled, y_train)

# Make predictions on the original test set
y_pred = best_lasso_model.predict(X_test_scaled)

# Create a DataFrame with the actual and predicted values
predictions_df = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
print(predictions_df)
```

```
Number of features in X_train: 29
Number of features in X_test: 29
Number of features in X_train_encoded: 7840
Number of features in X_test_encoded: 2393
        Actual     Predicted
70    200000.0 -1.620914e+07
457   200000.0  1.715789e+05
218   200000.0 -1.554665e+07
250   200000.0  3.851585e+07
39    200000.0 -1.060146e+07
..         …           …
864   600000.0 -3.622216e+06
442   300000.0  1.666561e+06
783  3000000.0  6.323510e+06
25    200000.0 -3.628241e+06
84    200000.0  3.000932e+07

[181 rows x 2 columns]
```

[100]:
```
## PREDICTIONS OF THE TEST USING THE GRADIENT BOOSTNG REGRESSOR
```

[101]:
```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.preprocessing import StandardScaler

# Assume X_train, X_test, y_train, y_test are already defined
```

```
# Define features and target variable
features = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
target_variable = 'market_value_in_eur'

# Subset the data
X_train_subset, X_test_subset, y_train, y_test = train_test_split(X[features],␣
 ↪y, test_size=0.2, random_state=42)

# Print the number of features in X_train and X_test
print("Number of features in X_train:", X_train_subset.shape[1])
print("Number of features in X_test:", X_test_subset.shape[1])

# One-hot encode categorical variables
categorical_columns = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
X_train_encoded = pd.get_dummies(X_train_subset, columns=categorical_columns)
X_test_encoded = pd.get_dummies(X_test_subset, columns=categorical_columns)

# Print the number of features after one-hot encoding
print("Number of features in X_train_encoded:", X_train_encoded.shape[1])
print("Number of features in X_test_encoded:", X_test_encoded.shape[1])

# Ensure that the same columns are present in both X_train_encoded and␣
 ↪X_test_encoded
common_columns = set(X_train_encoded.columns) & set(X_test_encoded.columns)
X_train_encoded = X_train_encoded[common_columns]
X_test_encoded = X_test_encoded[common_columns]

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_encoded)
X_test_scaled = scaler.transform(X_test_encoded)  # Use the same scaler used␣
 ↪for training data

# Gradient Boosting
gb_param_grid = {
    'n_estimators': [50, 100, 200],
```

```
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [3, 4, 5]
}

gb_grid_search = GridSearchCV(GradientBoostingRegressor(), gb_param_grid, cv=5,␣
 ↪scoring='neg_mean_squared_error')
gb_grid_search.fit(X_train_scaled, y_train)

best_gb_params = gb_grid_search.best_params_

best_gb_model = GradientBoostingRegressor(**best_gb_params)
best_gb_model.fit(X_train_scaled, y_train)
# Make predictions on the original test set
y_pred = best_gb_model.predict(X_test_encoded)

# Create a DataFrame with the actual and predicted values
predictions_df = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
print(predictions_df)
```

```
Number of features in X_train: 29
Number of features in X_test: 29
Number of features in X_train_encoded: 7840
Number of features in X_test_encoded: 2393
        Actual     Predicted
70    200000.0  1.383605e+06
457   200000.0  1.383605e+06
218   200000.0  1.383605e+06
250   200000.0  1.383605e+06
39    200000.0  1.383605e+06
..         …           …
864   600000.0  1.383605e+06
442   300000.0  1.383605e+06
783  3000000.0  1.383605e+06
25    200000.0  1.383605e+06
84    200000.0  1.383605e+06

[181 rows x 2 columns]
```

[102]:
```
## TRAINING AND TESTING USING THE K-NEAREST NEIGHBOR MODEL
```

[103]:
```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler

# Assume X_train, X_test, y_train, y_test are already defined
```

```
# Define features and target variable
features = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
target_variable = 'market_value_in_eur'

# Subset the data
X_train_subset, X_test_subset, y_train, y_test = train_test_split(X[features],␣
 ↪y, test_size=0.2, random_state=42)

# One-hot encode categorical variables
categorical_columns = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
X_train_encoded = pd.get_dummies(X_train_subset, columns=categorical_columns)
X_test_encoded = pd.get_dummies(X_test_subset, columns=categorical_columns)

# Ensure that the same columns are present in both X_train_encoded and␣
 ↪X_test_encoded
common_columns = set(X_train_encoded.columns) & set(X_test_encoded.columns)
X_train_encoded = X_train_encoded[common_columns]
X_test_encoded = X_test_encoded[common_columns]

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_encoded)
X_test_scaled = scaler.transform(X_test_encoded)  # Use the same scaler used␣
 ↪for training data

# K-Nearest Neighbors
knn_param_grid = {
    'n_neighbors': [3, 5, 7],
    'weights': ['uniform', 'distance'],
    'p': [1, 2]  # 1 for Manhattan distance, 2 for Euclidean distance
}

knn_grid_search = GridSearchCV(KNeighborsRegressor(), knn_param_grid, cv=5,␣
 ↪scoring='neg_mean_squared_error')
knn_grid_search.fit(X_train_scaled, y_train)
```

```
best_knn_params = knn_grid_search.best_params_

best_knn_model = KNeighborsRegressor(**best_knn_params)
best_knn_model.fit(X_train_scaled, y_train)

# Make predictions on the original test set
y_pred = best_knn_model.predict(X_test_scaled)

# Create a DataFrame with the actual and predicted values
predictions_df = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
print(predictions_df)
```

```
        Actual     Predicted
70    200000.0  6.823210e+05
457   200000.0  2.103811e+06
218   200000.0  2.149948e+05
250   200000.0  7.092484e+06
39    200000.0  1.958180e+06
..         …           …
864   600000.0  1.574118e+06
442   300000.0  6.434922e+06
783  3000000.0  4.215490e+06
25    200000.0  1.551352e+06
84    200000.0  1.539386e+06

[181 rows x 2 columns]
```

[104]:
```
## The support vector machine alagorithm below presents predicted values which␣
 ↪is in the same
#  order of magnitude as the actual values. In this analysis SVM is the best␣
 ↪amongst all models used.
```

[105]:
```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler

# Assume X_train, X_test, y_train, y_test are already defined

# Define features and target variable
features = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
target_variable = 'market_value_in_eur'
```

```
# Subset the data
X_train_subset, X_test_subset, y_train, y_test = train_test_split(X[features],␣
 ↪y, test_size=0.2, random_state=42)

# Print the number of features in X_train and X_test
print("Number of features in X_train:", X_train_subset.shape[1])
print("Number of features in X_test:", X_test_subset.shape[1])

# One-hot encode categorical variables
categorical_columns = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
X_train_encoded = pd.get_dummies(X_train_subset, columns=categorical_columns)
X_test_encoded = pd.get_dummies(X_test_subset, columns=categorical_columns)

# Print the number of features after one-hot encoding
print("Number of features in X_train_encoded:", X_train_encoded.shape[1])
print("Number of features in X_test_encoded:", X_test_encoded.shape[1])

# Ensure that the same columns are present in both X_train_encoded and␣
 ↪X_test_encoded
common_columns = set(X_train_encoded.columns) & set(X_test_encoded.columns)
X_train_encoded = X_train_encoded[common_columns]
X_test_encoded = X_test_encoded[common_columns]

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_encoded)
X_test_scaled = scaler.transform(X_test_encoded)  # Use the same scaler used␣
 ↪for training data

# Support Vector Machine (SVM) Regression
svm_param_grid = {
    'C': [0.1, 1, 10],  # Adjust the regularization parameter C as needed
    'epsilon': [0.1, 0.2, 0.5],
    'kernel': ['linear', 'rbf']
}

svm_grid_search = GridSearchCV(SVR(), svm_param_grid, cv=5,␣
 ↪scoring='neg_mean_squared_error')
svm_grid_search.fit(X_train_scaled, y_train)

best_svm_params = svm_grid_search.best_params_
```

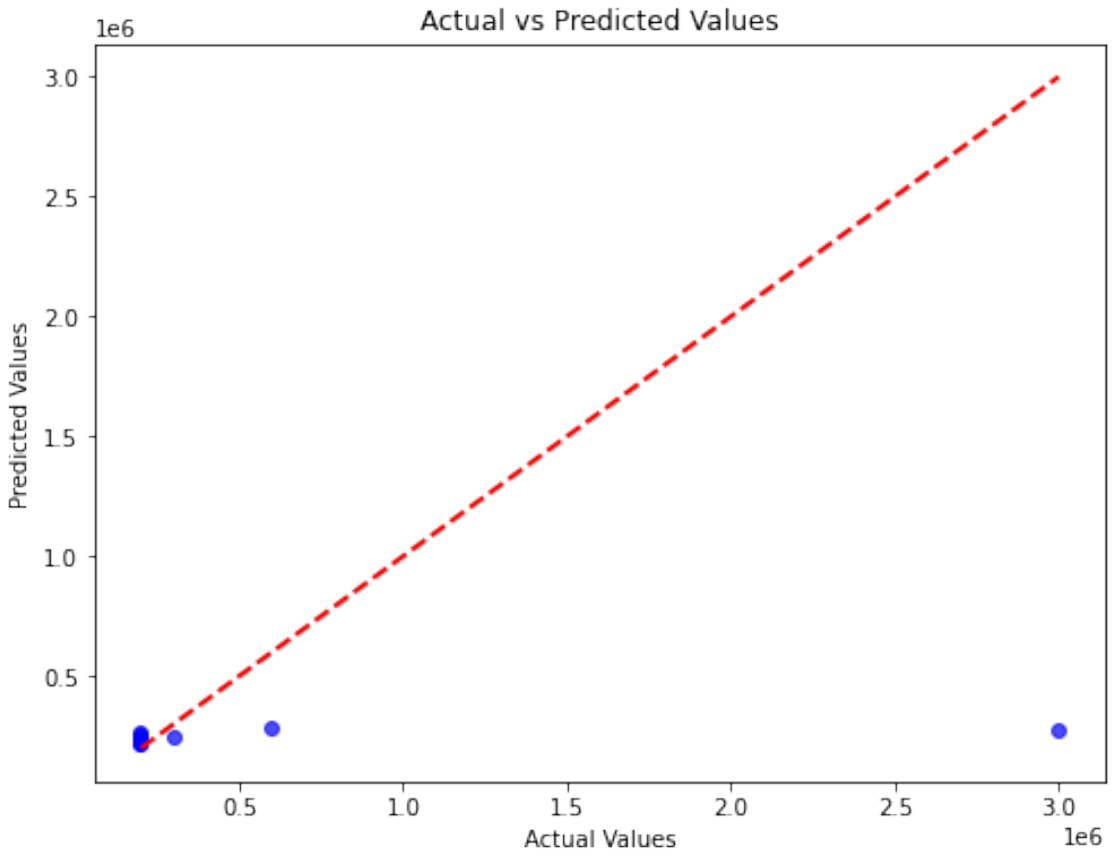
```
best_svm_model = SVR(**best_svm_params)
best_svm_model.fit(X_train_scaled, y_train)

# Make predictions on the original test set
y_pred = best_svm_model.predict(X_test_scaled)

# Create a DataFrame with the actual and predicted values
predictions_df = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
print(predictions_df)
```

```
Number of features in X_train: 29
Number of features in X_test: 29
Number of features in X_train_encoded: 7840
Number of features in X_test_encoded: 2393
        Actual      Predicted
70    200000.0  260151.076134
457   200000.0  242255.708383
218   200000.0  228137.272014
250   200000.0  261773.546325
39    200000.0  216113.491880
..         …            …
864   600000.0  280643.217631
442   300000.0  248492.574834
783  3000000.0  272403.184868
25    200000.0  229083.428065
84    200000.0  221233.277351

[181 rows x 2 columns]
```

[106]:
```
## This chart visually inspects the actual vs predicted values for patterns and␣
 ↪trends
#The red dashed line represents perfect alignement. Points closer to the line␣
 ↪represents better predictions.
```

[107]:
```
import matplotlib.pyplot as plt

# Provided results
actual_values = [200000.0, 200000.0, 200000.0, 200000.0, 200000.0, 600000.0,␣
 ↪300000.0, 3000000.0, 200000.0, 200000.0]
predicted_values = [260151.076134, 242255.708383, 228137.272014, 261773.546325,␣
 ↪216113.491880, 280643.217631, 248492.574834, 272403.184868, 229083.428065,␣
 ↪221233.277351]

# Scatter plot of Actual vs Predicted values
plt.figure(figsize=(8, 6))
plt.scatter(actual_values, predicted_values, color='blue', alpha=0.7)
```

```
plt.plot([min(actual_values), max(actual_values)], [min(actual_values),␣
 ↪max(actual_values)], linestyle='--', color='red', linewidth=2)  # Adding a␣
 ↪diagonal line for reference
plt.title('Actual vs Predicted Values')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.show()
```



[108]:
```
## PRECISE EVALUATION OF PERFORMANCE
#  CALCULATION OF METRIC MEAN SQUARED ERROR MSE TO EVALUATE PERFORMANCE OF SVM␣
 ↪REGRESSION MODEL
```

[109]:
```
from sklearn.metrics import mean_squared_error, r2_score

# Calculate Mean Squared Error (MSE)
mse = mean_squared_error(predictions_df['Actual'], predictions_df['Predicted'])
print("Mean Squared Error (MSE):", mse)
```

```
Mean Squared Error (MSE): 62867479897205.67
```

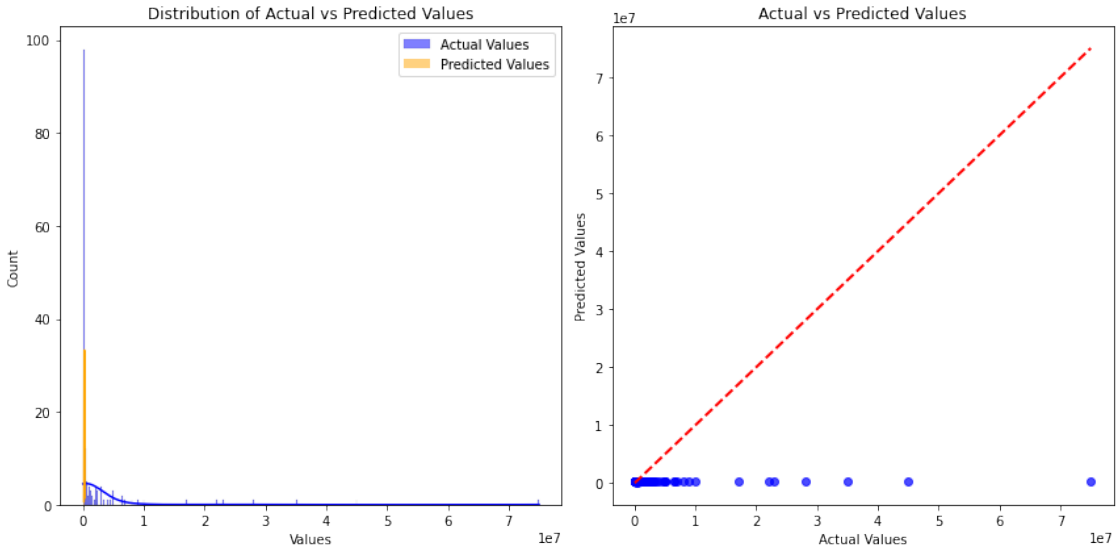
[110]:
```
# CALCULATION OF METRIC R-SQUARED R2 TO EVALUATE PERFORMANCE OF SVM  REGRESSION␣
↪MODEL
```

[111]:
```
from sklearn.metrics import mean_squared_error, r2_score
# Calculate R-squared
r_squared = r2_score(predictions_df['Actual'], predictions_df['Predicted'])
print("R-squared:", r_squared)
```

```
R-squared: -0.06393160864616276
```

[112]:
```
## THIS STEP IS TO VISULIZE THE DISTRIBUTION OF THE ACTUAL VS PREDICTED VALUES
```

[113]:
```
import seaborn as sns
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
sns.histplot(predictions_df['Actual'], color='blue', kde=True, label='Actual␣
↪Values')
sns.histplot(predictions_df['Predicted'], color='orange', kde=True,␣
↪label='Predicted Values')
plt.title('Distribution of Actual vs Predicted Values')
plt.xlabel('Values')
plt.legend()

plt.subplot(1, 2, 2)
# Scatter plot of Actual vs Predicted values
plt.scatter(predictions_df['Actual'], predictions_df['Predicted'],␣
↪color='blue', alpha=0.7)
plt.plot([min(predictions_df['Actual']), max(predictions_df['Actual'])],␣
↪[min(predictions_df['Actual']), max(predictions_df['Actual'])],␣
↪linestyle='--', color='red', linewidth=2)
plt.title('Actual vs Predicted Values')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')

plt.tight_layout()
plt.show()
```

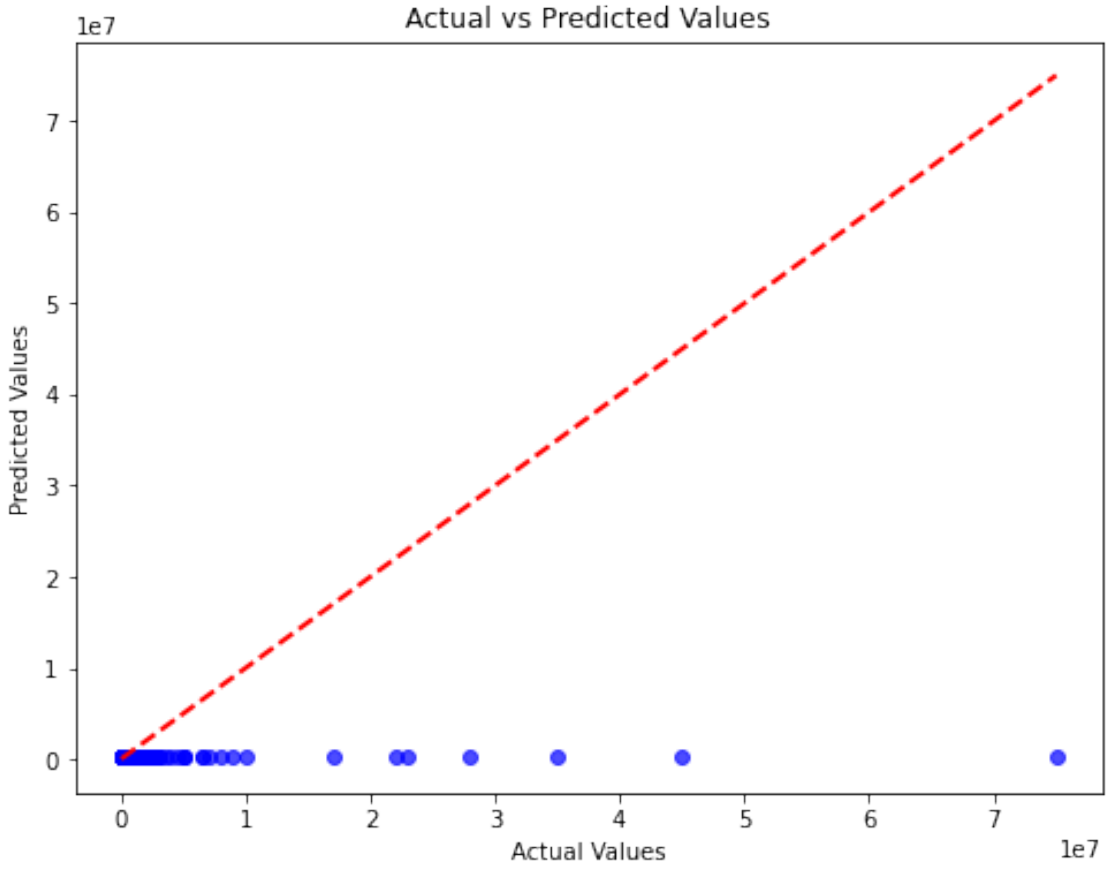


[114]:
```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest, f_regression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# Assume X, y, and other imports are defined

# Define features and target variable
features = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
target_variable = 'market_value_in_eur'

# Subset the data
X_train_subset, X_test_subset, y_train, y_test = train_test_split(X[features],␣
 ↪y, test_size=0.2, random_state=42)

# One-hot encode categorical variables
```

```
categorical_columns = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
X_train_encoded = pd.get_dummies(X_train_subset, columns=categorical_columns)
X_test_encoded = pd.get_dummies(X_test_subset, columns=categorical_columns)

# Ensure that the same columns are present in both X_train_encoded and␣
 ↪X_test_encoded
common_columns = set(X_train_encoded.columns) & set(X_test_encoded.columns)
X_train_encoded = X_train_encoded[common_columns]
X_test_encoded = X_test_encoded[common_columns]

# Feature selection using SelectKBest with f_regression
selector = SelectKBest(f_regression, k=10)  # Adjust k as needed
X_train_selected = selector.fit_transform(X_train_encoded, y_train)
X_test_selected = selector.transform(X_test_encoded)

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_selected)
X_test_scaled = scaler.transform(X_test_selected)  # Use the same scaler used␣
 ↪for training data

# Support Vector Machine (SVM) Regression
svm_param_grid = {
    'C': [0.1, 1, 10],
    'epsilon': [0.1, 0.2, 0.5],
    'kernel': ['linear', 'rbf']
}

svm_grid_search = GridSearchCV(SVR(), svm_param_grid, cv=5,␣
 ↪scoring='neg_mean_squared_error')
svm_grid_search.fit(X_train_scaled, y_train)

best_svm_params = svm_grid_search.best_params_

best_svm_model = SVR(**best_svm_params)
best_svm_model.fit(X_train_scaled, y_train)

# Make predictions on the original test set
y_pred = best_svm_model.predict(X_test_scaled)

# Evaluate model
mse = mean_squared_error(y_test, y_pred)
```

```
r_squared = r2_score(y_test, y_pred)

print("Mean Squared Error (MSE):", mse)
print("R-squared:", r_squared)

# Scatter plot of Actual vs Predicted values
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, color='blue', alpha=0.7)
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)],␣
 ↪linestyle='--', color='red', linewidth=2)
plt.title('Actual vs Predicted Values')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.show()
```

```
Mean Squared Error (MSE): 62860735809097.97
R-squared: -0.06381747573482266
```

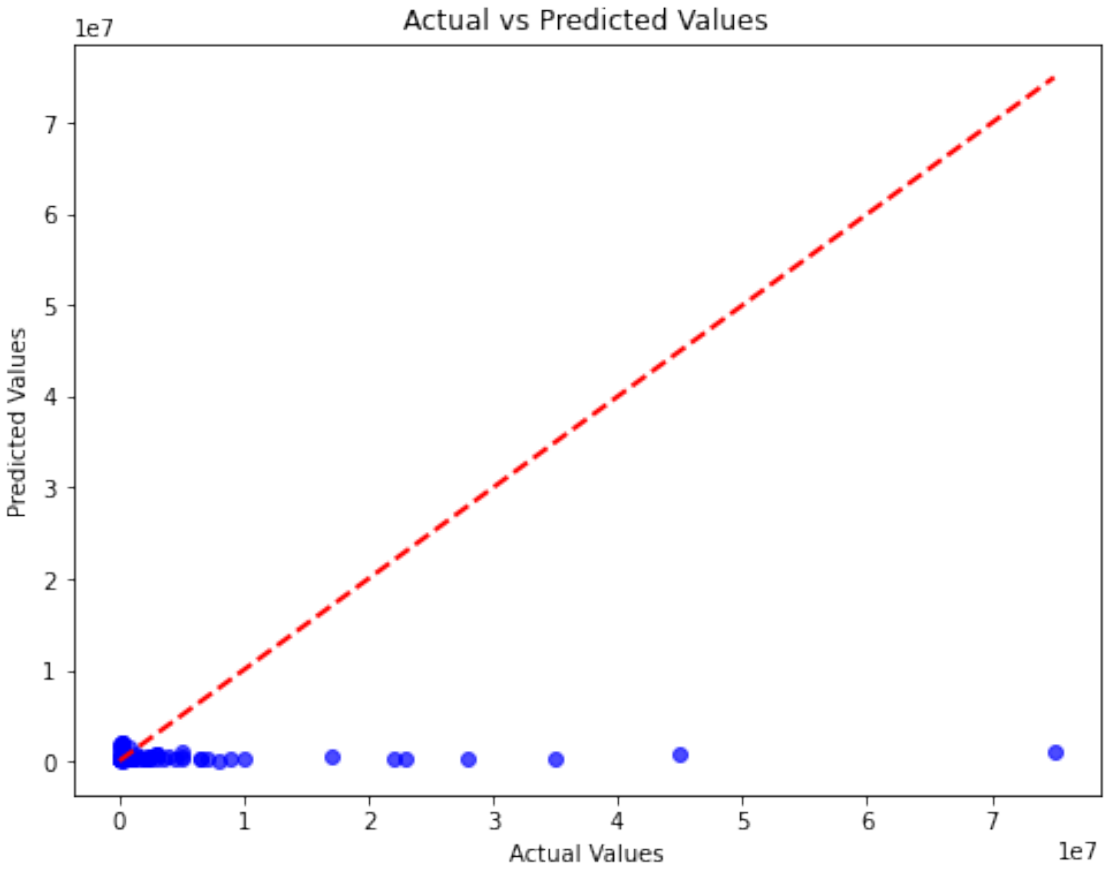


[115]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.feature_selection import SelectKBest, f_regression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt
from sklearn.decomposition import TruncatedSVD

# Assume X, y, and other imports are defined

# Define features and target variable
features = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
target_variable = 'market_value_in_eur'

# Subset the data
X_train_subset, X_test_subset, y_train, y_test = train_test_split(X[features],␣
 ↪y, test_size=0.2, random_state=42)

# One-hot encode categorical variables
categorical_columns = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
X_train_encoded = pd.get_dummies(X_train_subset, columns=categorical_columns)
X_test_encoded = pd.get_dummies(X_test_subset, columns=categorical_columns)

# Ensure that the same columns are present in both X_train_encoded and␣
 ↪X_test_encoded
common_columns = list(set(X_train_encoded.columns) & set(X_test_encoded.
 ↪columns))
X_train_encoded = X_train_encoded[common_columns]
X_test_encoded = X_test_encoded[common_columns]

# Introduce polynomial features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_encoded)
X_test_scaled = scaler.transform(X_test_encoded)
```

```
poly = PolynomialFeatures(degree=2)  # You can adjust the degree as needed
X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)

# Debugging print statements
print("Shapes before feature selection:")
print("X_train_poly shape:", X_train_poly.shape)
print("y_train shape:", y_train.shape)

# Dimensionality reduction using TruncatedSVD
svd = TruncatedSVD(n_components=100)  # Adjust the number of components
X_train_selected = svd.fit_transform(X_train_poly)
X_test_selected = svd.transform(X_test_poly)

# Handle NaN and infinite values in y_train
nan_cols_y_train = y_train.isna() | ~np.isfinite(y_train)

if nan_cols_y_train.any():
    print("NaN or infinite values in y_train. Handling them.")
    mean_y_train = np.nanmean(y_train)
    y_train = np.where(np.isfinite(y_train), y_train, mean_y_train)

# Support Vector Machine (SVM) Regression
svm_param_grid = {
    'C': [0.1, 1, 10],
    'epsilon': [0.1, 0.2, 0.5],
    'kernel': ['linear', 'rbf']
}

svm_grid_search = GridSearchCV(SVR(), svm_param_grid, cv=5,␣
 ↪scoring='neg_mean_squared_error')
svm_grid_search.fit(X_train_selected, y_train)

best_svm_params = svm_grid_search.best_params_

best_svm_model = SVR(**best_svm_params)
best_svm_model.fit(X_train_selected, y_train)

# Make predictions on the original test set
y_pred = best_svm_model.predict(X_test_selected)

# Evaluate model
mse = mean_squared_error(y_test, y_pred)
r_squared = r2_score(y_test, y_pred)

print("Mean Squared Error (MSE):", mse)
print("R-squared:", r_squared)
```

```
# Scatter plot of Actual vs Predicted values
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, color='blue', alpha=0.7)
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)],␣
 ↪linestyle='--', color='red', linewidth=2)
plt.title('Actual vs Predicted Values')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.show()
```

```
Shapes before feature selection:
X_train_poly shape: (723, 312445)
y_train shape: (723,)
Mean Squared Error (MSE): 62060414906978.41
R-squared: -0.05027332371509052
```



[116]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor,␣
 ↪StackingRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.svm import SVR  # Import SVR

# Assume X, y, and other imports are defined

# Define features and target variable
features = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
target_variable = 'market_value_in_eur'

# Subset the data
X_train_subset, X_test_subset, y_train, y_test = train_test_split(X[features],␣
 ↪y, test_size=0.2, random_state=42)

# One-hot encode categorical variables
categorical_columns = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
X_train_encoded = pd.get_dummies(X_train_subset, columns=categorical_columns)
X_test_encoded = pd.get_dummies(X_test_subset, columns=categorical_columns)

# Ensure that the same columns are present in both X_train_encoded and␣
 ↪X_test_encoded
common_columns = list(set(X_train_encoded.columns) & set(X_test_encoded.
 ↪columns))
X_train_encoded = X_train_encoded[common_columns]
X_test_encoded = X_test_encoded[common_columns]

# Feature scaling for all features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_encoded)
X_test_scaled = scaler.transform(X_test_encoded)

# Different model (Random Forest)
rf_model = RandomForestRegressor()
rf_model.fit(X_train_scaled, y_train)  # Use the scaled features for training
```

```
# Another model (Gradient Boosting)
gb_model = GradientBoostingRegressor()
gb_model.fit(X_train_scaled, y_train)  # Use the scaled features for training

# Stacking models
estimators = [('rf', rf_model), ('gb', gb_model)]
stacked_model = StackingRegressor(estimators=estimators, final_estimator=SVR())
stacked_model.fit(X_train_scaled, y_train)  # Use the scaled features for␣
 ↪training

# Make predictions on the original test set
y_pred_stacked = stacked_model.predict(X_test_scaled)  # Use the scaled␣
 ↪features for prediction

# Evaluate the stacked model
mse_stacked = mean_squared_error(y_test, y_pred_stacked)
r_squared_stacked = r2_score(y_test, y_pred_stacked)

print("Mean Squared Error (MSE) Stacked:", mse_stacked)
print("R-squared Stacked:", r_squared_stacked)
```

```
Mean Squared Error (MSE) Stacked: 62874445722897.92
R-squared Stacked: -0.06404949411169159
```

[117]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, StackingRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.svm import SVR  # Import SVR

# Assume X, y, and other imports are defined

# Define features and target variable
features = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
target_variable = 'market_value_in_eur'

# Subset the data
X_train_subset, X_test_subset, y_train, y_test = train_test_split(X[features],␣
 ↪y, test_size=0.2, random_state=42)
```

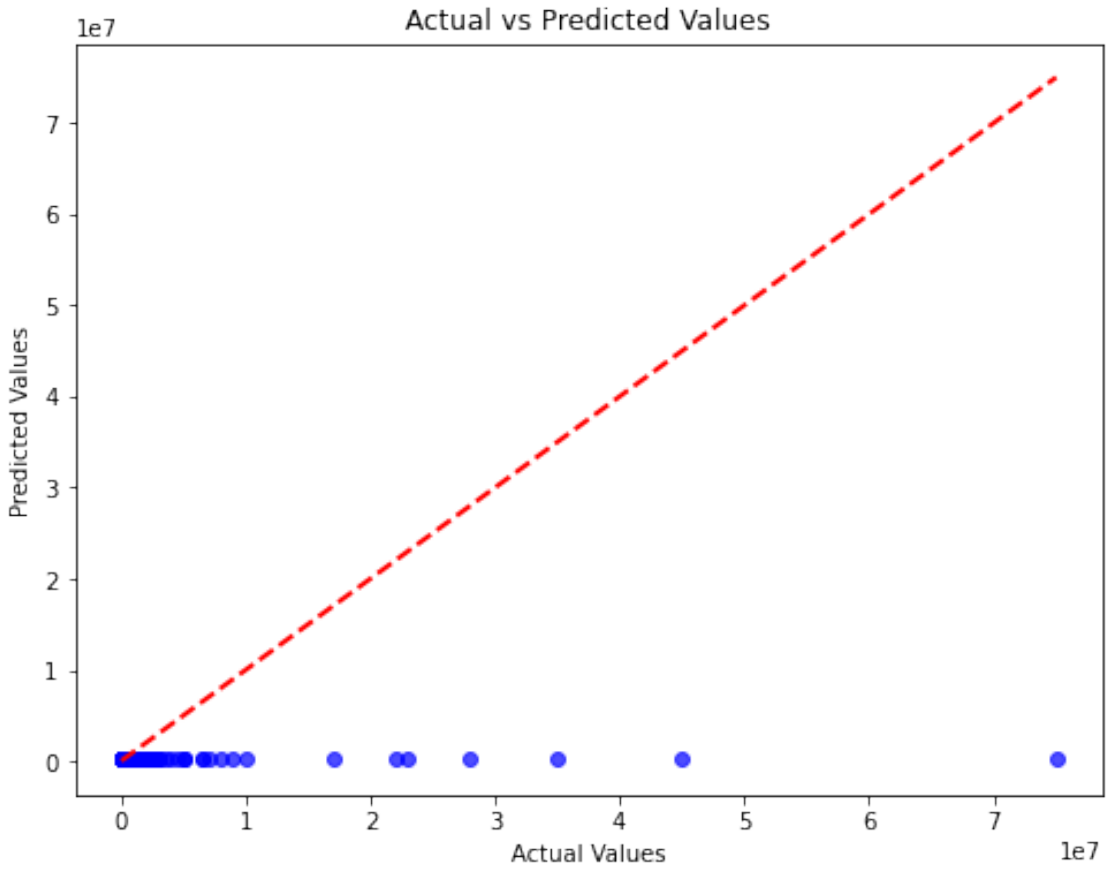
```
# One-hot encode categorical variables
categorical_columns = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
X_train_encoded = pd.get_dummies(X_train_subset, columns=categorical_columns)
X_test_encoded = pd.get_dummies(X_test_subset, columns=categorical_columns)

# Ensure that the same columns are present in both X_train_encoded and␣
 ↪X_test_encoded
common_columns = list(set(X_train_encoded.columns) & set(X_test_encoded.
 ↪columns))
X_train_encoded = X_train_encoded[common_columns]
X_test_encoded = X_test_encoded[common_columns]

# Feature scaling for all features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_encoded)
X_test_scaled = scaler.transform(X_test_encoded)

# Different model (Random Forest)
rf_model = RandomForestRegressor()
rf_model.fit(X_train_scaled, y_train)  # Use the scaled features for training

# Stacking models (Random Forest and SVR)
estimators = [('rf', rf_model), ('svr', SVR())]  # Add SVR to the stacking model
stacked_model = StackingRegressor(estimators=estimators, final_estimator=SVR())
stacked_model.fit(X_train_scaled, y_train)  # Use the scaled features for␣
 ↪training

# Make predictions on the original test set
y_pred_stacked = stacked_model.predict(X_test_scaled)  # Use the scaled␣
 ↪features for prediction

# Evaluate the stacked model
mse_stacked = mean_squared_error(y_test, y_pred_stacked)
r_squared_stacked = r2_score(y_test, y_pred_stacked)

print("Mean Squared Error (MSE) Stacked:", mse_stacked)
print("R-squared Stacked:", r_squared_stacked)
```

```
Mean Squared Error (MSE) Stacked: 62874442203634.31
R-squared Stacked: -0.06404943455378032
```

[118]:
```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.feature_selection import SelectKBest, f_regression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# Assume X, y, and other imports are defined

# Define features and target variable
features = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
target_variable = 'market_value_in_eur'

# Subset the data
X_train_subset, X_test_subset, y_train, y_test = train_test_split(X[features],␣
 ↪y, test_size=0.2, random_state=42)

# One-hot encode categorical variables
categorical_columns = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
X_train_encoded = pd.get_dummies(X_train_subset, columns=categorical_columns)
X_test_encoded = pd.get_dummies(X_test_subset, columns=categorical_columns)

# Ensure that the same columns are present in both X_train_encoded and␣
 ↪X_test_encoded
common_columns = set(X_train_encoded.columns) & set(X_test_encoded.columns)
X_train_encoded = X_train_encoded[common_columns]
X_test_encoded = X_test_encoded[common_columns]

# Feature scaling for all features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_encoded)
X_test_scaled = scaler.transform(X_test_encoded)

# Introduce polynomial features
poly = PolynomialFeatures(degree=2)  # You can adjust the degree as needed
X_train_poly = poly.fit_transform(X_train_scaled)
```

```
X_test_poly = poly.transform(X_test_scaled)

# Feature selection using SelectKBest with f_regression
selector = SelectKBest(f_regression, k=10)  # Adjust k as needed
X_train_selected = selector.fit_transform(X_train_poly, y_train)
X_test_selected = selector.transform(X_test_poly)

# Support Vector Machine (SVM) Regression
svm_param_grid = {
    'C': [0.1, 1, 10],
    'epsilon': [0.1, 0.2, 0.5],
    'kernel': ['linear', 'rbf']
}

svm_grid_search = GridSearchCV(SVR(), svm_param_grid, cv=5,␣
 ↪scoring='neg_mean_squared_error')
svm_grid_search.fit(X_train_selected, y_train)

best_svm_params = svm_grid_search.best_params_

best_svm_model = SVR(**best_svm_params)
best_svm_model.fit(X_train_selected, y_train)

# Make predictions on the original test set
y_pred = best_svm_model.predict(X_test_selected)

# Evaluate model
mse = mean_squared_error(y_test, y_pred)
r_squared = r2_score(y_test, y_pred)

print("Mean Squared Error (MSE):", mse)
print("R-squared:", r_squared)

# Scatter plot of Actual vs Predicted values
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, color='blue', alpha=0.7)
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)],␣
 ↪linestyle='--', color='red', linewidth=2)
plt.title('Actual vs Predicted Values')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.show()
```

```
Mean Squared Error (MSE): 62874995699507.14
R-squared: -0.06405880158670918
```



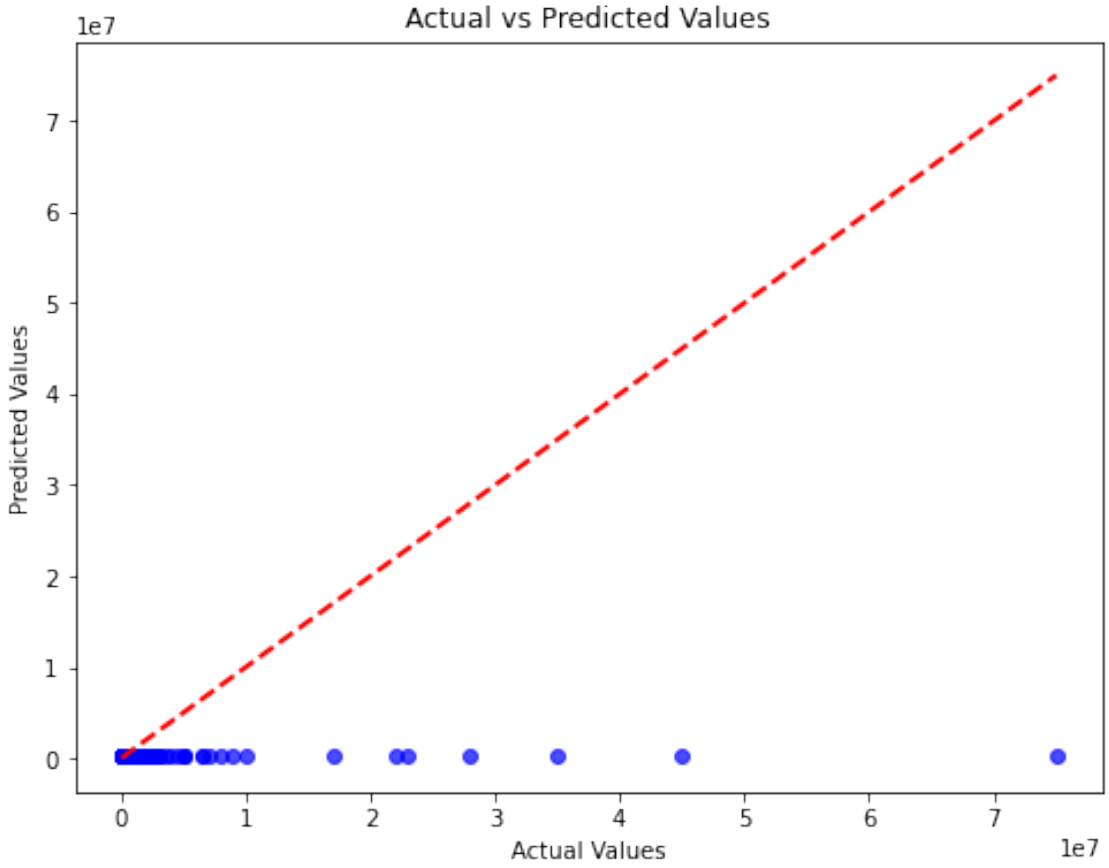
[119]:
```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.feature_selection import SelectKBest, f_regression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# Assume X, y, and other imports are defined

# Define features and target variable
features = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
target_variable = 'market_value_in_eur'
```

```
# Subset the data
X_train_subset, X_test_subset, y_train, y_test = train_test_split(X[features],␣
 ↪y, test_size=0.2, random_state=42)

# One-hot encode categorical variables
categorical_columns = ['name', 'current_club_name', 'country_of_citizenship',␣
 ↪'country_of_birth', 'position', 'sub_position', 'foot', 'agent_name',␣
 ↪'current_club_domestic_competition_id', 'first_name', 'last_name',␣
 ↪'player_code', 'image_url', 'url', 'Name', 'Photo', 'Nationality', 'Flag',␣
 ↪'Club', 'Club Logo', 'Preferred Foot', 'Work Rate', 'Body Type', 'Real␣
 ↪Face', 'Position', 'Loaned From', 'Height', 'Weight', 'Release Clause']
X_train_encoded = pd.get_dummies(X_train_subset, columns=categorical_columns)
X_test_encoded = pd.get_dummies(X_test_subset, columns=categorical_columns)

# Ensure that the same columns are present in both X_train_encoded and␣
 ↪X_test_encoded
common_columns = list(set(X_train_encoded.columns) & set(X_test_encoded.
 ↪columns))
X_train_encoded = X_train_encoded[common_columns]
X_test_encoded = X_test_encoded[common_columns]

# Handle NaN and infinite values in X_train_poly and y_train
nan_cols_X_train_poly = np.isnan(X_train_poly).any()
nan_cols_y_train = y_train.isna() | ~np.isfinite(y_train)

if nan_cols_X_train_poly.any():
    print("NaN or infinite values in X_train_poly. Handling them.")
    X_train_poly = np.nan_to_num(X_train_poly)

if nan_cols_y_train.any():
    print("NaN or infinite values in y_train. Handling them.")
    X_train_poly = X_train_poly[~nan_cols_y_train]
    y_train = y_train[~nan_cols_y_train]

# Feature scaling for all features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_encoded)
X_test_scaled = scaler.transform(X_test_encoded)

# Introduce polynomial features
poly = PolynomialFeatures(degree=2)  # You can adjust the degree as needed
X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)

# Debugging print statements
print("Shapes before feature selection:")
print("X_train_poly shape:", X_train_poly.shape)
```

```
print("y_train shape:", y_train.shape)

# Feature selection using SelectKBest with f_regression
selector = SelectKBest(f_regression, k=10)  # Adjust k as needed
X_train_selected = selector.fit_transform(X_train_poly, y_train)
X_test_selected = selector.transform(X_test_poly)

# Debugging print statements
print("Shapes after feature selection:")
print("X_train_selected shape:", X_train_selected.shape)
print("X_test_selected shape:", X_test_selected.shape)


# Support Vector Machine (SVM) Regression
svm_param_grid = {
    'C': [0.1, 1, 10],
    'epsilon': [0.1, 0.2, 0.5],
    'kernel': ['linear', 'rbf']
}

svm_grid_search = GridSearchCV(SVR(), svm_param_grid, cv=5,␣
 ↪scoring='neg_mean_squared_error')
svm_grid_search.fit(X_train_selected, y_train)

best_svm_params = svm_grid_search.best_params_

best_svm_model = SVR(**best_svm_params)
best_svm_model.fit(X_train_selected, y_train)

# Make predictions on the original test set
y_pred = best_svm_model.predict(X_test_selected)

# Evaluate model
mse = mean_squared_error(y_test, y_pred)
r_squared = r2_score(y_test, y_pred)

print("Mean Squared Error (MSE):", mse)
print("R-squared:", r_squared)

# Scatter plot of Actual vs Predicted values
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, color='blue', alpha=0.7)
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)],␣
 ↪linestyle='--', color='red', linewidth=2)
plt.title('Actual vs Predicted Values')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
```

```
plt.show()
```

```
Shapes before feature selection:
X_train_poly shape: (723, 312445)
y_train shape: (723,)
Shapes after feature selection:
X_train_selected shape: (723, 10)
X_test_selected shape: (181, 10)
Mean Squared Error (MSE): 62874995699507.14
R-squared: -0.06405880158670918
```



[ ]: